KU LEUVEN

FACULTY OF SCIENCE

Maximising Edge Density in Graphs Subject to Connectivity Constraints

Erdős Problem 915, from Proof to Search

Louis Vandenbruwaene

Promotor: Dr. Stijn Cambie
Readers: Prof. Jan Goedgebeur
Prof. Karel Dekimpe
KU Leuven

Thesis presented in
fulfilment of the requirements
for the degree of Master of Science
in Mathematics

Academic year 2026–2027

Copyright information

Student paper as part of an academic education and examination. No correction was made to the paper after examination.

© Copyright by KU Leuven

Without prior written permission from both the supervisor(s) and the author(s), copying, reproducing, using, or realizing this publication or parts thereof is prohibited. For requests or information regarding the copying and/or use and/or realization of parts of this publication, please contact KU Leuven, Faculty of Science, Celestijnenlaan 200H box 2100, 3001 Leuven (Heverlee), Telephone +32 16 32 14 01.

Prior written permission from the supervisor(s) is also required for using the (original) methods, products, circuits, and programs described in this work for industrial or commercial purposes, and for submitting this publication for scientific awards or contests.

Preface

First and foremost I would like to thank my girlfriend for her unwavering love and support. I could not have done this without her. I also want to express my gratitude for my family and friends who supported me throughout this journey. A person is pulled towards the average of the neighbours in the network of their connections.

This thesis asks how many edges a network can have when no two vertices may be joined by too many independent routes. The starting point is Erdős Problem 915, which asks this for edge-disjoint paths in simple graphs. After solving the assigned problem, I widened the scope towards more general networks and other kinds of disjoint routes, sixteen problems in total. After almost falling down the rabbit hole of asking the question “what if I do this instead?”, I answered the same question for the most common assumptions. If more exotic choices are what you seek, this thesis still contains the building blocks to craft the answers yourself alongside the instruction manual.

Chapter 1 states the problem and its variants, proves the main results, and lists the open problems. The definitions are in its first three sections, and the notation is listed under Abbreviations and Symbols. A reader who wants only the results can read these sections and then go to Section 1.5, where a table and a figure give what is known about each of the sixteen problems, followed by the open problems. To keep everything concise and engaging, the more tedious proofs were collected in Appendix A.

Chapter 2 explains how a Python program can measure connectivity and search for dense examples, using pruning and heuristics to make the search faster. No proof uses the program, so a reader interested only in the mathematics can skip this chapter. It needs only the definitions of Chapter 1. Appendix B prints the parts of the program that Chapter 2 leaves out, and the output of two full runs, one with each heuristic search.

Contribution Statement

I used artificial intelligence to write code, to suggest proofs, to help check them, to format LaTeX and to check spelling. Results from the literature are cited where they are used. The code is available in my personal repository [1].

Area	Main results
Undirected graphs and multigraphs	A different proof of Mader's bound, with the Gomory–Hu tree, an explicit construction of graphs that attain it for every $n \geq m$ (the value itself is due to Mader), and the exact multigraph edge value.
Multigraphs, vertex	The exact value at $m = 2$ and $m = 3$.
Hypergraphs, edge	The exact multihypergraph value for every $n \geq r$, and a range where simple hypergraphs attain it.
Hypergraphs, vertex	The exact value at $m = 2$ and $m = 3$, for simple hypergraphs at $m = 3$ when $2 < r < n$.
Simple directed graphs, $m = 2$	The exact value, for arc-disjoint and for internally vertex-disjoint routes.
Directed multigraphs	With arc-disjoint routes, the exact value for every n and m, and the extremal multigraphs for $n \geq \max(8, 2m)$. With internally vertex-disjoint routes, the exact value at $m = 2$ and the leading term for fixed m.
Simple directed graphs and directed hypergraphs	Upper bounds with the right leading term, $n^2/4$ for simple directed graphs and $(m-1)n^2/(4(r-1))$ for directed hypergraphs in the three orientation models, valid for every n.

Short Summary

Erdős Problem 915 asks how many edges a simple graph on n vertices can have if no pair is joined by m edge-disjoint paths [2, 3]. Writing the maximum as $\ell_m(n)$, Mader's bound is

$$\ell_m(n) \leq \left\lfloor \frac{m(n-1)}{2} \right\rfloor .$$

Bollobás and Erdős showed that the bound is attained when $m-1 \mid n-1$, and Mader showed it for every $n \geq m$. I prove the bound in a different way, with the Gomory–Hu tree, and give an explicit construction of graphs that reach it for every $n \geq m \geq 2$.

The thesis also studies the problem in eight models. Edges may be directed or not, may be repeated or not, and may join two vertices or more (hypergraphs). In each model the routes must be edge-disjoint, or separated, which means that they share no inner vertex and no edge. In graphs, separated routes are the internally vertex-disjoint routes. This gives sixteen problems.

The proved results include:

- Multigraphs: exactly $(m-1)(n-1)$ edges. With internally vertex-disjoint routes, exactly $n-1$ at $m=2$ and $2(n-1)$ at $m=3$.
- r-uniform multihypergraphs: exactly $\lfloor (m-1)(n-1)/(r-1) \rfloor$ hyperedges for every $n \geq r$. Simple hypergraphs attain the same value when $m-1 \leq \binom{n-2}{r-2}$.
- Hypergraphs with separated routes: exactly $\lfloor (n-1)/(r-1) \rfloor$ at $m=2$, and exactly $\lfloor 2(n-1)/(r-1) \rfloor$ at $m=3$, for simple hypergraphs when $2 < r < n$.
- Simple directed graphs at $m=2$: exactly $\max\bigl(2(n-1), \lfloor n^2/4 \rfloor\bigr)$ arcs, for arc-disjoint and for internally vertex-disjoint routes.
- Directed multigraphs with arc-disjoint routes: exactly $(m-1)\max\bigl(2(n-1), \lfloor n^2/4 \rfloor\bigr)$ arcs. With internally vertex-disjoint routes the value is $(m-1)n^2/4 + O_m(n)$ for fixed m.
- Simple directed graphs: $n^2/4 + \Theta_m(n)$ arcs for fixed $m \geq 3$, for arc-disjoint and for internally vertex-disjoint routes.
- Directed hypergraphs: $(1+o(1))(m-1)n^2/(4(r-1))$ hyperedges for fixed m and r, with $r \geq 3$ for simple directed hypergraphs, in the three orientation models.

Chapter 2 explains how a program measures connectivity with maximum flow and searches for dense examples.

Abbreviations and Symbols

Symbol	Meaning
$G = (V, E)$, $D = (V, A)$	vertex set V, with edge multiset E or arc multiset A
n, m, r	vertex count, number of edge-disjoint or separated routes that no pair may have, hyperedge size
$\lambda_G(u, v)$	maximum number of edge-disjoint u–v paths
$\kappa_G(u, v)$	maximum number of separated u–v paths
$\lambda_G^{\max}$, $\kappa_G^{\max}$	maximum local connectivity over distinct vertex pairs, ordered in directed models
$\ell_m(n)$, $L_m(n)$	simple and multigraph undirected edge maxima
$k_m(n)$, $K_m(n)$	simple and multigraph undirected vertex maxima
$\ell_m^{\mathrm{dir}}(n)$, $L_m^{\mathrm{dir}}(n)$	simple and multigraph directed arc maxima
$k_m^{\mathrm{dir}}(n)$, $K_m^{\mathrm{dir}}(n)$	simple and multigraph directed vertex maxima
$\ell_m^{(r)}(n)$, $L_m^{(r)}(n)$	simple and multi r-uniform hypergraph edge maxima
$k_m^{(r)}(n)$, $K_m^{(r)}(n)$	simple and multi r-uniform hypergraph vertex maxima
$M(n)$	$\max(2(n-1), \lfloor n^2/4 \rfloor)$, the directed arc maximum at $m = 2$
$\mu(u, v)$	edge or arc multiplicity
$O(f)$, $\Theta(f)$	at most, or between two positive constant multiples of, f for large n. A subscript such as Θ_m means the constants depend on m
$o(1)$	a quantity that tends to 0

Contents

Preface I

Contribution Statement II

Short Summary III

Abbreviations and Symbols IV

1 The Problem and Its Results 1

1.1 Graphs and routes . . . 1

1.2 Erdős Problem 915 . . . 2

1.2.1 Proof of Mader's bound . . . 3

1.3 Variants of the problem . . . 5

1.3.1 The incidence convention . . . 7

1.4 Results and proofs . . . 8

1.4.1 Undirected graphs . . . 8

1.4.2 Directed graphs at the first threshold . . . 9

1.4.3 Undirected hypergraphs . . . 13

1.4.4 Directed bounds beyond the first threshold . . . 18

1.5 Overview and open problems . . . 20

2 Measuring and Searching by Machine 22

2.1 Modelling . . . 22

2.2 Measuring . . . 24

2.2.1 Transformations . . . 25

2.3 Checking every graph with a fixed number of vertices . . . 26

2.3.1 Pruned search . . . 29

2.3.2 Checking one edge more . . . 30

2.4 The heuristic search . . . 35

2.4.1 Tabu search . . . 35

2.4.2 Random greedy growth . . . 37

2.4.3 Measuring only when needed . . . 37

2.5 Checking the output without trusting the program . . . 38

2.6 Running the program . . . 40
2.7 Comparing the two searches . . . 42

A Proofs **43**

PART I. UNDIRECTED GRAPHS . . . 44
A.1 Graphs that attain Mader's bound . . . 44
A.1.1 The vertex problem is a block problem . . . 46
PART II. DIRECTED GRAPHS . . . 49
A.2 The directed constructions . . . 49
A.3 Simple directed graphs beyond the first threshold . . . 51
A.3.1 The two-step budget . . . 51
A.4 The extremal directed multigraphs . . . 54
A.4.1 The extremal multigraphs for large n . . . 55
PART III. HYPERGRAPHS . . . 57
A.5 The hypergraph bounds . . . 57
A.6 Lower bounds for the hypergraph vertex problem . . . 60
A.7 Multigraphs with internally vertex-disjoint routes . . . 61
A.7.1 The value is a block problem . . . 65
A.7.2 The directed problem . . . 67
A.8 The directed hypergraph . . . 69

B The Program **74**

B.1 The remaining parts of the program . . . 74
B.2 Output . . . 78
B.3 Output with random greedy growth . . . 81

Chapter 1

The Problem and Its Results

This chapter defines the problem and its variants, and states the results that are proved.

1.1 Graphs and routes

A *multigraph* G consists of a finite set $V(G)$ of *vertices* and a multiset $E(G)$ of *edges*. Each edge joins two different vertices, and edges can have identical copies. The number of such identical edges between u and v is the multiplicity $\mu(u, v)$. If $\mu(u, v) \leq 1$ for every pair, then G is a *simple graph*. The *size* of G is its number of edges, counted with multiplicity: $|E(G)| = \sum_{\{u,v\}} \mu(u, v)$. This thesis tries to maximise the size. Drawing vertices as dots and edges as lines between them is the standard way to visualise graphs. A multigraph is then any network built by joining different points with lines. Some definitions also allow loops, edges that join a vertex to itself. This thesis excludes them. A route cannot revisit a vertex, so it never uses a loop, and loops leave every connectivity unchanged. They would still count towards the size, and if a vertex may carry any number of loops, the size has no maximum.

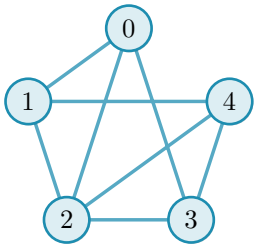


Figure 1.1. The simple graph $K_5 - \{04, 13\}$, the complete graph K_5 on five vertices with the edges 04 and 13 removed, has eight edges. In general K_n is the complete graph on n vertices.

We write $n := |V(G)|$. The *degree* of a vertex is the number of edges at it. From now on, graphs are simple unless we say otherwise. A *path* from u to v is a sequence $v_0, e_1, v_1, \ldots, e_q, v_q$ of distinct vertices v_i and edges e_i, where $v_0 = u$, $v_q = v$ and e_i joins v_{i-1} and v_i. Its *length* is its number of edges q. In a multigraph, two parallel edges give two different paths of length one. In a simple graph we can write a path as a list of vertices. A *walk* is a sequence of the same form in which vertices and edges may repeat. A *cycle* is a sequence $v_0, e_1, v_1, \ldots, e_q, v_q$ with $q \geq 2$, distinct edges e_i and distinct vertices $v_0, \ldots, v_{q-1}$, where $v_q = v_0$ and e_i joins v_{i-1} and v_i. In a simple graph a cycle has at least three edges, and in a multigraph two parallel edges form a cycle. A graph is *connected* if every two vertices are joined by a path. A *tree* is a connected graph in which every two vertices are joined by exactly one path. A tree on n vertices has $n - 1$ edges [4]. A *forest* is a graph without cycles. A *subgraph* of a graph G is a graph whose vertices and edges

all belong to G. A *component* of a graph is a maximal connected subgraph. A *spanning tree* of G is a subgraph of G that is a tree on all vertices of G. We also call a path a *route*. Two u–v paths are *edge-disjoint* if they have no edge in common. The *local edge-connectivity* $\lambda_G(u, v)$ is the largest number of pairwise edge-disjoint u–v paths. Its largest value over all pairs is

$$\lambda_G^{\max} = \max_{u \neq v} \lambda_G(u, v).$$

If G has fewer than two vertices, we set $\lambda_G^{\max} = 0$.

Theorem 1.1 (Menger [5, 6])**.** *Let u and v be distinct vertices of a multigraph G. The largest number of pairwise edge-disjoint u–v paths equals the smallest number of edges whose removal leaves no u–v path.*

Figure 1.2 shows an example.

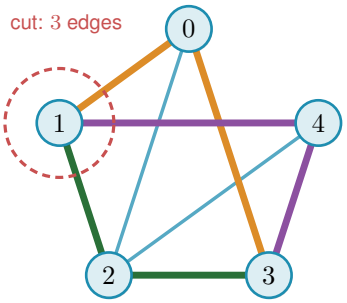


Figure 1.2. Menger's theorem on Figure 1.1: three edge-disjoint 1–3 routes (coloured) each use one of the three edges at vertex 1. Removing these three edges leaves no 1–3 route, so $\lambda(1, 3) = 3$.

1.2 Erdős Problem 915

Fix an integer $m \geq 2$. The condition $\lambda_G^{\max} \leq m - 1$ means that no two vertices are joined by m edge-disjoint paths. By Menger's theorem, it also means that any two vertices can be separated by removing at most $m - 1$ edges. We ask how many edges a graph can have under this condition.

Question 1.2 (Erdős Problem 915 [3]). How many edges can a graph on n vertices have if no two of its vertices are joined by m disjoint paths?

The database [3] notes that the original statement leaves open whether "disjoint" means that the paths share no edge, or that they share no vertex except their two ends. This thesis takes edge-disjoint paths as the main question and treats internally vertex-disjoint paths as a variant (Section 1.3).

We write

$$\ell_m(n) = \max\{\, |E(G)| : |V(G)| = n,\ \lambda_G^{\max} \leq m - 1 \,\}$$

for the answer. A graph with $\lambda_G^{\max} \leq m - 1$ is called *feasible*. Mader proved the following upper bound. Here $\lfloor x \rfloor$ is x rounded down and $\lceil x \rceil$ is x rounded up.

Theorem 1.3 (Mader [7]). *For all $n \geq 2$ and $m \geq 2$,*

$$\ell_m(n) \leq \left\lfloor \frac{m(n-1)}{2} \right\rfloor.$$

Bollobás and Erdős [2] found graphs that attain this bound when $m-1$ divides $n-1$: copies of K_m that share one vertex. For $n \geq m$ the bound is always attained. Mader proved this as well, so $\ell_m(n) = \lfloor m(n-1)/2 \rfloor$ for $n \geq m$ [7]. Theorem A.2 gives a different construction. For $2 \leq n < m$ the complete graph K_n satisfies the condition, so $\ell_m(n) = \binom{n}{2}$. This is smaller than $\lfloor m(n-1)/2 \rfloor$, except when $(n, m) = (2, 3)$. The reason is that $m \geq n+1$ gives $\lfloor m(n-1)/2 \rfloor \geq \binom{n}{2} + \lfloor (n-1)/2 \rfloor$, which exceeds $\binom{n}{2}$ once $n \geq 3$. At $n = 2$ the two values are 1 and $\lfloor m/2 \rfloor$, and these agree only at $m = 3$.

I prove Theorem 1.3 in a different way from Mader, with the Gomory–Hu tree. It stores the $\binom{n}{2}$ values $\lambda_G(u, v)$ in only $n-1$ weights.

Theorem 1.4 (Gomory–Hu [8]). *Let G be a multigraph on the vertex set V. There is a tree T on V, with a weight $c(t)$ on each tree edge t, such that:*

(1) Removing t from T splits V into two parts S and $V \setminus S$, and $c(t)$ is the number of edges of G with one end in S and one end in $V \setminus S$.

(2) For distinct u, v, the value $\lambda_G(u, v)$ is the smallest weight on the u–v path of T.

Such a tree is a *Gomory–Hu tree* of G. Its edges need not be edges of G. If $t = xy$ is a tree edge, then the x–y path of T is t itself, so $c(t) = \lambda_G(x, y)$ by (2). This thesis uses the theorem only for undirected graphs.

1.2.1 Proof of Mader's bound

Let T be a Gomory–Hu tree of G. For an edge e of G, let $\mathrm{dist}_T(e)$ be the number of tree edges on the path in T between the ends of e (Figure 1.3). An edge $e = uv$ of G *crosses* a tree edge t if u and v lie in different parts when t is removed from T. This happens exactly when t lies on the u–v path of T. If t lies on this path, removing t breaks the only u–v path of T, which puts u and v in different parts. If t does not lie on it, the path survives the removal and keeps u and v together. Hence e crosses $\mathrm{dist}_T(e)$ tree edges.

The proof has three steps.

(1) The tree weights add up to at most $(m-1)(n-1)$.

(2) The same sum counts every edge of G once for each tree edge it crosses (Lemma 1.5).

(3) In a simple graph, at most $n-1$ edges cross only one tree edge, and every other edge crosses at least two (Lemma 1.6).

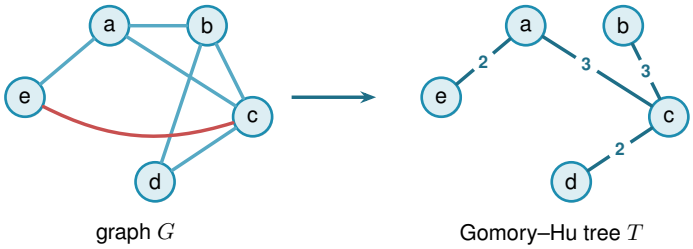


Figure 1.3. A graph G and a Gomory–Hu tree T on the same vertices. Each edge e of G crosses $\mathrm{dist}_T(e)$ tree edges. In a simple graph, at most $n-1$ edges have $\mathrm{dist}_T(e) = 1$.

Lemma 1.5 (Double-counting identity)**.** *Let G be a multigraph and T a Gomory–Hu tree of G with weights c. Then*

$$\sum_{t \in E(T)} c(t) \;=\; \sum_{e \in E(G)} \mathrm{dist}_T(e).$$

Parallel edges are counted separately in the right-hand sum.

Proof. Count the pairs (t, e) where t is a tree edge, e is an edge of G, and e crosses t. For a fixed t, exactly $c(t)$ edges cross t, by property (1) of the Gomory–Hu theorem. For a fixed e, exactly $\mathrm{dist}_T(e)$ tree edges are crossed by e. Both sums therefore count the same pairs. □

Lemma 1.6 (Sum of tree distances)**.** *For a simple graph G,* $\displaystyle\sum_{e \in E(G)} \mathrm{dist}_T(e) \;\geq\; 2|E(G)| - (n-1).$

Proof. Let E_1 be the set of edges with $\mathrm{dist}_T(e) = 1$. Such an edge joins two vertices that are adjacent in T. The tree T has $n-1$ edges, and since G is simple, each of them gives at most one edge of E_1, which gives $|E_1| \leq n-1$. Every other edge has $\mathrm{dist}_T(e) \geq 2$. Its distance is not 0, because its two ends are different, and not 1, because it is not in E_1. This gives

$$\sum_{e \in E(G)} \mathrm{dist}_T(e) \;\geq\; |E_1| + 2\big(|E(G)| - |E_1|\big) \;=\; 2|E(G)| - |E_1| \;\geq\; 2|E(G)| - (n-1). \qquad \square$$

Proof of Theorem 1.3. Let G be a simple graph on n vertices with $\lambda_G^{\max} \leq m-1$, and let T be a Gomory–Hu tree of G. Each tree edge $t = xy$ has weight $c(t) = \lambda_G(x, y) \leq m-1$. There are $n-1$ tree edges, so $\sum_t c(t) \leq (m-1)(n-1)$. With Lemmas 1.5 and 1.6 this gives

$$2|E(G)| - (n-1) \;\leq\; \sum_{e \in E(G)} \mathrm{dist}_T(e) \;=\; \sum_{t \in E(T)} c(t) \;\leq\; (m-1)(n-1).$$

This simplifies to $2|E(G)| \leq m(n-1)$. Since $|E(G)|$ is an integer, $|E(G)| \leq \left\lfloor \frac{m(n-1)}{2} \right\rfloor$. □

The factor $\frac{1}{2}$ comes from step (3). In a multigraph step (3) fails, and the bound is $(m-1)(n-1)$ (Theorem 1.8).

1.3 Variants of the problem

We change four assumptions. The first three give eight graph models. In each model we can ask for edge-disjoint or for separated routes, defined in Section 1.3.1.

The first assumption is the size of an edge. A *hyperedge* is a set of $r \geq 2$ vertices. In an r*-uniform hypergraph* every hyperedge has exactly r vertices. A *Berge path* is a sequence of distinct vertices and distinct hyperedges, alternating, in which each hyperedge contains the vertex before it and the vertex after it. For $r = 2$ it is an ordinary path. The size of a hypergraph is its number of hyperedges, counted with multiplicity.

The second assumption is repetition. A *simple* graph or hypergraph has each edge at most once. A *multigraph* or *multihypergraph* may repeat edges.

The third assumption is direction. A directed edge, or *arc*, is an ordered pair of distinct vertices. A directed hyperedge splits its r vertices into a non-empty set of *tails* and a non-empty set of *heads*. A directed Berge route enters a hyperedge at a tail and leaves it at a head. The allowed numbers of tails are a parameter of the directed hypergraph models. In the *forward* model every hyperedge has one tail and $r - 1$ heads, in the *backward* model it has $r - 1$ tails and one head, and the *general* model allows every split. For $r = 2$ the three coincide, and a hyperedge is an arc. Unless a statement names one of them, the directed hypergraph models are forward. In the figures, one colour is one hyperedge, not a path of ordinary edges. Figure 1.4 shows the eight models.

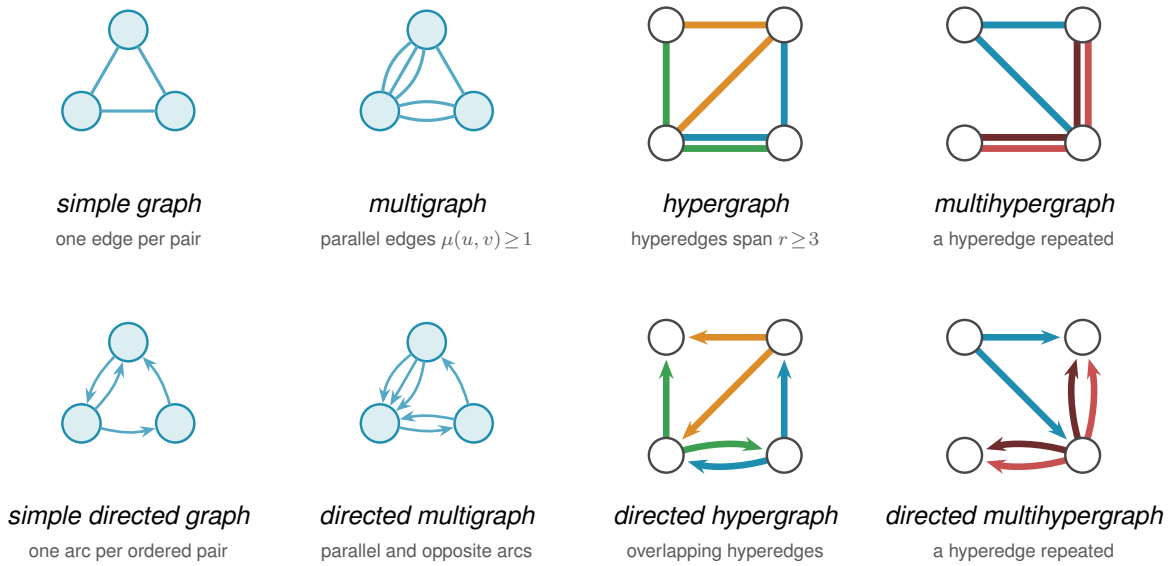


Figure 1.4. The eight models side by side, with the four undirected on top and the four directed below. The simple graph is a triangle, the multigraph has parallel edges, the hypergraph has three hyperedges, and the multihypergraph has one repeated hyperedge. The directed versions have arcs instead of edges, with the same pattern of repetition and hyperedges. In the two multihypergraph panels the two shades of red are the two copies of one hyperedge. Each directed hyperedge drawn has one tail.

Directed graphs have no Gomory–Hu tree in general. A tree stores one value per pair, read the same in both directions, while $\lambda(u, v)$ and $\lambda(v, u)$ can differ. Menger's theorem (Theorem 1.1)

does hold for directed multigraphs, with directed paths and arcs. In a directed graph, the *out-degree* $d^+(v)$ is the number of arcs that leave v. The vertices these arcs point to are the *out-neighbours* of v. The *in-degree* $d^-(v)$ is the number of arcs that enter v, and they come from the *in-neighbours* of v. The *total degree* is $d(v) = d^+(v) + d^-(v)$. Figure 1.5 shows three arc-disjoint routes from u to v. Removing three arcs cuts all of them, as Menger's theorem predicts. The out-degree of u and the in-degree of v also bound the number of routes:

Lemma 1.7 (Degree bound)**.** *Every pair u, v of distinct vertices of a graph G satisfies $\lambda_G(u, v) \le \min(\deg u, \deg v)$, and every ordered pair of a directed graph D satisfies $\lambda_D(u, v) \le \min(d^+(u), d^-(v))$.*

Proof. Every edge-disjoint u–v path uses a distinct edge at u, so there are at most $\deg u$ of them, and likewise at most $\deg v$. In the directed case each arc-disjoint path leaves u by a distinct out-arc and enters v by a distinct in-arc. □

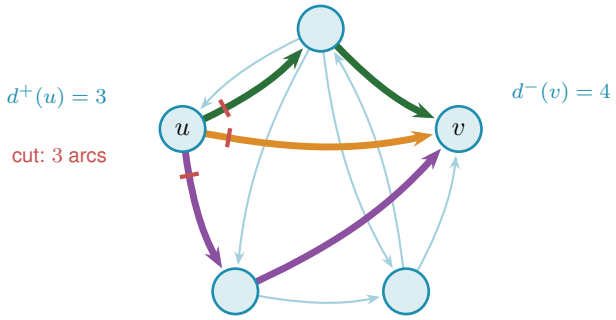


Figure 1.5. Three arc-disjoint $u \to v$ routes (coloured) and the corresponding three-arc directed cut (red) in a *simple* directed graph. Thus $\lambda(u, v) = 3$, meeting the degree bound $\min(d^+(u), d^-(v)) = 3$.

The fourth assumption is which routes count as disjoint. Two u–v paths are *internally vertex-disjoint* if they have no vertex in common except u and v. The largest number of such paths is the *local vertex-connectivity* $\kappa_G(u, v)$. Its largest value over all pairs is $\kappa_G^{\max}$. In a graph, internally vertex-disjoint paths are also edge-disjoint. An edge has two ends, and two such paths share no vertex other than u and v. A shared edge would therefore have to join u and v. A u–v path that uses such an edge consists of that edge alone, which makes two paths through the same edge the same path. This gives $\kappa_G(u, v) \le \lambda_G(u, v)$. Section 1.3.1 extends this to multigraphs and hypergraphs. Figure 1.6 shows that edge-disjoint and internally vertex-disjoint routes can differ.

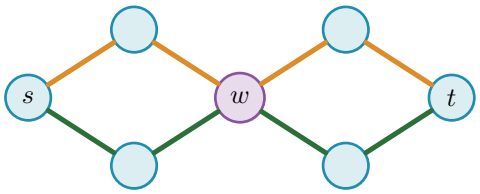


Figure 1.6. Edge-disjoint and internally vertex-disjoint routes differ: two edge-disjoint s–t routes both pass through w, so $\lambda(s, t) = 2$ but $\kappa(s, t) = 1$.

1.3.1 The incidence convention

For multigraphs and hypergraphs we must say when two routes share no inner vertex. This thesis uses the following convention. For simple graphs it gives the usual notion.

The *incidence graph* $I(\mathcal{H})$ of a multigraph or hypergraph $\mathcal{H}$ has a node for each vertex and a node for each edge copy. An edge copy is joined to the vertices it contains. A route from u to v is a path in $I(\mathcal{H})$. Two routes are *edge-disjoint* if they use no common edge copy. Such routes may still meet in inner vertices. Two routes are *separated* if they have no inner node in common, so they are both internally vertex-disjoint and edge-disjoint. For graphs and multigraphs, internally vertex-disjoint routes never share an edge copy, so there separated is the same as internally vertex-disjoint. As for graphs, $\lambda(u, v)$ is the largest number of pairwise edge-disjoint u–v routes, and $\kappa(u, v)$ the largest number of pairwise separated ones. In the directed case we use the directed incidence graph: each tail has an arc to the edge copy, and the edge copy has an arc to each head. For example, an arc $u \to v$ becomes $u \to e \to v$.

The inner nodes are vertices and edge copies, so separated routes share no inner vertex and no edge copy. Sharing no vertex is not enough for hypergraphs, because two Berge routes can use the same hyperedge without meeting in a vertex. Since separated routes also share no edge copy, $\kappa \leq \lambda$ holds in every model.

In a multigraph, q parallel copies of uv give q internally vertex-disjoint routes. This is why the vertex problems for multigraphs, written $K_m(n)$ and $K_m^{\mathrm{dir}}(n)$ in Table 1.1, differ from the simple ones. In a feasible multigraph no multiplicity is larger than $m - 1$, since m parallel copies already give m internally vertex-disjoint routes.

In every model, a graph is *feasible* if no two vertices are joined by m disjoint routes of the kind considered. Table 1.1 lists the notation for all sixteen problems.

Table 1.1. Notation for the extremal functions. The third column uses the incidence convention defined above, in every model.

Model	edge- or arc-disjoint routes	internally vertex-disjoint or separated routes
Simple graph	$\ell_m(n)$	$k_m(n)$
Multigraph	$L_m(n)$	$K_m(n)$
r-uniform simple hypergraph	$\ell_m^{(r)}(n)$	$k_m^{(r)}(n)$
r-uniform multihypergraph	$L_m^{(r)}(n)$	$K_m^{(r)}(n)$
Simple directed graph	$\ell_m^{\mathrm{dir}}(n)$	$k_m^{\mathrm{dir}}(n)$
Directed multigraph	$L_m^{\mathrm{dir}}(n)$	$K_m^{\mathrm{dir}}(n)$
Directed r-uniform simple hypergraph	$\ell_m^{\mathrm{dir}\,(r)}(n)$	$k_m^{\mathrm{dir}\,(r)}(n)$
Directed r-uniform multihypergraph	$L_m^{\mathrm{dir}\,(r)}(n)$	$K_m^{\mathrm{dir}\,(r)}(n)$

Menger's theorem, the Gomory–Hu tree, Mader's bound and the exact value $\ell_m(n) = \lfloor m(n-1)/2 \rfloor$ for $n \geq m$ are classical. The results of Bártfai, Bollobás, Leonard, Sørensen and Thomassen are classical as well. Table 1.2 lists the results of this thesis. It separates a new proof of a known theorem, a new construction for a known value, and a statement for which

Result	Where	Kind	Closest known result
Mader's bound $\ell_m(n) \leq \lfloor m(n-1)/2 \rfloor$	Section 1.2.1	New proof	Mader [7]
Graphs attaining Mader's bound for every $n \geq m$	Theorem A.2	New construction	Mader [7]. Bollobás–Erdős [2] when $(m-1) \mid (n-1)$
Multigraph edge value $L_m(n)$	Theorem 1.8	No earlier source found	Mader's bound for simple graphs
Multigraph vertex value at $m = 2, 3$	Theorem A.29 and Corollary A.31	No earlier source found	The simple graph values of Theorem 1.9
Exact multihypergraph edge value, and a range where simple hypergraphs attain it	Theorem 1.15	No earlier source found	Mader's bound for simple graphs
Hypergraph vertex value at $m = 2, 3$	Theorems 1.17 and 1.18	No earlier source found	The simple graph values of Theorem 1.9
Simple directed graph values at $m = 2$, for arc-disjoint and for internally vertex-disjoint routes	Theorems 1.11 and 1.12	No earlier source found	None found
Directed multigraph value with arc-disjoint routes, and its extremal multigraphs	Theorems A.19 and 1.22	No earlier source found	None found
Directed multigraph with internally vertex-disjoint routes: the value at $m = 2$ and the leading term	Corollary A.38	No earlier source found	None found
Simple directed graphs: at most $\lfloor n^2/4 \rfloor + (m-1)(n-1)$ arcs for every n	Theorem 1.21	Overlaps a known result	Huang–Lyu [9, Theorem 2] give the linear term $(m-1)n$ for n large in terms of m
Directed hypergraphs: the leading constant in three orientation models	Theorems 1.26 and A.46	No earlier source found	None found

Table 1.2. The results of this thesis, with the closest result I found in the literature.

I found no earlier source, and it names the closest result I know. I did not search the literature exhaustively, so the last kind means only that I found no earlier source.

1.4 Results and proofs

1.4.1 Undirected graphs

Theorem 1.8. *For $n \geq 1$ and $m \geq 2$, undirected multigraphs satisfy $L_m(n) = (m-1)(n-1)$.*

Proof of Theorem 1.8. For the lower bound, take any tree on the n vertices and give every edge multiplicity $m-1$. This multigraph has $(m-1)(n-1)$ edges. Two vertices u, v are joined by exactly one path in the tree. Removing the $m-1$ copies of one edge of this path separates u from v. Every pair therefore has $\lambda(u, v) \leq m-1$.

For the upper bound, let $\lambda_G^{\max} \le m-1$ and let T be a Gomory–Hu tree of G. Every edge of G crosses at least one tree edge, so by Lemma 1.5, $|E(G)| \le \sum_e \operatorname{dist}_T(e) = \sum_t c(t) \le (m-1)(n-1)$. □

In simple graphs, the edge and vertex problems have the same answer for $m \le 4$.

Theorem 1.9 (Bártfai [10], Bollobás [11], Leonard [12]). *For all $n \ge m$ and $2 \le m \le 4$,*

$$k_m(n) = \ell_m(n) = \left\lfloor \frac{m(n-1)}{2} \right\rfloor .$$

Since $\kappa \le \lambda$, every graph with $\lambda^{\max} \le m-1$ also has $\kappa^{\max} \le m-1$. Hence $k_m(n) \ge \ell_m(n)$ (Figure 1.7). Sørensen and Thomassen showed that the two differ for $m = 5$.

Theorem 1.10 (Sørensen–Thomassen [13]). *For simple graphs,*

$$k_5(n) = \left\lfloor \frac{5(n-1)}{2} \right\rfloor = \ell_5(n) \quad (5 \le n \le 13), \qquad k_5(n) = \left\lfloor \frac{8n}{3} \right\rfloor - 4 \quad (n \ge 14).$$

In particular $k_5(n) > \ell_5(n)$ for every $n \ge 14$. Sørensen and Thomassen count the least number of edges that forces m internally vertex-disjoint paths. This is one more than the maximum used here (Remark A.4). They give one formula with exceptions at $n = 7$ and $n = 12$. Splitting the statement at $n = 14$, as above, removes these exceptions.

1.4.2 Directed graphs at the first threshold

Theorem 1.11 (Directed arc value at $m = 2$). *For simple directed graphs,*

$$\ell_2^{\mathrm{dir}}(n) = \max\left(2(n-1),\ \left\lfloor \frac{n^2}{4} \right\rfloor\right).$$

Two constructions attain the two terms (Figure 1.8). The *bidirected star* joins one vertex to every other vertex by an arc in each direction, so it has $2(n-1)$ arcs. The one-way complete bipartite directed graph has all arcs from one half of the vertices to the other half, so it has $\lfloor n^2/4 \rfloor$ arcs. In both, every ordered pair is joined by at most one directed path. Section A.2 gives the details.

Theorem 1.12 (Directed vertex value at $m = 2$). *For simple directed graphs, $k_2^{\mathrm{dir}}(n) = \ell_2^{\mathrm{dir}}(n)$.*

The star has more arcs for $n < 7$, and the bipartite directed graph has more for $n > 7$. The directed value thus grows quadratically in n, while the undirected value grows linearly. In the bipartite directed graph no arc leaves B and no arc enters A. Every route is therefore a single arc, and each ordered pair has at most one route. An undirected graph with $\lambda^{\max} \le 1$ has no cycle, because two vertices on a cycle are joined by the two halves of the cycle, which are edge-disjoint paths. Such a graph is a forest and has at most $n-1$ edges.

For the upper bound we keep only the arcs that are needed to tell who can reach whom.

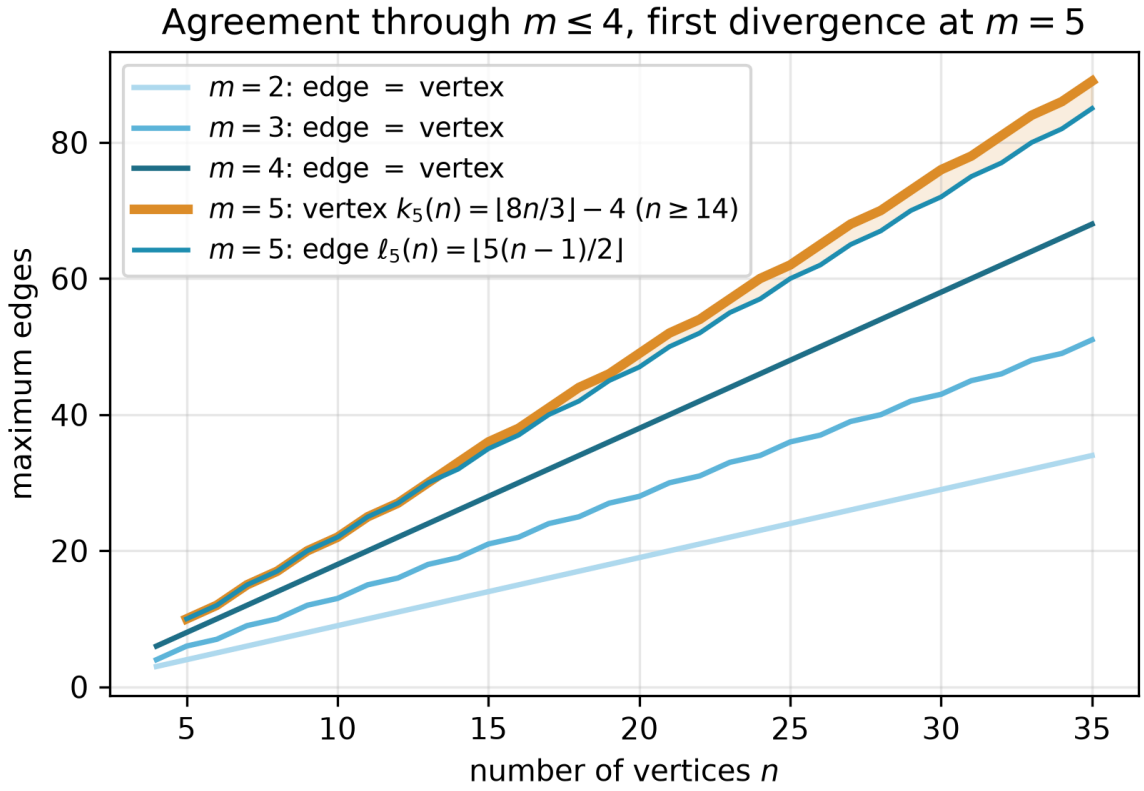


Figure 1.7. The edge and vertex extremal functions for $m = 2, 3, 4$ (blue shades, single curve per m since they agree) and $m = 5$ (thin blue for edge, thick orange for vertex). The vertex function diverges from the edge function at $n = 14$ and grows faster thereafter.

Definition 1.13 (Reachability skeleton)**.** Let D be a directed multigraph. A *reachability skeleton* of D is a subgraph R of D on the same vertices, with at most one copy of each arc, such that for all $u \neq v$: u can reach v in R exactly when u can reach v in D.

A skeleton keeps the information about who can reach whom. It does not keep the number of arc-disjoint paths.

Lemma 1.14 (A sparse reachability skeleton always exists)**.** *Every directed multigraph D on $n \geq 2$ vertices has a reachability skeleton R with*

$$|A(R)| \;\leq\; M(n) \;:=\; \max\Big(2(n-1),\; \Big\lfloor \frac{n^2}{4} \Big\rfloor\Big).$$

Proof. We build R first inside each strongly connected component, then between components, and then we count. A *strongly connected component* is a maximal set of vertices in which any two can reach each other. Every vertex lies in exactly one such component. This is because mutual reachability carries over: when u and v reach each other and v and w reach each other, u and w reach each other through v. Figure 1.9 shows an example.

Take a component with n_i vertices and choose a vertex r in it. We choose $n_i - 1$ arcs along which r reaches every vertex of the component. Start with r alone. As long as some vertex w of the component is not yet reached, take a directed path from r to w in D. Every vertex y on this

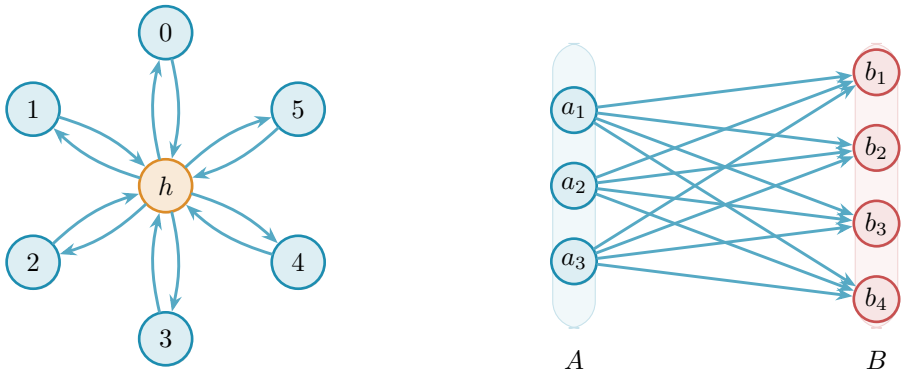


Figure 1.8. At $n = 7$ the two constructions have the same number of arcs, 12 each, and both have $\lambda^{\max} = 1$. Left: the bidirected star, a hub joined to six spokes both ways, $2(n-1) = 12$. Right: the one-way complete bipartite directed graph $B_{3,4}$, every arc from A to B, $\lfloor n^2/4 \rfloor = 12$.

path belongs to the component, because r reaches y and y reaches w, which in turn reaches r. The path starts at a reached vertex and ends at an unreached one. One of its arcs therefore goes from a reached vertex to an unreached vertex of the component. Add this arc. Each arc reaches one new vertex, so this ends after $n_i - 1$ arcs. In the same way, with all directions reversed, there are $n_i - 1$ arcs along which every vertex of the component reaches r. Any two vertices u, v of the component are then joined by a walk from u to r and on to v along these arcs, and this walk contains a u–v path. At most $2(n_i - 1)$ arcs therefore keep all reachability inside the component.

Replace each component by one point. Put an arc from one point to another if D has an arc from the first component to the second. This gives a directed graph Q_0 on q points, one per component. Q_0 has no *directed cycle*, that is, no sequence of arcs $x_1 \to x_2 \to \cdots \to x_k \to x_1$ through $k \geq 2$ different points. All vertices of the components on such a cycle could reach each other, so they would lie in one component. Now delete arcs from Q_0 one at a time, as long as this does not change which points of Q_0 reach which. Call the result Q. No arc of Q can be deleted without changing this.

The graph Q, with directions ignored, has no triangle. To see this, suppose three points of Q are joined in pairs. Two points are never joined both ways, since that would be a directed cycle. Each of the three points lies on two of the three arcs. Every arc leaves exactly one point, so the numbers of arcs leaving the three points add up to three. If every point had exactly one arc leaving it, the arcs would form a directed cycle. Some point x therefore has both of its arcs leaving. Name the other two points y and z so that the third arc is $y \to z$. The three arcs are then $x \to y$, $y \to z$ and $x \to z$. But then $x \to z$ could be deleted, since x still reaches z through y. This contradicts the choice of Q. By Mantel's theorem [14], a graph on q vertices with no triangle has at most $\lfloor q^2/4 \rfloor$ edges. So

$$|A(Q)| \;\leq\; \left\lfloor \frac{q^2}{4} \right\rfloor .$$

Let R consist of the $2(n_i - 1)$ arcs chosen above in every component, and, for each arc of Q, one arc of D between the corresponding components. These arcs keep reachability inside each component. Now suppose u reaches v in D, with u and v in different components. Then the

component of u reaches the component of v in Q_0, and so also in Q. Follow the arcs of Q from the first component to the last. For each arc of Q, the set R has an arc of D between the two components. Inside each component, the chosen arcs lead from the vertex where we enter to the vertex where we leave. This shows that u reaches v in R. Since R is part of D, it creates no new reachability, which makes R a reachability skeleton. If the components have sizes $n_1, \dots, n_q$, then

$$|A(R)| \;\le\; \sum_{i=1}^{q} 2(n_i - 1) \;+\; |A(Q)| \;\le\; 2(n-q) \;+\; \left\lfloor \frac{q^2}{4} \right\rfloor \;=:\; f(q).$$

It remains to bound $f(q)$ for $1 \le q \le n$. The identity $\lfloor (q+1)^2/4 \rfloor - \lfloor q^2/4 \rfloor = \lfloor (q+1)/2 \rfloor$ holds for every q. For $q = 2j$ the two terms are $j^2 + j$ and j^2, with difference j. For $q = 2j+1$ they are $(j+1)^2$ and $j^2 + j$, with difference $j+1$. This gives

$$f(q+1) - f(q) \;=\; -2 \;+\; \left\lfloor \frac{q+1}{2} \right\rfloor.$$

The difference is -1 for $q \le 2$, 0 for $q = 3$ and $q = 4$, and positive from $q = 5$ on. The function f goes down until $q = 3$, stays level until $q = 5$ and then goes up. Its largest value on $\{1, \dots, n\}$ is therefore at $q = 1$ or at $q = n$. Since $f(1) = 2(n-1)$ and $f(n) = \lfloor n^2/4 \rfloor$, we get $f(q) \le M(n)$. □

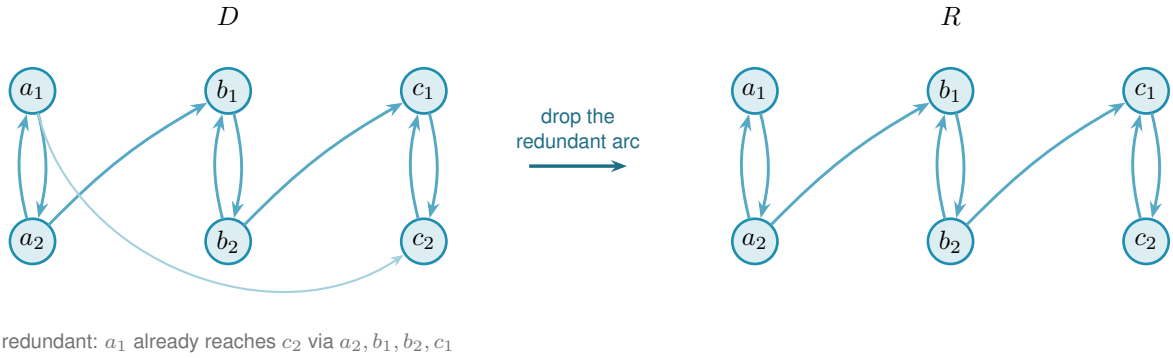


Figure 1.9. The construction of Lemma 1.14. Each bidirected pair A, B, C is a strongly connected component. Merging each pair into a single vertex gives $A \to B \to C$ and the redundant arc $A \to C$. Delete that arc to obtain the triangle-free directed graph Q. The skeleton R keeps the internal arcs of each pair and the two remaining connections between pairs. Although $a_1 \to c_2$ is removed, a_1 still reaches c_2 through B.

At $m = 2$ it is enough to show that a skeleton R from Lemma 1.14 contains every arc of a feasible simple directed graph D. Then $D = R$, which has at most $M(n)$ arcs.

Proof of Theorem 1.12. Lower bound. The bidirected star (Construction A.7 at $m = 2$) and the one-way bipartite directed graph (Construction A.8) have $\lambda^{\max} = 1$. Since $\kappa \le \lambda$, they also have $\kappa^{\max} \le 1$. This gives $\ell_2^{\mathrm{dir}}(n) \ge M(n)$ and $k_2^{\mathrm{dir}}(n) \ge M(n)$.

Upper bound. Let D be a simple directed graph on $n \ge 2$ vertices with $\kappa_D^{\max} \le 1$. By Lemma 1.14, D has a reachability skeleton R with $|A(R)| \le M(n)$.

We claim that R contains every arc of D. Suppose an arc (u, v) of D is not in R. Since u reaches v in D, it also reaches v in R, along a directed path P. The path P is not the arc (u, v), so it has length at least two. The arc $u \to v$ has no inner vertex, so P and the arc are two internally vertex-disjoint routes. This gives $\kappa_D(u, v) \geq 2$, a contradiction.

Hence $|A(D)| = |A(R)| \leq M(n)$. With the lower bound, $k_2^{\mathrm{dir}}(n) = M(n)$. □

Proof of Theorem 1.11. The two constructions above give $\ell_2^{\mathrm{dir}}(n) \geq M(n)$. Since $\kappa \leq \lambda$, every arc-feasible directed graph is vertex-feasible, so Theorem 1.12 bounds it. This gives

$$M(n) \;\leq\; \ell_2^{\mathrm{dir}}(n) \;\leq\; k_2^{\mathrm{dir}}(n) \;\leq\; M(n),$$

and all three are equal. □

1.4.3 Undirected hypergraphs

Theorem 1.15 (Hypergraph edge bound)**.** *For $n \geq r \geq 2$ and $m \geq 2$, an r-uniform multihypergraph on n vertices with $\lambda^{\max} \leq m - 1$ has size at most $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$. There are multihypergraphs attaining the bound for every $n \geq r$, and simple hypergraphs attaining it when $m - 1 \leq \binom{n-2}{r-2}$.*

Lemma 1.16 (Cut induction for multihypergraphs)**.** *Let $\mathcal{H}$ be a multihypergraph on $n \geq 1$ vertices, not assumed uniform, with $\lambda_{\mathcal{H}}^{\max} \leq k$. Then, the sum running over hyperedge copies,*

$$\sum_e \big(|e| - 1\big) \;\leq\; k\,(n - 1).$$

Proof. Induction on n. At $n = 1$ there is no hyperedge, since a hyperedge has at least two vertices, and both sides are 0.

Let $n \geq 2$ and fix two vertices $u \neq v$. Since $\lambda_{\mathcal{H}}(u, v) \leq k$, Theorem A.20 gives a set F of at most k hyperedge copies whose removal separates u from v. Let S be the set of vertices that u can reach in $\mathcal{H} - F$, and let $\bar{S} = V \setminus S$. Then $u \in S$ and $v \in \bar{S}$, so both sets are non-empty. Every copy that meets both S and $\bar{S}$ lies in F. A copy outside F with a vertex $x \in S$ and a vertex $y \in \bar{S}$ would let u reach x and then y in $\mathcal{H} - F$, so y would lie in S. At most k copies therefore meet both sides.

Form two *trace* multihypergraphs $\mathcal{H}[S]$ and $\mathcal{H}[\bar{S}]$: replace every copy e by $e \cap S$, or by $e \cap \bar{S}$, and drop the sets with fewer than two vertices. A Berge route in a trace is also a Berge route in $\mathcal{H}$, if we replace each trace copy by the copy it came from. Different trace copies come from different copies of $\mathcal{H}$. A family of hyperedge-disjoint routes in a trace therefore gives one of the same size in $\mathcal{H}$, and both traces have $\lambda^{\max} \leq k$. Both have fewer than n vertices, so the induction hypothesis applies to both.

Now count. By induction, the copies of $\mathcal{H}[S]$ contribute at most $k(|S|-1)$, and those of $\mathcal{H}[\bar{S}]$ at most $k(|\bar{S}|-1)$. A copy inside one side contributes $|e|-1$ to that side. A copy that meets both sides satisfies

$$|e|-1 \;=\; \big(|e\cap S|-1\big) \;+\; \big(|e\cap \bar{S}|-1\big) \;+\; 1,$$

where each bracket is the contribution to one trace. A crossing copy has at least one vertex on each side. Each bracket is therefore at least zero, with zero exactly when that part was dropped. Summing over all copies gives

$$\sum_{e}\big(|e|-1\big) \;\le\; k\big(|S|-1\big)+k\big(|\bar{S}|-1\big)+\big|\{\text{crossing copies}\}\big| \;\le\; k(n-2)+k \;=\; k(n-1). \;\square$$

A *hypertree* is a hypergraph whose incidence graph (Section 1.3.1) is a tree.

Proof of Theorem 1.15. Upper bound. Every hyperedge has $|e|=r$, so Lemma 1.16 with $k=m-1$ gives $(r-1)|E(\mathcal{H})|\le(m-1)(n-1)$. Since $|E(\mathcal{H})|$ is an integer, we can round down.

Lower bound. Write $N=n-1$, $s=r-1$ and $q=\left\lfloor\frac{(m-1)N}{s}\right\rfloor$. Take a hub h and number the other vertices $1,\dots,N$. Write the list $1,2,\dots,N$ out $m-1$ times in a row, keep its first qs entries, and cut these into q consecutive blocks of s entries. A block holds $s\le N$ consecutive entries of the list, so its entries are different vertices. For every block G, take one copy of the hyperedge $\{h\}\cup G$. Every vertex other than h appears at most $m-1$ times in the list, so it lies in at most $m-1$ hyperedge copies. Every pair contains such a vertex, and every route from or to it uses one of these copies. Removing these copies therefore separates the pair, and every pair has $\lambda\le m-1$. There are $q=\left\lfloor\frac{(m-1)(n-1)}{r-1}\right\rfloor$ hyperedge copies. For example, at $n=6$, $r=4$ and $m=3$ the blocks are $\{1,2,3\}$, $\{4,5,1\}$ and $\{2,3,4\}$, and the three hyperedges reach the bound $\lfloor 2\cdot 5/3\rfloor=3$.

When $r-1$ divides $n-1$, the blocks are the groups $G_1,G_2,\dots$ of a split of the other $n-1$ vertices into sets of size $r-1$, each repeated $m-1$ times. The incidence graph of the hypergraph with hyperedges $\{h\}\cup G_i$ is a tree, so we call it a *star hypertree*. The construction is then the star hypertree with every hyperedge at multiplicity $m-1$.

If $m-1\le\binom{n-2}{r-2}$ and $r\ge 3$, Theorem A.22 below attains the bound with a simple hypergraph. If $r=2$, this condition means $m=2$. Then the bound is $n-1$, and a tree attains it. $\square$

The star hypertree of Figure 1.10 attains the bound at $m=2$. It has a hub joined to disjoint groups of $r-1$ vertices. For $r\ge 3$, simple hypergraphs attain the bound whenever $m-1\le\binom{n-2}{r-2}$ (Theorem A.22). For $r\ge 3$, unlike for graphs, repeated edges therefore often do not help. The condition says that there are enough different hyperedges. When it fails, the simple value can be smaller. For example, at $n=r=m=3$ there is only one possible hyperedge. A simple hypergraph has one hyperedge, while a multihypergraph can have two copies of it (Remark A.23).

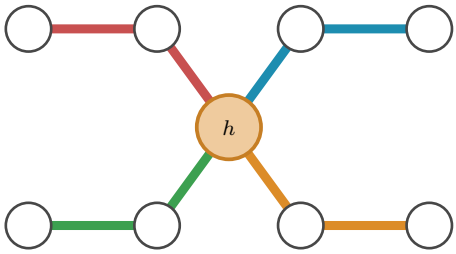


Figure 1.10. The $r = 3$, $n = 9$ star hypertree: four hyperedges meet only at the hub h, attaining $\lfloor (n-1)/(r-1) \rfloor = 4$ with $\lambda^{\max} = 1$.

Theorem 1.17 (Hypergraph vertex value at $m = 2$)**.** *For $r \geq 2$ and $n \geq 1$, the maximum number of hyperedges of an r-uniform multihypergraph on n vertices with $\kappa^{\max} \leq 1$ is*

$$K_2^{(r)}(n) \;=\; \left\lfloor \frac{n-1}{r-1} \right\rfloor,$$

attained by the star hypertree of Figure 1.10, which is simple. In particular $K_2^{(r)}(n) = k_2^{(r)}(n) = \ell_2^{(r)}(n)$: the vertex and edge problems agree at $m = 2$ for every uniformity, and repeated hyperedges never help.

Proof of Theorem 1.17. Let q be the number of hyperedges. We claim that $\kappa_{\mathcal{H}}^{\max} \leq 1$ exactly when $I(\mathcal{H})$ is a forest. The graph $I(\mathcal{H})$ is bipartite, with vertex nodes on one side and hyperedge nodes on the other, so a cycle in it alternates between the two kinds of nodes. Suppose $I(\mathcal{H})$ has a cycle $v_1, e_1, v_2, e_2, \ldots, v_\ell, e_\ell, v_1$, with $\ell \geq 2$ and all v_i and e_i distinct. Then v_1, e_1, v_2 and $v_1, e_\ell, v_\ell, \ldots, e_2, v_2$ are two Berge routes from v_1 to v_2 that share no hyperedge and no inner vertex, which gives $\kappa(v_1, v_2) \geq 2$. Conversely, two such routes between u and v are two paths in $I(\mathcal{H})$ with the same ends and no common inner node. Together they form a cycle in $I(\mathcal{H})$. This proves the claim.

Upper bound. Each hyperedge copy contains r vertices, so $I(\mathcal{H})$ has rq edges and $n + q$ nodes. A forest on $n + q$ nodes has at most $n + q - 1$ edges, because each of its components is a tree with one edge fewer than nodes. This gives $rq \leq n + q - 1$, that is, $q \leq (n-1)/(r-1)$. Since q is an integer, $q \leq \lfloor (n-1)/(r-1) \rfloor$.

Lower bound. Take the star hypertree with $\lfloor (n-1)/(r-1) \rfloor$ hyperedges, and leave the remaining vertices in no hyperedge. Its hyperedges meet only at the hub, and every other vertex lies in at most one hyperedge, so its incidence graph is a forest. It therefore has $\kappa^{\max} \leq 1$, and it is simple.

The three values agree. The star hypertree also has $\lambda^{\max} \leq 1$, since removing the one hyperedge at a non-hub vertex separates it from every other vertex. It is therefore feasible in all three problems, and $\ell_2^{(r)}(n), k_2^{(r)}(n), K_2^{(r)}(n) \geq \lfloor (n-1)/(r-1) \rfloor$. A simple hypergraph is a multihypergraph, and $\kappa \leq \lambda$, so $\ell_2^{(r)}(n) \leq k_2^{(r)}(n) \leq K_2^{(r)}(n)$. The upper bound above gives $K_2^{(r)}(n) \leq \lfloor (n-1)/(r-1) \rfloor$, so all three are equal. □

Theorem 1.18 (Hypergraph vertex value at $m = 3$)**.** *For $r \geq 2$ and $n \geq 1$, every r-uniform multihypergraph on n vertices with $\kappa^{\max} \leq 2$ has at most $\left\lfloor \frac{2(n-1)}{r-1} \right\rfloor$ hyperedges. For $2 < r < n$, which is exactly the range where $2 \leq \binom{n-2}{r-2}$, the bound is attained by a simple hypergraph, so*

$$k_3^{(r)}(n) \;=\; \left\lfloor \frac{2(n-1)}{r-1} \right\rfloor \;=\; \ell_3^{(r)}(n).$$

For multihypergraphs the bound is attained for every $n \geq r$, so $K_3^{(r)}(n) = \lfloor 2(n-1)/(r-1) \rfloor$ for all $n \geq r \geq 2$. For all $n \geq 1$ and $r \geq 2$,

$$k_3^{(r)}(n) \;=\; \ell_3^{(r)}(n).$$

The upper bound at $m = 3$ follows from the edge bound of Theorem 1.15. At threshold two, the edge problem and the vertex problem allow exactly the same multihypergraphs. The proof uses a *theta*: three paths between two vertices $a \neq b$ that pairwise share no edge and no inner vertex.

Lemma 1.19 (Three routes give a theta)**.** *Let $\mathcal{H}$ be a multihypergraph, not assumed uniform. Then $\kappa_{\mathcal{H}}^{\max} \leq 2$ exactly when $\lambda_{\mathcal{H}}^{\max} \leq 2$.*

Proof. Since $\kappa \leq \lambda$, the condition $\lambda^{\max} \leq 2$ gives $\kappa^{\max} \leq 2$. For the converse, suppose that two vertices u, v are joined by three edge-disjoint routes P_1, P_2, P_3. We find two vertices joined by three separated routes.

Every hyperedge copy used by one of the routes is used by only one of them, and that route passes through it from one vertex to the next. Replace the copy by an ordinary edge between these two vertices, and remember the copy it came from. This gives a multigraph G on the vertices of $\mathcal{H}$, in which P_1, P_2, P_3 are three edge-disjoint u–v paths.

Take an edge $e = xy$ of P_1, with x the end closer to u along P_1. Walk along P_1 from x back to u, then along P_2 to v, then along P_1 back to y. This walk does not use e, because P_2 shares no edge with P_1. Cutting out the parts where it repeats a vertex leaves an x–y path, which closes with e to a cycle C_2 in $P_1 \cup P_2$ through e. In the same way $P_1 \cup P_3$ contains a cycle C_3 through e. The two cycles are different. A cycle in both would lie in P_1, because P_2 and P_3 share no edge, and the path P_1 contains no cycle.

C_3 therefore has an edge that is not an edge of C_2. Follow C_3 from this edge in both directions until it first reaches a vertex of C_2. This gives a path S between two vertices a, b of C_2, with no edge of C_2 and no inner vertex on C_2. The vertices a and b are different. C_3 passes each of its vertices once, and it meets C_2 in at least the two ends of e, so S cannot run all the way round C_3. The two a–b arcs of C_2 and the path S form a theta. Parallel edges may occur in G, and a cycle may then have only two edges. The argument is the same.

Now put the hyperedge copies back. The edges of the theta come from different copies, so its three paths become three a–b routes in $\mathcal{H}$ that share no hyperedge copy. Their inner vertices are

the inner vertices of the three paths, which are disjoint. The three routes are therefore separated, and $\kappa_{\mathcal{H}}(a, b) \geq 3$. □

The theta may join a different pair than u, v. In two triangles that share one vertex, a vertex of one triangle and a vertex of the other have $\lambda = 2$ and $\kappa = 1$, while $\lambda^{\max} = \kappa^{\max} = 2$. The lemma is about the largest values over all pairs, not about each pair.

Proof of Theorem 1.18. Upper bound. Let $\mathcal{H}$ have q hyperedge copies and $\kappa^{\max} \leq 2$. By Lemma 1.19, $\lambda^{\max} \leq 2$. Every hyperedge has r vertices, so Lemma 1.16 with $k = 2$ gives $(r-1)q \leq 2(n-1)$. Since q is an integer, $q \leq \lfloor 2(n-1)/(r-1) \rfloor$.

Lower bound, simple. For $2 < r < n$, this is Proposition A.26 at $m = 3$. It needs $2 \leq \binom{n-2}{r-2}$, which holds exactly when $2 < r < n$. At $r = 2$ and at $r = n$ the binomial coefficient equals 1. When $2 < r < n$ we have $1 \leq r - 2 \leq n - 3$, and the coefficient is at least $n - 2 \geq 2$.

The edge value. Since $\kappa \leq \lambda$, every hypergraph with $\lambda^{\max} \leq 2$ also has $\kappa^{\max} \leq 2$, so $\ell_3^{(r)}(n) \leq k_3^{(r)}(n)$. For $2 < r < n$, Theorem 1.15 gives a simple hypergraph with $\lambda^{\max} \leq 2$ and $\lfloor 2(n-1)/(r-1) \rfloor$ hyperedges, so $\ell_3^{(r)}(n)$ also equals the bound.

Lower bound, multihypergraphs. For every $n \geq r$, Proposition A.27 gives it.

Equal values. A simple hypergraph is a multihypergraph, so by Lemma 1.19 a simple hypergraph has $\kappa^{\max} \leq 2$ exactly when it has $\lambda^{\max} \leq 2$. The two problems allow the same simple hypergraphs, and $k_3^{(r)}(n) = \ell_3^{(r)}(n)$ for all $n \geq 1$ and $r \geq 2$. For $n < r$ both values are 0, and for $n = r$ both are 1. □

Remark 1.20 (Where the method stops). Lemma 1.19 is false at threshold three. In a triangle with every edge doubled, two vertices are joined by their two parallel edges and by one more separated route through the third vertex, so $\kappa^{\max} = 3$. Using both copies of the other two edges gives four edge-disjoint routes, so $\lambda^{\max} = 4$. For $m = 4$ a different bound would be needed.

The multigraph results of Section A.7 and the block decomposition in Appendix A use three more words. For a connected multigraph G, the *cycle rank* is $\operatorname{rank}(G) = |E(G)| - |V(G)| + 1$. A tree has cycle rank 0, and each extra edge raises it by one. A *cut vertex* is a vertex whose removal disconnects the graph, that is, leaves more components than before. A graph on more than k vertices is *k-connected* if removing any $k - 1$ vertices leaves it connected.

For $r = 2$ these results are about multigraphs. They give $K_2(n) = n - 1$, attained by trees, and $K_3(n) = 2(n-1)$, attained by a tree with every edge doubled (Corollary A.31). For $m = 5$ the vertex and edge values differ. On four vertices, join two of them by 2 parallel edges, and join each of the other two to both of them by 3 parallel edges. This multigraph has 14 edges and $\kappa^{\max} = 4$ (Section A.7), while $L_5(4) = 12$. For $m = 4$ this is open. Section A.7 discusses larger m.

1.4.4 Directed bounds beyond the first threshold

Theorem 1.21 (The quadratic bound with a linear error)**.** *For all $m \geq 2$ and $n \geq 2$, every simple directed graph D on n vertices with $\lambda_D^{\max} \leq m-1$, or with $\kappa_D^{\max} \leq m-1$, satisfies*

$$|A(D)| \;\leq\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; (m-1)(n-1),$$

so for each fixed m,

$$\ell_m^{\mathrm{dir}}(n) \;=\; \frac{n^2}{4} \;+\; \Theta_m(n) \quad \textit{and} \quad k_m^{\mathrm{dir}}(n) \;=\; \frac{n^2}{4} \;+\; \Theta_m(n) \qquad (m \geq 3).$$

Appendix A gives the proof. It counts routes of length two. For an ordered pair (u, v), the vertex x is the *midpoint* of the route $u \to x \to v$. Routes with different midpoints, together with the arc $u \to v$, are arc-disjoint. A feasible directed graph therefore has at most $m-1$ of them. Take a vertex v of largest out-degree Δ. For each of the $n-1$ pairs (v, w), the midpoints x of routes $v \to x \to w$, together with the arc $v \to w$ if it exists, number at most $m-1$. Adding this over w counts every arc from an out-neighbour of v to a vertex other than v, and the Δ arcs out of v. The arcs from out-neighbours back to v are not counted, but there are at most Δ of them. The out-neighbours of v therefore send at most $(m-1)(n-1)$ arcs in total. The other $n-1-\Delta$ vertices send at most Δ arcs each. Together with the Δ arcs out of v this gives at most $\Delta(n-\Delta) + (m-1)(n-1)$ arcs, and $\Delta(n-\Delta) \leq \lfloor n^2/4 \rfloor$. The routes of length two are also internally vertex-disjoint, so the same proof works for internally vertex-disjoint routes.

Theorem 1.22 (Directed multigraph value with arc-disjoint routes, every n and m)**.** *For all $n \geq 2$ and $m \geq 2$,*

$$L_m^{\mathrm{dir}}(n) \;=\; (m-1)\, M(n), \qquad M(n) := \max\Big(2(n-1),\; \left\lfloor \frac{n^2}{4} \right\rfloor\Big).$$

The lower bound uses thickened trees and thickened one-way bipartite directed graphs.

If we multiply every multiplicity by the same integer, every cut and so every connectivity is multiplied by that integer.

Lemma 1.23 (Thickening)**.** *Let D_0 be a directed multigraph and $k \geq 1$ an integer. Write $k \cdot D_0$ for the multigraph obtained by multiplying every multiplicity $\mu(u, v)$ by k. Then $\lambda_{k \cdot D_0}(u, v) = k \cdot \lambda_{D_0}(u, v)$ for every ordered pair $u \neq v$, and $|A(k \cdot D_0)| = k\,|A(D_0)|$.*

Proof. By Menger's theorem, $\lambda_{D_0}(u, v)$ is the smallest number of arcs, counted with multiplicity, in a u–v cut. Multiplying every multiplicity by k multiplies the size of every cut by k. It therefore multiplies the smallest cut by k too. The number of arcs is also multiplied by k. □

Construction 1.24 (Thickened tree and thickened bipartite directed graph)**.** Let $m \geq 2$. (a) Take any tree on the n vertices and put $m-1$ arcs in each direction on every tree edge. With one arc in each direction this is the bidirected tree. Each ordered pair is joined by exactly one directed

path, which follows the path of the tree, giving $\lambda^{\max} = 1$. By Lemma 1.23, the thickened tree then has $\lambda^{\max} = m - 1$ and $2(m-1)(n-1)$ arcs. (b) Take the one-way bipartite directed graph of Construction A.8 and repeat every arc $m - 1$ times. By the same lemma it has $\lambda^{\max} = m - 1$ and $(m-1)\lfloor n^2/4 \rfloor$ arcs.

The larger of the two gives $L_m^{\mathrm{dir}}(n) \geq (m-1)M(n)$.

For the upper bound we want to delete a part of D that lowers every positive connectivity, and whose size we can bound. Deleting one vertex does not work: it need not lower every connectivity. Instead we delete a reachability skeleton. By Lemma 1.14 it has at most $M(n)$ arcs, and the next lemma shows that deleting it lowers every positive connectivity.

Lemma 1.25 (Deleting a skeleton lowers every positive connectivity)**.** *Let D be a directed multigraph with $\lambda^{\max}(D) \leq k$ for some $k \geq 1$, and let R be a reachability skeleton of D. Let $D - R$ be D with one copy of each arc of R removed. Then for every ordered pair $u \neq v$,*

$$\lambda_{D-R}(u,v) \;\leq\; \max\bigl(\lambda_D(u,v) - 1, 0\bigr),$$

and consequently $\lambda^{\max}(D - R) \leq k - 1$.

The skeleton keeps reachability, so if routes from u to v remain after the deletion, one more route runs inside the deleted arcs.

Proof. Fix $u \neq v$ and let $t = \lambda_{D-R}(u,v)$. If $t = 0$ there is nothing to prove. Otherwise $D - R$ has $t \geq 1$ arc-disjoint u–v paths, so u reaches v in D. Since R is a reachability skeleton, there is a u–v path P in R. The arcs of P are the removed copies, so P shares no arc with the t paths in $D - R$. Together with P, this gives $t + 1$ arc-disjoint u–v paths in D, and $t + 1 \leq \lambda_D(u,v)$. Taking the maximum over all pairs gives $\lambda^{\max}(D - R) \leq k - 1$. □

Proof of Theorem 1.22. The lower bound is Construction 1.24. For the upper bound, fix n. We show by induction on $k \geq 0$ that every directed multigraph D on n vertices with $\lambda^{\max}(D) \leq k$ has $|A(D)| \leq k\,M(n)$. For $k = 0$ there are no arcs, since one arc is already one route.

Let $k \geq 1$, and let R be a reachability skeleton of D with $|A(R)| \leq M(n)$ (Lemma 1.14). By Lemma 1.25, $\lambda^{\max}(D - R) \leq k - 1$, so by induction $|A(D - R)| \leq (k-1)M(n)$. Every arc of D is either in R or in $D - R$. So

$$|A(D)| \;=\; |A(R)| \;+\; |A(D-R)| \;\leq\; M(n) \;+\; (k-1)M(n) \;=\; k\,M(n).$$

With $k = m - 1$ this gives $L_m^{\mathrm{dir}}(n) \leq (m-1)M(n)$. □

For $n \geq \max(8, 2m)$ the bipartite construction is the only extremal multigraph, up to reversing all arcs (Theorem A.19).

Theorem 1.26 (The directed hypergraph constant)**.** *Let $r \geq 2$ and $m \geq 2$. Every forward directed r-uniform multihypergraph $\mathcal{H}$ on $n \geq 2$ vertices with $\kappa_{\mathcal{H}}^{\max} \leq m-1$ satisfies*

$$|E(\mathcal{H})| \;\leq\; \frac{m-1}{r-1}\left\lfloor \frac{n^2}{4} \right\rfloor \;+\; (m-1)^2(n-1).$$

Since $\kappa \leq \lambda$, the same bound holds when $\lambda_{\mathcal{H}}^{\max} \leq m-1$. The construction of Proposition A.41 has $(1+o(1))(m-1)n^2/(4(r-1))$ hyperedges, so the leading term is right.

Appendix A gives the proof. It uses the *shadow* directed graph, which has an arc $u \to v$ when some hyperedge has tail u and head v. Each hyperedge gives $r-1$ arcs of the shadow, and each arc comes from at most $m-1$ hyperedges. The route condition on the hypergraph limits the routes of length two in the shadow. Lemma A.15 then bounds the number of arcs of the shadow.

The backward model is the forward model with every hyperedge reversed, so it has the same extremal values (Lemma A.43). The general model has the same leading term, with a larger linear term (Theorem A.46).

At $r = 2$ this also covers directed multigraphs with internally vertex-disjoint routes. At $m = 2$ no arc can be repeated, because two copies of an arc are two internally vertex-disjoint routes. A feasible directed multigraph is then a simple directed graph, which gives $K_2^{\mathrm{dir}}(n) = k_2^{\mathrm{dir}}(n) = \ell_2^{\mathrm{dir}}(n)$ by Theorem 1.12. For fixed m, the extremal multigraphs of Theorem 1.22 are feasible here too, since $\kappa \leq \lambda$. With Theorem 1.26 at $r = 2$ this gives $K_m^{\mathrm{dir}}(n) = (m-1)n^2/4 + O_m(n)$ (Corollary A.38).

Section A.3.1 compares Theorem 1.21 with a theorem of Huang and Lyu [9].

1.5 Overview and open problems

Table 1.3 collects what is known about the sixteen problems.

The main open problems are:

- Find $\ell_m^{\mathrm{dir}}(n)$ for $m \geq 3$, or at least the coefficient of the linear term.
- Is $k_m^{\mathrm{dir}}(n) = \ell_m^{\mathrm{dir}}(n)$ for $m \geq 3$?
- Find the simple vertex value $k_m(n)$ for $m \geq 6$, and the multigraph vertex value $K_m(n)$ for $m \geq 4$. Is $K_4(n) = L_4(n) = 3(n-1)$ for every n?
- Find the simple hypergraph edge value outside the proved attainment range, and a vertex bound for hypergraphs at $m \geq 4$.
- Find exact values for directed hypergraphs, and compare the different ways to orient a hyperedge (Section A.8) beyond the leading term.

Model	Edge-disjoint or arc-disjoint routes	Internally vertex-disjoint or separated routes
Simple graph	$\lfloor m(n-1)/2 \rfloor$ for $n \geq m$	Exact for $m \leq 5$. Open for $m \geq 6$
Multigraph	$(m-1)(n-1)$	$n-1$ at $m=2$, $2(n-1)$ at $m=3$. Open for $m \geq 4$
Simple hypergraph	At most $\lfloor (m-1)(n-1)/(r-1) \rfloor$, equal when $m-1 \leq \binom{n-2}{r-2}$	$\lfloor (n-1)/(r-1) \rfloor$ at $m=2$, $\lfloor 2(n-1)/(r-1) \rfloor$ at $m=3$ for $2 < r < n$. Open for $m \geq 4$
Multihypergraph	$\lfloor (m-1)(n-1)/(r-1) \rfloor$ for $n \geq r$	$\lfloor (n-1)/(r-1) \rfloor$ at $m=2$, $\lfloor 2(n-1)/(r-1) \rfloor$ at $m=3$. Open for $m \geq 4$
Simple directed graph	$M(n)$ at $m=2$. $n^2/4 + \Theta_m(n)$ for $m \geq 3$	Same
Directed multigraph	$(m-1)M(n)$	$M(n)$ at $m=2$. $(m-1)n^2/4 + O_m(n)$
Directed simple hypergraph	$(1+o(1))(m-1)n^2/(4(r-1))$	Same
Directed multihypergraph	$(1+o(1))(m-1)n^2/(4(r-1))$	Same

Table 1.3. Status of the sixteen problems. Here $m \geq 2$, hypergraphs have $r \geq 3$, and asymptotic statements are for fixed m and r as $n \to \infty$. The directed hypergraph rows hold in the forward, backward and general models.

Chapter 2

Measuring and Searching by Machine

Chapter 1 stated the problem and the proved results. This chapter explains how a program can measure connectivity in every variant, check small graphs one by one, and search for dense examples. Its output for $n, m \leq 7$ is printed in Appendix B. Section 2.5 shows how an output can be checked without reading the code.

The program is one Python file of about 700 lines, `program/erdos915.py`. It uses the library `networkx` for maximum flows, and the programs `geng`, `directg` and `genbg` of `nauty` to list graphs. This chapter prints the main parts of the program, each next to the idea it implements. The remaining parts, such as the imports and small helper functions, are in Appendix B, so every line of the program is printed once. The names in the code follow the words of the text. Figure 2.1 shows the structure of the program.

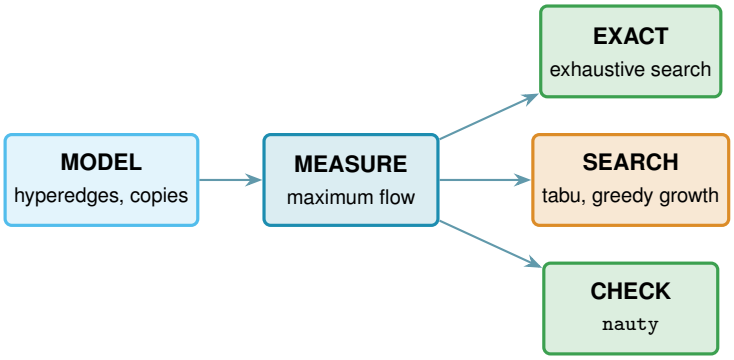


Figure 2.1. One model feeds the maximum-flow computation. Exhaustive search uses it to find exact values, heuristic search uses it to find dense examples, and the check with `nauty` uses it to show that no feasible graph has one edge more than the densest one found (Section 2.3.2).

2.1 Modelling

The program treats all eight models in one way. A model is described by four facts: whether it is directed, whether edges may repeat, the edge size r, and, for directed hypergraphs, the allowed numbers of tails. A graph is the case $r = 2$. The program runs each directed hypergraph model in three versions, forward, backward and general (Section 1.3), so `ALL_MODELS` lists twelve entries for the eight models. Whether the routes must be edge-disjoint or separated is a separate choice, so the eight models give the sixteen problems. The code calls this choice the *separation*, and its printed tables say *edge separation* for edge-disjoint routes and *vertex separation* for separated routes, which in graphs are the internally vertex-disjoint routes. A `Problem` bundles a model,

this choice, the number of vertices n and the threshold m, and every function below takes one (Listing 2.1).

```python
@dataclass(frozen=True)
class Model:
    NAME: str
    IS_DIRECTED: bool
    ALLOWS_REPEATED_EDGES: bool
    EDGE_SIZE: int               # r: 2 for graphs, at least 3 otherwise
    TAIL_COUNTS: tuple = (1,)    # directed: the allowed numbers of tails


def ALL_MODELS(HYPEREDGE_SIZE=3):
    r = HYPEREDGE_SIZE
    splits = [("forward", (1,)), ("backward", (r - 1,)),
              ("general", tuple(range(1, r)))]
    return [
        Model("simple graph",             False, False, 2),
        Model("multigraph",               False, True,  2),
        Model("simple hypergraph",        False, False, r),
        Model("multihypergraph",          False, True,  r),
        Model("simple directed graph",    True,  False, 2),
        Model("directed multigraph",      True,  True,  2),
    ] + [
        Model(f"{name}, {split}", True, repeated, r, tail_counts)
        for name, repeated in [("directed hypergraph", False),
                               ("directed multihypergraph", True)]
        for split, tail_counts in splits
    ]


class Separation(Enum):
    EDGE_DISJOINT = "edge"
    VERTEX_DISJOINT = "vertex"


@dataclass(frozen=True)
class Problem:
    MODEL: Model
    SEPARATION: Separation
    VERTEX_COUNT: int            # n
    THRESHOLD_M: int             # feasible: fewer than m disjoint routes
```

Listing 2.1. The models, with three versions of each directed hypergraph model, and the choice between edge-disjoint and separated routes.

Every edge is stored as a hyperedge: its vertices, and its tails if the model is directed. An ordinary edge uv is the hyperedge $\{u, v\}$ without tails, and an arc $u \to v$ is the hyperedge $\{u, v\}$ with tail u. A route may enter a hyperedge only at one of its *entry vertices*, and leave it only at one of its *exit vertices* (`entry_vertices` and `exit_vertices` in the code). In an undirected

hyperedge every vertex is both. In a directed one only the tails are entries and only the heads are exits. The program stores a candidate graph by giving every possible hyperedge a multiplicity, the number of copies it has. We call the models without repeated edges *simple models*, and the models with repeated edges *multi models*. In a simple model every multiplicity is 0 or 1. In a multi model it is at most $m - 1$, since m copies of one edge already give m routes that are both edge-disjoint and separated (Listing 2.2). Figure 2.2 shows two graphs stored in this way.

```
@dataclass(frozen=True, order=True)
class Hyperedge:
    TAILS: tuple                # empty for an undirected hyperedge
    MEMBERS: tuple              # all its vertices, sorted

    def entry_vertices(self):
        return self.TAILS or self.MEMBERS

    def exit_vertices(self):
        return tuple(v for v in self.MEMBERS if v not in self.TAILS)


def possible_hyperedges(problem):
    model, n = problem.MODEL, problem.VERTEX_COUNT
    hyperedges = []
    for members in itertools.combinations(range(n), model.EDGE_SIZE):
        if not model.IS_DIRECTED:
            hyperedges.append(Hyperedge((), members))
            continue
        for tail_count in model.TAIL_COUNTS:
            hyperedges += [Hyperedge(tails, members) for tails
                           in itertools.combinations(members, tail_count)]
    return hyperedges


def largest_multiplicity(problem):
    if problem.MODEL.ALLOWS_REPEATED_EDGES:
        return problem.THRESHOLD_M - 1
    return 1
```

Listing 2.2. Hyperedges, the list of all possible hyperedges, and the largest useful multiplicity.

2.2 Measuring

Every search below needs $\lambda^{\max}$ or $\kappa^{\max}$ of a candidate graph. This section explains how the program computes them.

To compute $\lambda(u, v)$ we use a maximum flow. Think of the graph as a network of pipes in which every link has a *capacity*, the most that can pass through it. A *flow* from u to v sends an amount along every link, never more than its capacity. At every node other than u and v, the amount coming in equals the amount going out. The value of the flow is the amount that leaves u, which is also the amount that arrives at v. A *maximum flow* is a flow of largest value. We give

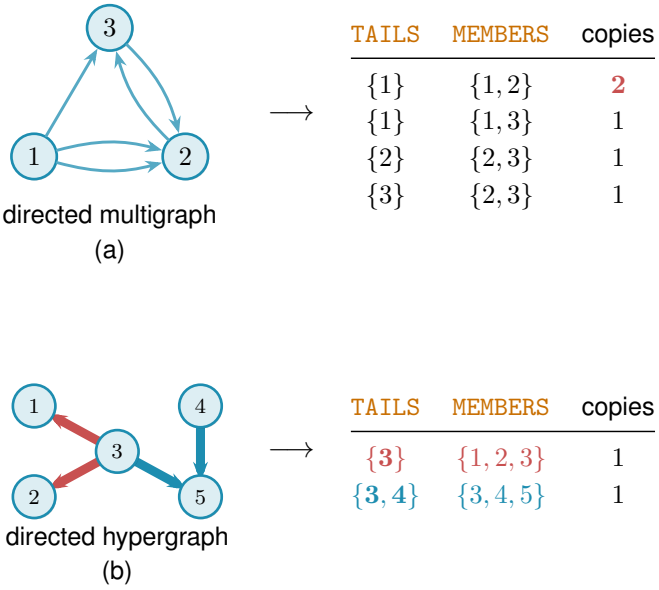


Figure 2.2. The program stores every model in one way: each hyperedge with its number of copies. (a) A directed multigraph. The arc $u \to v$ is the hyperedge with members $\{u, v\}$ and tail u, and the two parallel arcs from 1 to 2 are one hyperedge with 2 copies. (b) A directed 3-uniform hypergraph in the general model. Each hyperedge records its tails and its three members: the red one has tail 3, the blue one has tails 3 and 4 and head 5. In an undirected model there are no tails. A hyperedge that does not occur has 0 copies and is left out.

each pair of vertices a link with capacity $\mu(u, v)$. In a directed graph the link has a direction. In an undirected graph it can be used both ways. A *cut* is a set of links whose removal leaves no way from u to v. Its capacity is the sum of the capacities of its links. The maximum-flow minimum-cut theorem of Ford and Fulkerson [6] says that the largest value of a flow equals the smallest capacity of a cut. In this network a cut of smallest capacity is a smallest set of edges, counted with multiplicity, that separates u from v. Menger's theorem says its size is $\lambda(u, v)$. Computing a maximum flow therefore computes $\lambda(u, v)$. We call this the *maximum-flow computation*.

2.2.1 Transformations

The maximum-flow computation works on one kind of network for all sixteen problems.

Every hyperedge, including an ordinary edge, becomes a *gate* (Figure 2.4b). The gate has an entry node and an exit node, joined by one arc whose capacity is the multiplicity of the hyperedge. Each vertex of the hyperedge gets an arc of unlimited capacity to the entry node and one from the exit node. The arc inside the gate is therefore the only bottleneck. For a directed hyperedge, only the tails have an arc to the entry, and only the heads have an arc from the exit (Figure 2.4c). A route must then go from a tail to a head. We call the result the *gate network*. Theorem A.20 shows that the maximum flow in this network is the number of hyperedge-disjoint Berge routes. Figure 2.3 shows an example.

Every vertex v is split into two nodes v^{in} and v^{out}, joined by one arc (Figure 2.4a). Routes arrive at v^{in} and leave from v^{out}. For separated routes this arc has capacity one, so at most one route can pass through v. For edge-disjoint routes its capacity is unlimited, and it changes

nothing. The gates keep their capacity, so parallel copies still give different routes, as in the incidence convention of Section 1.3.1. Listing 2.3 builds this network.

```
def build_flow_network(problem, multiplicity):
    network = networkx.DiGraph()
    if problem.SEPARATION is Separation.VERTEX_DISJOINT:
        through_vertex = 1
    else:
        through_vertex = math.inf
    for v in range(problem.VERTEX_COUNT):
        network.add_edge(("in", v), ("out", v), capacity=through_vertex)
    for hyperedge, copies in multiplicity.items():
        if copies == 0:
            continue
        enter, leave = ("enter", hyperedge), ("exit", hyperedge)
        network.add_edge(enter, leave, capacity=copies)
        for v in hyperedge.entry_vertices():
            network.add_edge(("out", v), enter, capacity=math.inf)
        for v in hyperedge.exit_vertices():
            network.add_edge(leave, ("in", v), capacity=math.inf)
    return network
```

Listing 2.3. The flow network: split vertices and one gate per hyperedge.

The largest number of edge-disjoint routes, or of separated routes, from u to v is the maximum flow from u^{out} to v^{in} (Listing 2.4). A graph is feasible when this number is below m for every pair, ordered in directed models. The library builds the flow in steps, and one step can raise its value by more than one. With the `stop_at` argument it stops at the first step that brings the value to m or more, since a larger value changes nothing: the pair already breaks the condition.

```
def disjoint_routes(network, source, target, stop_at=None):
    return networkx.maximum_flow_value(
        network, ("out", source), ("in", target),
        flow_func=networkx.algorithms.flow.edmonds_karp, cutoff=stop_at)


def is_feasible(problem, multiplicity):
    m = problem.THRESHOLD_M
    network = build_flow_network(problem, multiplicity)
    return all(disjoint_routes(network, u, v, stop_at=m) < m
               for u, v in vertex_pairs(problem))
```

Listing 2.4. Counting edge-disjoint or separated routes, and the feasibility test.

2.3 Checking every graph with a fixed number of vertices

The program stores a graph by giving each possible hyperedge its number of copies (Figure 2.2). The possible hyperedges depend on the model:

- in a graph, the $\binom{n}{2}$ pairs $\{u, v\}$,

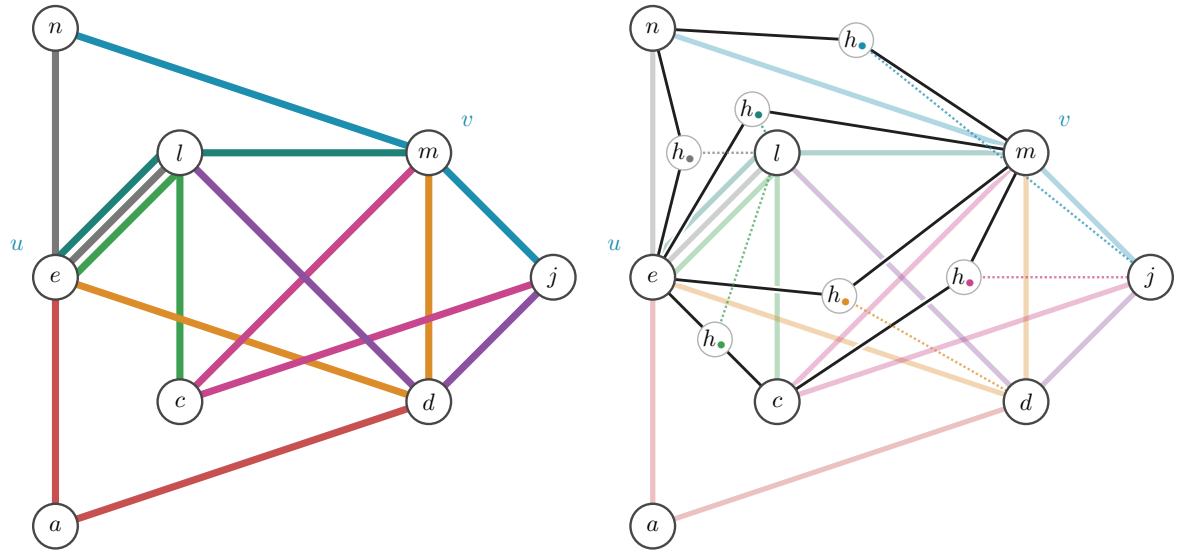


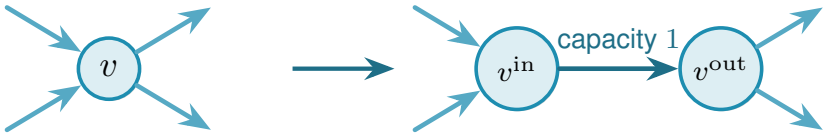
Figure 2.3. (a) An undirected 3-uniform hypergraph. (b) The relevant part of its gate network for the pair u, v. The four black routes use pairwise disjoint sets of gates, so $\lambda(u, v) \geq 4$. The vertex v lies in only four hyperedges, and removing those four hyperedges separates u from v, so $\lambda(u, v) \leq 4$. Hence $\lambda(u, v) = 4$. Each gate is drawn as one point. A dotted line joins a gate to the third vertex of its hyperedge, which the route does not use.

- in a directed graph, the $n(n-1)$ ordered pairs, one for each arc $u \to v$,
- in an r-uniform hypergraph, the $\binom{n}{r}$ sets of r vertices, called *r-sets*,
- in a directed r-uniform hypergraph, an r-set together with a choice of which of its vertices are tails. A forward hyperedge has one tail, so each r-set gives r possible hyperedges. A backward hyperedge has one head, which also gives r. In the general model the tails form any non-empty proper subset, which gives $2^r - 2$.

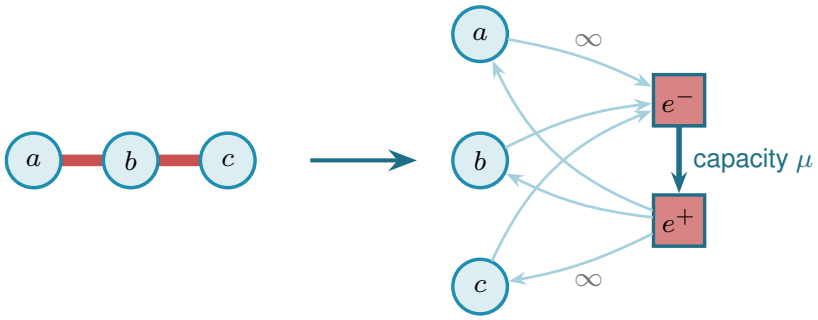
The number of copies of each possible hyperedge is between 0 and the largest multiplicity: 1 in a simple model and $m - 1$ in a multi model. To find an exact value, the program must show that no graph with more edges than the best feasible graph found is feasible. If the search stops early, it gives only a lower bound.

The number of vertices matters most. The number of candidates is (values per entry) to the power (number of entries). For graphs the number of entries grows like n^2, and for r-uniform hypergraphs like n^r. A larger m gives more values per entry. Direction gives more entries, since both directions of a pair are chosen separately. Directed hypergraphs have the most candidates, because each r-set carries several possible hyperedges. The choice between edge-disjoint and separated routes does not change the candidates. It only changes the test for each candidate.

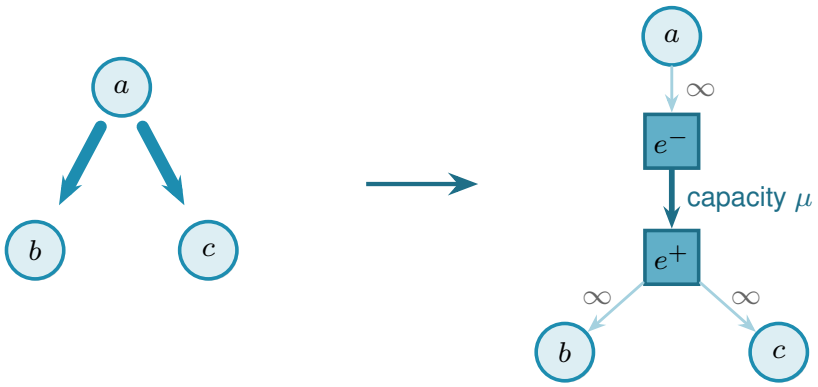
For example, there are $2^{\binom{n}{2}}$ simple graphs and $2^{n(n-1)}$ simple directed graphs on n vertices. With multiplicities at most $m - 1$ there are $m^{\binom{n}{2}}$ multigraphs and $m^{n(n-1)}$ directed multigraphs. Figure 2.5 shows how fast this grows. The program uses two exact methods: a pruned search,



(a) Splitting each vertex v into v^{in} and v^{out}, joined by an arc of capacity one, turns separated routes into arc-disjoint routes. Only this inner arc has capacity one.



(b) Transform a hyperedge into a gate with capacity equal to the multiplicity of the hyperedge.



(c) A directed hyperedge with one tail and two heads is transformed into a gate. This gate has capacity equal to the multiplicity of the hyperedge and every route must enter at the tail and leave at a head. With several tails, each tail has an arc to e^-.

Figure 2.4. The building blocks of the maximum-flow computation: vertex splitting, hyperedge gates and directed hyperedge gates.

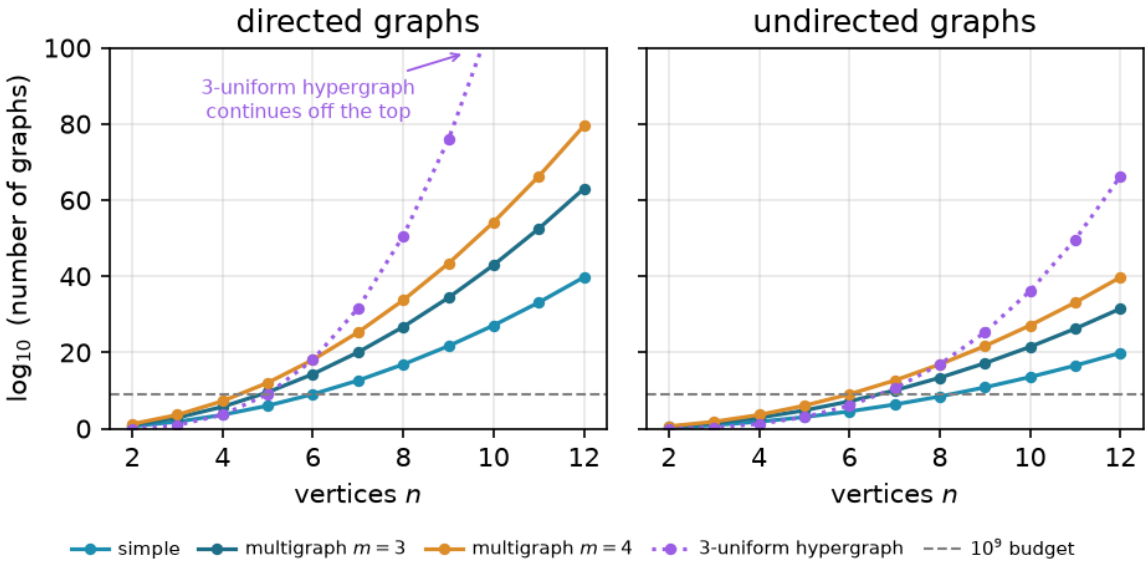


Figure 2.5. The number of candidates for blind enumeration, on a logarithmic scale. Every model passes 10^9 candidates before $n = 10$. Directed models grow fastest, because they have more entries to choose. The directed hypergraph curve is the forward model.

for every model, and a check of the graphs with one edge more, for every model except directed hypergraphs.

2.3.1 Pruned search

Checking every possible graph one by one gives the exact answer, but there are far too many of them. A *pruned search* reaches the same answer while skipping most of them.

Fix an order on the possible edges and decide them one at a time: first the number of copies of the first edge, then of the second, and so on. These choices form a tree, drawn in Figure 2.6. At the top nothing is decided yet. Each choice for the next edge leads one level down, and at the bottom every possible graph appears exactly once. A point halfway down this tree is a partly built graph. Everything below it is its *branch*: all graphs that agree with it on the edges decided so far. *Pruning* means cutting off a whole branch without looking at any of its graphs. This is only allowed when a rule shows that the branch holds no new best graph. The search uses two such rules.

The first rule uses that adding an edge never lowers connectivity. Every graph in a branch contains the edges placed so far. When a partly built graph already breaks the condition, every graph in its branch breaks it as well, and the branch is dropped.

The second rule bounds the size of every graph in a branch by the edges already placed plus the number of undecided edges times the largest allowed multiplicity. When this bound does not exceed the size of the best feasible graph found so far, nothing in the branch can beat that graph, and the branch is dropped (Figure 2.6). Suppose that 10 edges are placed, 3 are undecided, the largest multiplicity is 1, and a feasible graph with 13 edges is already known. The bound is then 13, and the branch is dropped.

Both rules only drop graphs that are infeasible or cannot beat the best one. A finished pruned search therefore gives the same answer as checking every graph.

Listing 2.5 decides the possible hyperedges one at a time, in a fixed order. For each it tries $1, 2, \ldots$ copies, and stops at the first number of copies that makes the graph infeasible, since more copies stay infeasible. Then it also tries 0 copies. The test on `undecided` in `decide` is the second rule: if even all undecided hyperedges at full multiplicity cannot beat the best graph so far, the branch is dropped. After the time limit the search stops and reports the best graph so far as a lower bound only.

```
168 def exhaustive_search(problem, deadline):
169     hyperedges = possible_hyperedges(problem)
170     cap = largest_multiplicity(problem)
171     best = {"size": 0, "graph": {}}         # the empty graph
172     multiplicity = {}
173 
174     def decide(index, size):
175         if time.time() > deadline:
176             raise OutOfTime
177         undecided = len(hyperedges) - index
178         if size + undecided * cap <= best["size"]:
179             return                          # cannot beat the best
180         if index == len(hyperedges):
181             best["size"], best["graph"] = size, dict(multiplicity)
182             return
183         hyperedge = hyperedges[index]
184         for copies in range(1, cap + 1):
185             multiplicity[hyperedge] = copies
186             if not is_feasible(problem, multiplicity):
187                 break                       # more copies stay infeasible
188             decide(index + 1, size + copies)
189         multiplicity.pop(hyperedge, None)
190         decide(index + 1, size)
191 
192     try:
193         decide(0, 0)
194         return best["size"], best["graph"], True
195     except OutOfTime:
196         return best["size"], best["graph"], False
```

Listing 2.5. The pruned exhaustive search.

2.3.2 Checking one edge more

The pruned search treats two graphs that differ only in the names of their vertices as different graphs. On the vertices $1, 2, 3$, for example, the path $1, 2, 3$ and the path $2, 1, 3$ have different edges, yet one is the other with the names 1 and 2 swapped. Renaming the vertices of a graph gives a graph with the same number of edges and the same connectivities, so the renamed graph

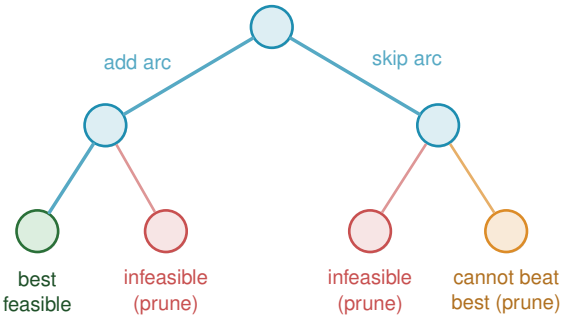


Figure 2.6. Each level of the search decides one arc. Partly built graphs that are already infeasible (red), and branches that cannot beat the best graph found so far (orange), are dropped.

is feasible exactly when the original is. A graph on n vertices has up to $n!$ renamings, which means the pruned search may check the same graph up to $n!$ times under different names.

The second method avoids this. It starts from a feasible graph with L edges, found by the pruned search or by the heuristic search of Section 2.4. Removing an edge creates no new route, so it never makes a feasible graph infeasible. If a feasible graph with more than L edges existed, removing edges from it one at a time would give a feasible graph with exactly $L + 1$ edges. It is therefore enough to check every graph with exactly $L + 1$ edges. If none of them is feasible, L is the maximum. If one of them is feasible, the program keeps it, replaces L by $L + 1$, and checks again.

To list these graphs, each only once up to renaming the vertices, the program uses the toolkit `nauty` of McKay and Piperno [15], to which Prof. Jan Goedgebeur pointed me. Three of its programs cover six of the eight models:

- ★ `geng` lists the simple graphs on n vertices with a given number of edges,
- ★ `directg` takes each graph from `geng` and directs each of its edges one way or both ways, which lists the simple directed graphs,
- ★ `genbg` lists bipartite graphs: graphs whose vertices are split into two sides, with every edge joining the two sides. A hypergraph is the bipartite graph with the n vertices on one side and the hyperedges on the other side, each hyperedge joined to its r vertices. With the option `-z` no two hyperedges have the same vertices, so the hypergraph is simple.

`nauty` has no program that lists directed hypergraphs, so the program does not run this check for them. Their exact values come from the pruned search only.

`nauty` lists simple graphs only. The *support* of a multigraph is the simple graph of the pairs, or r-sets, that carry at least one copy. For a multi model the program takes each support from `nauty` and chooses the number of copies of each of its edges, between 1 and $m - 1$, with $L + 1$ copies in total. A support with k edges carries between k and $k(m - 1)$ copies, so only supports with between $\lceil (L + 1)/(m - 1) \rceil$ and $L + 1$ edges are listed. Renaming the vertices of a multigraph renames its support in the same way, so every multigraph with $L + 1$ edges arises from some listed support. Two choices of copies on the same support can give the same multigraph up to

renaming. This repeats work but does not affect the result. In a simple model the support is the graph itself, with one copy of each edge. The copies are chosen one edge at a time, as in the pruned search: the program tries $1, 2, \ldots$ copies and stops at the first number that makes the graph infeasible, since more copies stay infeasible.

Listing 2.6 builds the `nauty` commands and reads their output one support at a time, so the check can stop at its time limit before all supports are listed. For directed graphs two programs work together: `geng` passes every graph it lists to `directg`, which directs its edges. The program counts the list as complete only if both programs exit normally. If `geng` failed and `directg` did not, `directg` would receive no graphs and exit normally. The list would then look empty, and the check would report a value as exact without having checked anything. Listing 2.7 chooses the copies, and repeats the check while it finds a feasible graph with one edge more.

```
def nauty_commands(problem, size):
    """The nauty programs that list every support that can carry size
    edges, each once up to renaming the vertices. Each entry is one
    pipeline: a list of commands, the output of one feeding the next.
    None for directed hypergraphs, for which nauty has no program."""
    model, n = problem.MODEL, problem.VERTEX_COUNT
    r = model.EDGE_SIZE
    fewest_edges = math.ceil(size / largest_multiplicity(problem))
    most_edges = min(size, len(possible_hyperedges(problem)))
    if model.IS_DIRECTED and r > 2:
        return None
    if fewest_edges > most_edges:
        return []                       # no graph has this many edges
    edges = f"{fewest_edges}:{most_edges}"
    if not model.IS_DIRECTED and r == 2:
        return [["geng", "-q", str(n), edges]]
    if model.IS_DIRECTED:                       # one or two arcs per pair
        pairs = f"{math.ceil(fewest_edges / 2)}:{most_edges}"
        return [["geng", "-q", str(n), pairs],
                ["directg", "-q", "-T", f"-e{edges}"]]
    return [["genbg", "-q", "-z", f"-d0:{r}", f"-D{k}:{r}",
             str(n), str(k)]
            for k in range(fewest_edges, most_edges + 1)]


def read_support(problem, line):
    """One line of nauty output as a list of hyperedges."""
    n = problem.VERTEX_COUNT
    if problem.MODEL.IS_DIRECTED:    # directg: n, arc count, arcs
        numbers = [int(word) for word in line.split()[2:]]
        return [Hyperedge((tail,), tuple(sorted((tail, head))))
                for tail, head in zip(numbers[::2], numbers[1::2])]
    graph = networkx.from_graph6_bytes(line.strip())
    if problem.MODEL.EDGE_SIZE == 2:         # geng: the graph itself
        return [Hyperedge((), tuple(sorted(edge)))
                for edge in graph.edges()]
```

```
    # genbg: first the n vertices, then one node per hyperedge
    return [Hyperedge((), tuple(sorted(graph[x])))
            for x in range(n, graph.number_of_nodes())]


def start_pipeline(pipeline):
    """Starts the programs of a pipeline. If one cannot start, those
    already started are stopped before the error is passed on."""
    processes = []
    try:
        for command in pipeline:
            previous = processes[-1].stdout if processes else None
            processes.append(subprocess.Popen(command, stdin=previous,
                                              stdout=subprocess.PIPE))
            if previous:
                previous.close()    # only the next program reads it
    except BaseException:
        stop_pipeline(processes)
        for process in processes:
            process.stdout.close()
            process.wait()
        raise
    return processes


def stop_pipeline(processes):
    for process in processes:
        if process.poll() is None:
            process.kill()


def supports(problem, size, deadline):
    """Streams the supports from nauty, one at a time. The list counts
    as complete only if every program of every pipeline exits normally,
    so a failed geng is never mistaken for an empty list. nauty can work a
    long time between two lines, so a timer stops it at the deadline."""
    for pipeline in nauty_commands(problem, size):
        processes = start_pipeline(pipeline)
        timer = threading.Timer(max(0, deadline - time.time()),
                                stop_pipeline, [processes])
        timer.start()
        try:
            for line in processes[-1].stdout:
                if time.time() > deadline:
                    raise OutOfTime
                yield read_support(problem, line)
            statuses = [process.wait() for process in processes]
            if any(statuses):
                if time.time() > deadline:
                    raise OutOfTime         # the timer stopped it
```

```
                raise RuntimeError(f"{pipeline} exited with {statuses}")
        finally:
            timer.cancel()
            stop_pipeline(processes)        # the search stopped early
            for process in processes:
                process.wait()
```

Listing 2.6. Listing the supports with `geng`, `directg` and `genbg` from `nauty`.

```
def add_copies(problem, support, size, deadline):
    """Gives every edge of the support between 1 and the largest
    multiplicity copies, size in total. Returns a feasible choice, or
    None if there is none."""
    cap = largest_multiplicity(problem)
    multiplicity = dict.fromkeys(support, 1)

    def choose(index, missing):
        if time.time() > deadline:
            raise OutOfTime
        if missing == 0:
            return dict(multiplicity)
        if missing > (len(support) - index) * (cap - 1):
            return None                     # too few edges left
        edge = support[index]
        for copies in range(1, min(cap, missing + 1) + 1):
            multiplicity[edge] = copies
            if copies > 1 and not is_feasible(problem, multiplicity):
                break               # more copies stay infeasible
            found = choose(index + 1, missing - (copies - 1))
            if found:
                return found
        multiplicity[edge] = 1
        return None

    if not is_feasible(problem, multiplicity):
        return None
    return choose(0, size - len(support))


def feasible_graph_with(problem, size, deadline):
    listed = supports(problem, size, deadline)
    try:
        for support in listed:
            found = add_copies(problem, support, size, deadline)
            if found:
                return found
        return None
    finally:
        listed.close()                      # stops nauty at once


```

```
339 def prove_largest(problem, size, graph, deadline):
340     """Raises size while a feasible graph with one edge more exists.
341     When none exists, size is the maximum."""
342     if nauty_commands(problem, size + 1) is None:
343         return size, graph, False
344     try:
345         while larger := feasible_graph_with(problem, size + 1,
346                                             deadline):
347             size, graph = size + 1, larger
348         return size, graph, True
349     except OutOfTime:
350         return size, graph, False
```

Listing 2.7. Checking every graph with one edge more.

2.4 The heuristic search

For larger n the full search is impossible. Instead the program searches for dense graphs with a *heuristic*: a method that usually finds good graphs quickly, without proving that they are the best. It has two of them, tabu search and random greedy growth. Both work for every model. The setting `HEURISTIC_SEARCH` chooses which one runs, and Section 2.7 compares them.

2.4.1 Tabu search

The first heuristic is *tabu search* [16]. Tabu search is a general strategy that steers a simple search, here adding random copies, so that it does not stop at the first graph where it gets stuck. Such a graph is a *local optimum*, a graph to which no copy can be added although denser graphs may exist. Tabu search moves on from it by also making moves that make the graph smaller.

Tabu search does this as follows. The search starts at the empty graph and moves to another graph by adding or removing one copy of an edge. The choice of move is random. As long as a copy can be added, the search adds one, chosen at random among the copies that can be added. At a local optimum, or when every copy that can be added is on the tabu list below, it removes a random copy instead. Adding that copy back would lead straight back to the local optimum. To prevent this, the search keeps a *tabu list* of the edges changed in its last ten moves, and it may not change an edge on this list. The only exception is a move that gives more edges than any graph seen so far. The search runs until its time limit and returns the densest graph it has seen. It stops earlier only on tiny cases, where every move can be forbidden.

Whether a copy can be added is tested only when needed, as in the pruned search. The search goes through the possible additions in a random order and stops at the first one that is allowed. A copy that is not allowed is remembered in `blocked` and is not tested again until the next removal, because adding copies never lowers a connectivity. Listing 2.8 contains this choice, and Listing 2.9 is the search itself.

```
355 def stays_feasible(problem, multiplicity, hyperedge, connectivity):
356     """One copy more raises a value by at most one, so only pairs
357     already at m-1 need a check."""
```

```
    cap = problem.THRESHOLD_M - 1
    if max(connectivity.values(), default=0) < cap:
        return True
    network = build_flow_network(problem,
                                 apply(multiplicity, (hyperedge, +1)))
    return all(disjoint_routes(network, u, v, stop_at=cap + 1) <= cap
               for (u, v), value in connectivity.items() if value >= cap)


def next_addition(problem, multiplicity, connectivity, blocked, tabu,
                  best_size, generator):
    """A random hyperedge that can get one copy more, or None."""
    cap = largest_multiplicity(problem)
    candidates = [hyperedge for hyperedge in possible_hyperedges(problem)
                  if multiplicity.get(hyperedge, 0) < cap
                  and hyperedge not in blocked]
    generator.shuffle(candidates)
    for hyperedge in candidates:
        if hyperedge in tabu and edge_count(multiplicity) + 1 <= best_size:
            continue                          # tabu, and not a record
        if stays_feasible(problem, multiplicity, hyperedge, connectivity):
            return hyperedge
        blocked.add(hyperedge)                # infeasible until a removal
    return None
```

Listing 2.8. Choosing a random copy that can be added.

```
TABU_LIST_LENGTH = 10


def tabu_search(problem, deadline, seed):
    generator = random.Random(seed)
    multiplicity, blocked, tabu = {}, set(), []
    connectivity = all_connectivities(problem, multiplicity)
    best_size, best_graph = 0, {}
    while time.time() < deadline:
        hyperedge = next_addition(problem, multiplicity, connectivity,
                                  blocked, tabu, best_size, generator)
        change = +1
        if hyperedge is None:                 # a local optimum
            present = [e for e, copies in multiplicity.items()
                       if copies and e not in tabu]
            if not present:
                break                         # no legal move left
            hyperedge, change = generator.choice(present), -1
            blocked = set()                   # a removal can unblock
                ↪ additions
        multiplicity = apply(multiplicity, (hyperedge, change))
        tabu = (tabu + [hyperedge])[-TABU_LIST_LENGTH:]
        connectivity = all_connectivities(problem, multiplicity)
        if edge_count(multiplicity) > best_size:
```

```
            best_size, best_graph = edge_count(multiplicity), multiplicity
    return best_size, best_graph
```

Listing 2.9. Tabu search.

2.4.2 Random greedy growth

Random greedy growth makes the same moves as tabu search until the first local optimum. There it does not remove a copy. It keeps the graph if it is the densest so far, and starts again from the empty graph with new random choices. It repeats this until its time limit and returns the densest graph it has found. Different random choices lead to different local optima, some denser than others, and many fresh starts give many chances to reach a dense one. The two searches therefore differ only at a local optimum: tabu search changes the graph a little, while random greedy growth starts over.

```
def random_greedy_growth(problem, deadline, seed):
    generator = random.Random(seed)
    best_size, best_graph = 0, {}
    while time.time() < deadline:
        multiplicity, blocked = {}, set()
        connectivity = all_connectivities(problem, multiplicity)
        while time.time() < deadline:
            hyperedge = next_addition(problem, multiplicity, connectivity,
                                      blocked, [], best_size, generator)
            if hyperedge is None:
                break                          # a local optimum: restart
            multiplicity = apply(multiplicity, (hyperedge, +1))
            connectivity = all_connectivities(problem, multiplicity)
        if edge_count(multiplicity) > best_size:
            best_size, best_graph = edge_count(multiplicity), multiplicity
    return best_size, best_graph
```

Listing 2.10. Random greedy growth.

2.4.3 Measuring only when needed

A move adds or removes one copy of one hyperedge. Removing a copy never raises a connectivity, since every route of the smaller graph is also a route of the larger one. Adding a copy raises $\lambda(u, v)$ and $\kappa(u, v)$ by at most one for every pair. Edge-disjoint routes share no copy by definition, so after the addition at most one of them uses the new copy, and the others were already there. Separated routes also share no copy, which is why $\kappa \leq \lambda$ holds in every model (Section 1.3.1), so the same holds for κ. This needs the convention. If two routes were only required to share no vertex, they could pass through the same hyperedge (Section 1.3.1), and one new hyperedge could start several new routes at once. The search therefore does not need to run the maximum-flow computation for every candidate move. Removing a copy from a feasible graph keeps it feasible. Adding a copy to a graph with $\lambda^{\max} \leq m - 2$, or with $\kappa^{\max} \leq m - 2$ for separated routes, also keeps it feasible. Also, an addition can only push a pair above $m - 1$ if that pair is already at $m - 1$. For this reason `stays_feasible` in Listing 2.8 answers without the

maximum-flow computation when the graph stays feasible for sure. Otherwise it measures only the pairs whose λ, or κ for separated routes, is already $m - 1$, and stops each measurement once it reaches m. After a move is made, the search measures all pairs once, so the next step again starts from exact values.

2.5 Checking the output without trusting the program

The program is too long to check by hand, so a value it prints is not a proof. We solve this by separating *finding* from *checking*. The program may find a graph in any way, correct or not. Together with the graph it writes a short *certificate*: a list that a reader can check by hand, or with a few lines of independent code, without trusting the program.

Edge-disjoint routes. For every pair u, v, the certificate gives a set of at most $m - 1$ edges (or arcs, or hyperedge copies) whose removal leaves no route from u to v. To check it, remove the set and follow the remaining edges from u: v must not be reached. Then every route from u to v uses an element of the set. Edge-disjoint routes use different elements, so there are at most $m - 1$ of them, and no pair has m edge-disjoint routes. This check uses only that a route through a removed set must use one of its elements. It needs neither Menger's theorem nor a maximum flow. For undirected graphs one tree is enough: any tree on the vertices such that, for each tree edge, at most $m - 1$ edges of G cross it. Take a pair u, v and a tree edge on the tree path between them. Removing this tree edge splits the vertices into two parts, with u and v on different sides. Every u–v route uses an edge of G that crosses between the parts, and there are at most $m - 1$ of these. In this way $n - 1$ counts replace $\binom{n}{2}$ checks. A Gomory–Hu tree (Theorem 1.4) of a feasible graph is such a tree.

Separated routes. For every pair u, v, the certificate gives a set of at most $m - 1$ inner nodes of the incidence graph, vertices or edge copies, whose removal leaves no route from u to v. It is checked in the same way. Separated routes share no inner node, so again each of them uses a different element of the set.

Size and model. The number of edges is counted directly. Every hyperedge must also be one that the model allows: its vertices are among the n vertices, it has r of them, its split into tails and heads is allowed, and its number of copies is a whole number between 0 and the largest multiplicity, which is 1 in a simple model and $m - 1$ otherwise. Without this check, two parallel edges with a correct certificate would pass as a simple graph.

The program writes the certificate from a smallest cut of the flow network (Listing 2.11). A cut uses only arcs of limited capacity: the arc inside a gate, which stands for a hyperedge, and, for separated routes, the arc inside a split vertex. The cut therefore translates directly into removed hyperedges and removed vertices.

```
444 def write_certificate(problem, multiplicity):
445     network = build_flow_network(problem, multiplicity)
446     certificate = {}
447     for u, v in vertex_pairs(problem):
448         _, (source_side, _) = networkx.minimum_cut(
```

```
            network, ("out", u), ("in", v))
        removed_vertices, removed_hyperedges = [], []
        for node in source_side:
            for neighbour in network.successors(node):
                if neighbour in source_side:
                    continue
                if node[0] == "in":
                    removed_vertices.append(node[1])
                else:
                    removed_hyperedges.append(node[1])
        certificate[(u, v)] = (removed_vertices, removed_hyperedges)
    return certificate
```

Listing 2.11. Writing a certificate from smallest cuts.

The certificate checker in Listing 2.12 computes no maximum flow and uses none of the search code. It uses only the list of possible hyperedges, the largest multiplicity, the list of vertex pairs, the edge count and the entry and exit vertices of a hyperedge, from Listings 2.2 and B.2. First `belongs_to_model` checks the size and the model. Then, for each pair, the certificate checker counts the removed elements, with every copy of a removed hyperedge, and then follows the remaining hyperedges from u, entering each only at an entry vertex. If v is never reached, the certificate is correct for this pair. Its correctness can be read off from its 45 lines and these short helpers.

```
def belongs_to_model(problem, claimed_size, multiplicity):
    """Every hyperedge is one the model allows, on vertices 0..n-1 with
    an allowed split into tails and heads, with a whole number of
    copies up to the largest multiplicity, and the copies add up to the
    size."""
    allowed = set(possible_hyperedges(problem))
    cap = largest_multiplicity(problem)
    for hyperedge, copies in multiplicity.items():
        if hyperedge not in allowed:
            return False
        if type(copies) is not int or not 0 <= copies <= cap:
            return False
    return edge_count(multiplicity) == claimed_size


def check_certificate(problem, claimed_size, multiplicity, certificate):
    """No flows and no search code, only following edges."""
    if not belongs_to_model(problem, claimed_size, multiplicity):
        return False
    for u, v in vertex_pairs(problem):
        removed_vertices, removed_hyperedges = certificate[(u, v)]
        if problem.SEPARATION is Separation.EDGE_DISJOINT \
                and removed_vertices:
            return False
        if u in removed_vertices or v in removed_vertices:
            return False
```

```
        size = len(set(removed_vertices)) + sum(
            multiplicity[e] for e in set(removed_hyperedges))
        if size > problem.THRESHOLD_M - 1:
            return False
        reached, frontier = {u}, [u]
        while frontier:
            vertex = frontier.pop()
            for hyperedge, copies in multiplicity.items():
                if copies == 0 or hyperedge in removed_hyperedges:
                    continue
                if vertex not in hyperedge.entry_vertices():
                    continue
                for nxt in hyperedge.exit_vertices():
                    if nxt not in reached and nxt not in removed_vertices:
                        reached.add(nxt)
                        frontier.append(nxt)
        if v in reached:
            return False
    return True
```

Listing 2.12. The independent certificate checker.

Every lower bound the program finds can therefore be checked: the graph is feasible by its certificate, and its size is counted. The program saves every graph with its certificate in the file `witnesses_tabu.json`, or `witnesses_greedy.json` when it runs random greedy growth. A second checker, the *file checker*, checks this file (Listing B.5 in Appendix B). It reads only the file and calls no other function of the program, so it can be read on its own. It checks the model, the size and every certificate as described above. The command `python erdos915.py verify` runs it on `witnesses_tabu.json`, and `python erdos915.py verify witnesses_greedy.json` on the other file. The file checker itself uses only the Python standard library, but the program imports `networkx` at the start, so `networkx` must be installed to run these commands. An upper bound is different. A certificate that no graph on n vertices has more edges would be the whole search, which is too long to check. For this reason the thesis takes no upper bound or exact value from the program. Every upper bound is proved by hand, and every construction in the thesis is checked by hand in Chapter 1 or Appendix A. The exhaustive and heuristic searches of this chapter show which bounds are worth trying to prove, and which constructions to try.

2.6 Running the program

Listing 2.13 puts the parts together. For each case, the driver allocates a nominal sixty-second search budget, in three parts of twenty seconds. The first part goes to the pruned search. If it does not finish, the second part goes to the heuristic search chosen by `HEURISTIC_SEARCH`, tabu search or random greedy growth, in every model. The third part goes to checking one edge more (Section 2.3.2), starting from the densest graph found so far. Directed hypergraphs have no third part, so their budget is split into two parts of thirty seconds. The program checks the time only between operations, so one operation can run past the budget. Writing and checking

the certificate takes extra time. Every result is then certified and checked. The program saves, for each case, the model, the choice between edge-disjoint and separated routes, n and m, the hyperedges of the graph with their multiplicities, its size, the certificate for every vertex pair, and whether the search finished and the certificate passed. It writes these to the file of its search, `witnesses_tabu.json` or `witnesses_greedy.json`, so a reader can recount the edges and check every certificate without rerunning the search, by hand or with the file checker of Listing B.5. The cases are independent, so the program runs them in parallel, one per processor core. For each problem, and each version of a directed hypergraph problem, it prints a table with one row per n and one column per m (Listing B.6). Each entry is the number of edges of the densest feasible graph found. It is marked $*$ when the search finished, and `!` if the certificate failed. A finished search gives an exact value, but only the lower bound is certified (Section 2.5).

The true maximum never decreases when n or m grows: an isolated vertex can be added, and a graph with $\lambda^{\max} \leq m - 1$ also has $\lambda^{\max} \leq m$, and the same holds for $\kappa^{\max}$. The printed entries need not follow this. A larger n or m also gives more candidates, often many more, and every case gets the same sixty-second budget. An unfinished search therefore covers a smaller part of a larger search space, and its entry can be smaller than the entry for a smaller n or m.

```
SECONDS_PER_CASE = 60
HEURISTIC_SEARCH = "tabu"                    # or "greedy", for every model
SEARCHES = {"tabu": tabu_search, "greedy": random_greedy_growth}


def solve(problem, search_name=HEURISTIC_SEARCH):
    has_nauty = nauty_commands(problem, 1) is not None
    part = SECONDS_PER_CASE / (3 if has_nauty else 2)
    size, graph, exact = exhaustive_search(problem, time.time() + part)
    if not exact:
        search = SEARCHES[search_name]
        found_size, found_graph = search(problem, time.time() + part,
                                         seed=1)
        if found_size > size:
            size, graph = found_size, found_graph
        size, graph, exact = prove_largest(problem, size, graph,
                                           time.time() + part)
    certificate = write_certificate(problem, graph)
    checked = check_certificate(problem, size, graph, certificate)
    return size, exact, checked, witness(problem, graph, certificate)


def witness(problem, graph, certificate):
    """The graph and its certificate, in a form a reader can check."""
    def as_list(hyperedge):
        return [list(hyperedge.TAILS), list(hyperedge.MEMBERS)]
    return {
        "hyperedges": [as_list(e) + [copies]
                       for e, copies in sorted(graph.items()) if copies],
        "certificate": [[u, v, removed_vertices,
```

```
                                [as_list(e) for e in removed_hyperedges]]
                            for (u, v), (removed_vertices, removed_hyperedges)
                            in certificate.items()]}
```

Listing 2.13. The driver: a sixty-second budget per case, then a certificate.

Section B.2 prints the full output for $2 \leq m \leq 7$ and $n \leq 7$.

2.7 Comparing the two searches

Section B.2 prints the output of a run with tabu search, and Section B.3 the output of a run with random greedy growth. An entry marked $*$ is the true maximum, so no entry of the other run lies above it. Whether a run proves an entry exact can still differ, because the check of one edge more starts from the graph the heuristic search found, and it has a time limit. For the directed multigraph with internally vertex-disjoint routes at $n = 4$, $m = 5$, both runs find 28 arcs, but only the run with random greedy growth proves that this is the maximum.

The two searches can be pictured with a Galton board, a board of pegs on which a dropped ball bounces left or right at random until it lands in one of the slots at the bottom. Growing a graph from the empty graph by random additions is like dropping a ball, and the local optimum it reaches is the slot where the ball lands. Random greedy growth drops many balls and keeps the best slot. Tabu search drops one ball and then nudges it to nearby slots. Dropping every possible ball would reach every local optimum, and so also the densest graph, but there are far too many balls. Within the time limit the program runs one of the two searches. A run for a single model, or a single n and m, can combine them: many fresh starts, each followed by a short tabu search.

Tabu search finds more edges in most of the differing entries for the general version of directed hypergraphs and directed multihypergraphs, for example 41 against 35 hyperedges for the general directed multihypergraph with hyperedge-disjoint routes at $n = 7$, $m = 7$. Random greedy growth finds more in every differing entry for multigraphs and directed multigraphs with internally vertex-disjoint routes, for example 74 arcs against 68 for the directed multigraph with internally vertex-disjoint routes at $n = 7$, $m = 6$, and in most of the differing entries for the forward and backward versions of directed multihypergraphs. For the forward and backward versions of directed hypergraphs the two searches differ in at most three entries per table, some in favour of each search. In the other problems both searches give the same entries.

One possible explanation for these differences, which this thesis does not test, is how the local optima lie. If they are shallow and close together, a small change leads from one to a denser one, and tabu search can follow these steps. If they are deep and far apart, a small change mostly leads back to the same one, and fresh starts may help to reach others.

Appendix A

Proofs

This appendix contains the proofs that are not in Chapter 1, and the lemmas they need. The definitions are in Chapter 1. New terms are defined where they are first used.

The appendix has three parts. PART I is about undirected graphs, PART II about directed graphs, and PART III about hypergraphs. Part II introduces the two-step budget, a bound on the number of routes with at most one inner vertex between every two vertices, and Part III uses it again for directed hypergraphs.

Part I. Undirected graphs

Graphs that attain Mader's bound, and the vertex problem as a problem about blocks.

A.1 Graphs that attain Mader's bound

Construction A.1 (Hub-and-spokes graph)**.** Let $n \geq m \geq 2$. The graph $H_{m,n}$ has a vertex h and further vertices $v_0, \dots, v_{n-2}$. Join h to every v_i. On $v_0, \dots, v_{n-2}$, add a graph with $\lfloor (m-2)(n-1)/2 \rfloor$ edges in which every vertex has degree at most $m-2$.

This graph can be written down directly. Write $N = n-1$ and $d = m-2$. Since $n \geq m$, $0 \leq d \leq N-1$, and the vertices are $v_0, \dots, v_{N-1}$. Place them around a circle in this order, and take indices modulo N. For each $s = 1, \dots, \lfloor d/2 \rfloor$, join every v_i to v_{i+s}. The numbers $\pm 1, \dots, \pm \lfloor d/2 \rfloor$ are $2\lfloor d/2 \rfloor$ different numbers modulo N, because none is zero and all are smaller than $N/2$ in absolute value. Each v_i is therefore joined to $2\lfloor d/2 \rfloor$ different vertices $v_{i \pm s}$. Every vertex gets degree $2\lfloor d/2 \rfloor$, and the number of edges, half the sum of the degrees, is $N\lfloor d/2 \rfloor$. If d is odd, also join v_i to $v_{i+\lfloor N/2 \rfloor}$ for $i = 0, \dots, \lfloor N/2 \rfloor - 1$. These $\lfloor N/2 \rfloor$ edges share no vertex. They are new: going around the circle, the edges of the first kind join vertices at distance at most $\lfloor d/2 \rfloor$, the new edges join vertices at distance $\lfloor N/2 \rfloor$, and $d \leq N-1$ with d odd gives $\lfloor d/2 \rfloor < \lfloor N/2 \rfloor$. Every degree is therefore at most d, and the number of edges is $N\lfloor d/2 \rfloor$ for even d and $N\lfloor d/2 \rfloor + \lfloor N/2 \rfloor$ for odd d. In both cases this is $\lfloor dN/2 \rfloor = \lfloor (m-2)(n-1)/2 \rfloor$. If $m-1$ divides $n-1$, one can instead take disjoint copies of K_{m-1}. In that case $H_{m,n}$ consists of copies of K_m that share the vertex h.

Theorem A.2 (Extremal graphs exist)**.** *For all $n \geq m \geq 2$ the graph $H_{m,n}$ has $\left\lfloor \frac{m(n-1)}{2} \right\rfloor$ edges and $\lambda^{\max} \leq m-1$.*

Proof. The hub h has $n-1$ edges, and the other edges number $\lfloor (m-2)(n-1)/2 \rfloor$. Since $n-1$ is an integer, the total is

$$(n-1) + \left\lfloor \frac{(m-2)(n-1)}{2} \right\rfloor = \left\lfloor \frac{2(n-1) + (m-2)(n-1)}{2} \right\rfloor = \left\lfloor \frac{m(n-1)}{2} \right\rfloor.$$

Now take two vertices u, v. Only one vertex is not of the form v_i, so at least one of u, v is some v_i. This v_i has one edge to h and at most $m-2$ other edges, so its degree is at most $m-1$. Lemma 1.7 says that $\lambda(u,v)$ is at most the degree of u and at most the degree of v, so $\lambda(u,v) \leq m-1$. □

For $n \geq m$ the bound of Theorem 1.3 is therefore attained. The next theorem describes what equality means for the Gomory–Hu tree.

Theorem A.3 (Equality in the Gomory–Hu bound)**.** *Let G be simple with $\lambda_G^{\max} \leq m-1$ and $|E(G)| = \left\lfloor \frac{m(n-1)}{2} \right\rfloor$, and let T be a Gomory–Hu tree of G.*

(a) If $m(n-1)$ is even, then every tree edge has weight $m-1$, every tree edge is an edge of G, and every edge of G has $\mathrm{dist}_T(e) \le 2$.

(b) If $m(n-1)$ is odd, then exactly one of these three statements fails, and it fails by one: one tree edge has weight $m-2$, or one tree edge is not an edge of G, or one edge of G has $\mathrm{dist}_T(e) = 3$.

Proof. We first recall the proof of Theorem 1.3 in Chapter 1. The tree T has $n-1$ edges, and a tree edge t has weight $c(t)$. An edge e of G crosses $\mathrm{dist}_T(e)$ tree edges: the edges of the path in T between the ends of e. Let E_1 be the set of edges e of G with $\mathrm{dist}_T(e) = 1$. The proof uses one identity and three inequalities.

- The identity is $\sum_e \mathrm{dist}_T(e) = \sum_t c(t)$ (Lemma 1.5).
- Every weight is at most $m-1$, so $\sum_t c(t) \le (m-1)(n-1)$.
- Every edge outside E_1 has $\mathrm{dist}_T(e) \ge 2$, so $\sum_e \mathrm{dist}_T(e) \ge |E_1| + 2\big(|E| - |E_1|\big) = 2|E| - |E_1|$.
- An edge in E_1 joins the two ends of a tree edge. Since G is simple, each tree edge gives at most one such edge, so $|E_1| \le n-1$.

Each inequality has a gap, which is a non-negative integer:

$$S_1 = (m-1)(n-1) - \sum_t c(t), \qquad S_2 = \sum_{e:\, \mathrm{dist}_T(e) \ge 2} \big(\mathrm{dist}_T(e) - 2\big), \qquad S_3 = (n-1) - |E_1|.$$

In words, S_1 is the sum of $(m-1) - c(t)$ over the tree edges, S_2 adds up how far edges go beyond tree distance two, and S_3 is the number of tree edges whose ends are not joined by an edge of G. The edges in E_1 add $|E_1|$ to $\sum_e \mathrm{dist}_T(e)$. The other edges add $2(|E| - |E_1|) + S_2$. With the identity this gives two expressions for the same sum:

$$\sum_e \mathrm{dist}_T(e) \;=\; 2|E| - |E_1| + S_2 \qquad \text{and} \qquad \sum_e \mathrm{dist}_T(e) \;=\; \sum_t c(t) \;=\; (m-1)(n-1) - S_1.$$

We substitute $|E_1| = (n-1) - S_3$ into the first and compare with the second:

$$\begin{aligned} 2|E| - (n-1) + S_2 + S_3 \;&=\; (m-1)(n-1) - S_1, \\ \text{so} \qquad 2|E| \;&=\; m(n-1) - (S_1 + S_2 + S_3). \end{aligned}$$

If $m(n-1)$ is even, then $|E| = m(n-1)/2$, so $S_1 + S_2 + S_3 = 0$. The three gaps are non-negative, so all three are zero. $S_1 = 0$ says that every tree edge has weight $m-1$. $S_3 = 0$ says that every tree edge is an edge of G. $S_2 = 0$ says that every edge of G has $\mathrm{dist}_T(e) \le 2$. This is (a).

If $m(n-1)$ is odd, then $|E| = (m(n-1) - 1)/2$, so $S_1 + S_2 + S_3 = 1$. Exactly one gap is then one, and the other two are zero. $S_1 = 1$ says that one tree edge has weight $m-2$. $S_3 = 1$ says that one tree edge is not an edge of G. $S_2 = 1$ says that one edge of G has $\mathrm{dist}_T(e) = 3$. This is (b). □

For $m = 2$ this shows that the extremal graphs are exactly the trees on the n vertices. Here $m(n-1) = 2(n-1)$ is even, so by (a) every tree edge is an edge of G, which means that G contains T. Both have $n-1$ edges, so $G = T$ is a tree. Conversely, a tree has $n-1$ edges and $\lambda^{\max} = 1$.

Remark A.4 (Forcing and avoiding counts). This thesis maximises the number of edges with no pair joined by m edge-disjoint routes, or by m separated routes in the vertex problems. Sørensen and Thomassen, and the Erdős Problems database, give instead the least number of edges that forces such a pair [3, 13]. Their number is one more than the maximum used here.

For example, Theorem 4 of [13] concerns simple graphs on $n \geq 6$ vertices. It states that every simple graph on n vertices with at least $\lfloor 8n/3 \rfloor - 3$ edges has two vertices joined by 5 internally vertex-disjoint paths, and that no smaller number of edges forces this. The exceptions are $n = 7$ and $n = 12$, where the smallest forcing numbers are 16 and 28. Subtracting one gives the largest number of edges of a simple graph on n vertices without such a pair, which is $k_5(n)$. For $n \geq 14$ this is $\lfloor 8n/3 \rfloor - 4$, the second case of Theorem 1.10. For $6 \leq n \leq 13$ the forcing numbers, the two exceptions included, equal $\lfloor 5(n-1)/2 \rfloor + 1$, so $k_5(n) = \lfloor 5(n-1)/2 \rfloor$, the first case. Theorem 4 does not cover $n = 5$. There the complete graph K_5 has $\kappa^{\max} = 4$, so $k_5(5) = \binom{5}{2} = 10 = \lfloor 5 \cdot 4/2 \rfloor$, which is the first case as well. At $n = 13$ both k_5 and ℓ_5 equal 30, and $k_5(n)$ first exceeds $\ell_5(n)$ at $n = 14$.

The same care is needed for the Bollobás–Erdős conjecture of Section A.1.1. Its forcing number $1 + \binom{m}{2}p$ is for graphs on $1 + (m-1)p$ vertices. Each copy of K_m adds $\binom{m}{2}$ edges but only $m-1$ vertices, so in the notation of this thesis this gives $m/2$ edges per added vertex.

A.1.1 The vertex problem is a block problem

The vertex problem $k_m(n)$ is open for $m \geq 6$. This section reduces it to 2-connected graphs (Theorem A.5).

The words component, cut vertex and k-connected are defined in Chapter 1. A *bridge* is an edge whose removal disconnects its component. A *block* is a maximal connected subgraph that has an edge and no cut vertex of its own. In a simple graph, a block is either a bridge or a maximal 2-connected subgraph.

Call $|E(G)|/(|V(G)| - 1)$ the *rate* of a graph G, with edges counted with multiplicity in a multigraph. It is the number of edges per vertex after the first. The next theorem says that an extremal graph is built from blocks: choose block sizes b_i, where a block of size b_i costs $b_i - 1$ vertices and gives at most $h_m(b_i)$ edges, with total cost at most $n - 1$.

Theorem A.5 (The vertex problem reduces to blocks). *For $b \geq 2$ let*

$$h_m(b) \;:=\; \max\Big\{\, |E(B)| \;:\; B \text{ 2-connected or a single edge},\ |V(B)| = b,\ \kappa_B^{\max} \leq m-1 \,\Big\},$$

with $h_m(b) := -\infty$ when no such B exists. Then for all $m \geq 2$ and $n \geq 2$,

$$k_m(n) \;=\; \max\Big\{ \sum_i h_m(b_i) \;:\; b_i \geq 2,\ \sum_i (b_i - 1) \leq n-1 \Big\}.$$

Consequently $k_m(n) \geq k_m(n_1) + k_m(n_2)$ *whenever* $n = n_1 + n_2 - 1$*, and*

$$c_m \;:=\; \lim_{n\to\infty} \frac{k_m(n)}{n-1} \;=\; \sup_{b\geq 2} \frac{h_m(b)}{b-1}$$

exists, the limit being approached from below. The supremum is finite: Remark A.6 gives $c_m \leq m-1$.

Proof. We use two facts about blocks [4, Chapter 3]. First, every edge lies in exactly one block. Second, a connected graph can be built by adding its blocks one at a time, so that each new block shares exactly one vertex with the blocks before it. A block B therefore adds $|V(B)| - 1$ new vertices, and in a component with c vertices, $\sum_B (|V(B)| - 1) = c - 1$.

Upper bound. Let G be feasible, and let u, v lie in a common block B. Suppose a u–v path leaves B at some vertex c of B. Since the path ends at $v \in B$, it comes back to B, say first at the vertex c'. Let P be the part of the path from c to c'. All inner vertices of P lie outside B. Suppose $c' \neq c$. Then B together with P is connected and has no cut vertex. Removing a vertex of B leaves the rest of B connected, with P still attached at c or at c'. Removing an inner vertex of P leaves B whole, with the two pieces of P attached at c and at c'. This subgraph is larger than B, which contradicts the maximality of B. This forces $c' = c$. The path would then visit c twice, which a path never does. Every u–v path therefore stays in B, and $\kappa_G(u, v) = \kappa_B(u, v) \leq m - 1$. This makes every block feasible. A block is a single edge or 2-connected, so $|E(B)| \leq h_m(|V(B)|)$. Every edge lies in exactly one block, so $|E(G)| = \sum_B |E(B)| \leq \sum_B h_m(|V(B)|)$. Adding the second fact over the components gives $\sum_B (|V(B)| - 1) \leq n - 1$. The block sizes of G are therefore one of the choices in the maximum, and $|E(G)|$ is at most that maximum.

Lower bound. Take blocks $B_1, \ldots, B_t$ with sizes b_i, with $\sum(b_i - 1) \leq n - 1$, each feasible with $h_m(b_i)$ edges. Glue them at one common vertex, and keep them disjoint otherwise. This graph has $1 + \sum(b_i - 1) \leq n$ vertices. Add isolated vertices to reach n. Its blocks are the B_i, so by the argument above a pair inside one block keeps its value of κ, which is at most $m - 1$. Two vertices in different blocks are separated by removing the common vertex, so $\kappa \leq 1 \leq m - 1$. The graph is therefore feasible, with $\sum_i h_m(b_i)$ edges.

Gluing. Take an extremal graph on n_1 vertices and one on n_2 vertices, and glue them at one vertex. Pairs inside one part keep their value of κ, as above, and pairs in different parts have $\kappa \leq 1$. The result is feasible on $n_1 + n_2 - 1$ vertices, so $k_m(n) \geq k_m(n_1) + k_m(n_2)$ for $n = n_1 + n_2 - 1$.

The limit. Write $a_n = k_m(n)/(n-1)$ and $s = \sup_{b\geq 2} h_m(b)/(b-1)$. In the formula for $k_m(n)$, every block of size b_i gives at most $s\,(b_i - 1)$ edges. Summing gives $k_m(n) \leq s \sum_i (b_i - 1) \leq s\,(n-1)$, that is, $a_n \leq s$ for every n.

A lemma of Fekete [17] says: if numbers $x_1, x_2, \ldots$ satisfy $x_{i+j} \geq x_i + x_j$ for all i, j, then x_t/t tends to $\sup_t x_t/t$. Take $x_t = k_m(t+1)$. Gluing with $n_1 = i + 1$ and $n_2 = j + 1$ gives $x_{i+j} \geq x_i + x_j$. Since also $x_t/t = a_{t+1}$, the sequence a_n tends to $\sup_n a_n$. This supremum is at most s, since every $a_n \leq s$. It is also at least s: one block of size b is a feasible graph on b

vertices, so $a_b \geq h_m(b)/(b-1)$ for every b. Hence a_n tends to s. Since $a_n \leq s$ for every n, it approaches s from below. □

We note three consequences.

First, at $m = 3$ the reduction gives Theorem 1.9 again. A block with $\kappa^{\max} \leq 2$ is an edge or a cycle. To see this, let B be a 2-connected block that is not a cycle. Since B is 2-connected, it contains a cycle C. Without one, B would be a tree on at least three vertices, and every vertex of degree at least two in such a tree is a cut vertex. Since B is connected and is not C, it has an edge wz that is not on C, with z on C. If w is on C, let P be this edge. Otherwise, $B - z$ is still connected, so it has a path from w to C. Let y be the first vertex of C on it, and let P be the path $z, w, \ldots, y$. In both cases P joins two different vertices x, y of C and has no inner vertex on C. The cycle C gives two internally vertex-disjoint x–y paths, and P is a third. This gives $\kappa(x, y) \geq 3$, which breaks the condition. It follows that $h_3(2) = 1$ and $h_3(b) = b$ for $b \geq 3$, where b is the length of the cycle. The rate $h_3(b)/(b-1)$ is 1 at $b = 2$, $3/2$ at $b = 3$ and $b/(b-1) < 3/2$ for $b \geq 4$. Every block thus gives at most $3/2$ edges per vertex it costs, which bounds $k_3(n)$ by $3(n-1)/2$. As $k_3(n)$ is an integer, $k_3(n) \leq \lfloor 3(n-1)/2 \rfloor$. The bound is reached by $\lfloor (n-1)/2 \rfloor$ triangles, together with one extra edge when $n - 1$ is odd. They cost $n - 1$ vertices and give $3\lfloor (n-1)/2 \rfloor + ((n-1) \bmod 2) = \left\lfloor \frac{3(n-1)}{2} \right\rfloor$ edges.

Second, at $m = 4$, Theorem 1.9 gives $h_4(b) = 2(b-1)$ for every $b \geq 4$. The rate $h_4(b)/(b-1)$ is then 2 for $b \geq 4$, against 1 at $b = 2$ and $3/2$ at $b = 3$. Every block thus gives at most 2 edges per vertex it costs, which bounds $k_4(n)$ by $2(n-1)$. For $n \geq 4$ a single block of size n reaches this bound, giving $k_4(n) = 2(n-1)$. This uses Theorem 1.9, so it is not a new proof. It does show the extremal block: the hub-and-spokes graph $H_{4,b}$ of Construction A.1, with the graph on $v_0, \ldots, v_{b-2}$ chosen to be one cycle, so that $H_{4,b}$ is a hub joined to all vertices of a cycle. It is a single block with $2(b-1)$ edges.

Third, the Bollobás–Erdős construction, copies of K_m sharing one vertex, is the case $b = m$: $h_m(m) = \binom{m}{2}$, with rate $m/2$. Their conjecture says that $c_m = m/2$, so that no block beats K_m. A *bouquet* is a set of blocks that share one vertex and are otherwise disjoint. A bouquet with blocks B_i of sizes b_i has rate

$$\frac{\sum_i |E(B_i)|}{\sum_i (b_i - 1)} = \sum_i \frac{b_i - 1}{\sum_j (b_j - 1)} \cdot \frac{|E(B_i)|}{b_i - 1},$$

a weighted average of the rates of its blocks. For any graph G, the proof of Theorem A.5 gives $|E(G)| = \sum_B |E(B)|$ and $|V(G)| - 1 \geq \sum_B (|V(B)| - 1)$. The rate of G is therefore at most this weighted average for its own blocks, and some block has a rate at least the rate of G. In particular, a counterexample must contain a block with rate above $m/2$. A single edge has rate $1 \leq m/2$, so this block is 2-connected.

The conjecture is false. Sørensen and Thomassen [13] showed that for each fixed m,

$$c_m \geq \frac{m(m-1) - 2}{2m - 3} = \frac{m}{2} + \frac{1}{4} + O(1/m).$$

This is larger than $m/2$ for every $m \geq 5$, because

$$2\big(m(m-1)-2\big) - m(2m-3) \;=\; m-4.$$

At $m = 5$, Theorem 1.10 gives $k_5(14) = \lfloor 112/3 \rfloor - 4 = 33$ edges on 14 vertices. Its rate is $33/13 > 5/2$, so by the bouquet argument above a simple graph on 14 vertices with 33 edges and $\kappa^{\max} \leq 4$ contains a block with rate above $5/2$. In summary, $c_m = m/2$ for $m \leq 4$, $c_5 = 8/3$ by Theorem 1.10, and $m/2 < c_m \leq m-1$ for $m \geq 6$ by the next remark.

Remark A.6 (Feasible graphs are $(m-1)$-degenerate). A theorem of Mader [18, 19] states that every simple graph of minimum degree at least $d \geq 2$ contains two adjacent vertices joined by d internally vertex-disjoint paths. Consequently, every graph feasible for the vertex problem has a vertex of degree at most $m-1$. Since taking a subgraph cannot increase local vertex-connectivity, every non-empty subgraph of a feasible graph has such a vertex. A graph is *d-degenerate* when every non-empty subgraph has a vertex of degree at most d. Every feasible graph is therefore $(m-1)$-degenerate.

Delete from G a vertex of degree at most $m-1$, then such a vertex from what is left, and continue until no vertex remains. Number the vertices $x_n, x_{n-1}, \ldots, x_1$ in the order of deletion. When x_i is deleted, only $x_1, \ldots, x_i$ are left. At that moment x_i has at most $m-1$ neighbours among $x_1, \ldots, x_{i-1}$, and at most $i-1$ because there are only $i-1$ of them. Every edge is counted once, at its end with the larger number. This gives

$$\begin{aligned} k_m(n) \;\leq\; \sum_{i=1}^{n} \min(i-1,\, m-1) \;&=\; \binom{m-1}{2} + (n-m+1)(m-1) \\ &=\; (m-1)n - \binom{m}{2} \qquad (n \geq m-1). \end{aligned}$$

Hence $c_m \leq m-1$.

Part II. Directed graphs

The directed constructions, the bounds for $m \geq 3$, and the extremal directed multigraphs.

A.2 The directed constructions

Three constructions give the directed lower bounds. Figure 1.8 shows the first two. Here we define them and check their connectivity.

Construction A.7 (Directed hub)**.** For $n \geq m \geq 2$, take a hub h and label the other $N = n-1$ vertices $0, \ldots, N-1$. Add the $2N$ hub arcs $h \to i$ and $i \to h$, and on the non-hub vertices the $(m-2)N$ ring arcs $i \to i+j \bmod N$ for $j = 1, \ldots, m-2$. The total is $m(n-1)$ arcs and every non-hub vertex has $d^+ = d^- = m-1$.

The ring arcs at i go to $m-2$ different vertices, since $m-2<N$. With the hub arcs, every non-hub vertex has out-degree and in-degree $1+(m-2)=m-1$. Every ordered pair (u,v) contains a vertex other than the hub. Lemma 1.7 says that $\lambda(u,v)\le d^+(u)$ and $\lambda(u,v)\le d^-(v)$. Hence $\lambda(u,v)\le m-1$, that is, $\lambda^{\max}\le m-1$. At $m=2$ there are no ring arcs, and we get the *bidirected star*.

Construction A.8 (One-way bipartite directed graph)**.** Split the vertices into parts A and B of sizes $\lfloor n/2\rfloor$ and $\lceil n/2\rceil$, and include every arc from A to B and none the other way. The total is $\lfloor n^2/4\rfloor$ arcs.

No arc leaves B and no arc enters A. Every route therefore has length one, and $\lambda^{\max}=1$.

Construction A.9 (Augmented bipartite directed graph)**.** For $n\ge m\ge 2$, set $|B|=\lceil(n+m-2)/2\rceil$ and $|A|=n-|B|$. Number the vertices of B as $b_1,\dots,b_{|B|}$, and read the indices modulo $|B|$. Include every arc from A to B, and inside B give each b_j the $m-2$ in-arcs from $b_{j-1},\dots,b_{j-(m-2)}$. The total is $\lfloor(n+m-2)^2/4\rfloor$ arcs.

Since $n\ge m$, we have $|B|\ge(2m-2)/2=m-1$, so the vertices $b_{j-1},\dots,b_{j-(m-2)}$ are different from each other and from b_j.

Lemma A.10 (The augmented bipartite directed graph is feasible)**.** *For $m\ge 2$ and $n\ge m$, the directed graph of Construction A.9 satisfies $\lambda^{\max}=m-1$ and has $\lfloor(n+m-2)^2/4\rfloor$ arcs. Hence $\ell_m^{\mathrm{dir}}(n)\ge\lfloor(n+m-2)^2/4\rfloor$.*

Proof. Counting. Each vertex of B receives one arc from every vertex of A and $m-2$ arcs from inside B. No other arcs exist, so the number of arcs is $|B|\,(|A|+m-2)$. Write $N=n+m-2$. Then $|B|=\lceil N/2\rceil$ and $|A|+m-2=N-|B|=\lfloor N/2\rfloor$. For every integer N, $\lfloor N/2\rfloor\lceil N/2\rceil=\lfloor N^2/4\rfloor$. When $N=2j$ both sides equal j^2, and when $N=2j+1$ both equal j^2+j.

Pairs from A to B. Take $a\in A$ and $b\in B$. The direct arc $a\to b$ is one route. For each of the $m-2$ in-neighbours b' of b in B, the route $a\to b'\to b$ is another. These $m-1$ routes use different arcs. Now take any a–b route. No arc enters A, so the route never comes back to A. Its last arc must be $a\to b$ or one of the $m-2$ arcs into b from B. Removing these $m-1$ arcs therefore separates a from b. By Menger's theorem, $\lambda(a,b)=m-1$.

Other pairs. For $b_i,b_j\in B$, a route from b_i stays in B, since no arc leaves B, which makes its last arc one of the $m-2$ arcs into b_j from B, and $\lambda(b_i,b_j)\le m-2$. No arc leaves B, so there is no route from B to A. No arc goes from A to A, and a route from A never comes back to A, so there is no route between two vertices of A. This gives $\lambda^{\max}=m-1$. □

For example, take $m=3$ and $n=10$. Then $|B|=6$ and $|A|=4$. There are 24 arcs from A to B, and 6 arcs forming a cycle inside B. Each pair a_i,b_j has the direct arc and one detour, so $\lambda^{\max}=2$. This gives 30 arcs, more than the $\max(m(n-1),\lfloor n^2/4\rfloor)=27$ of the first two

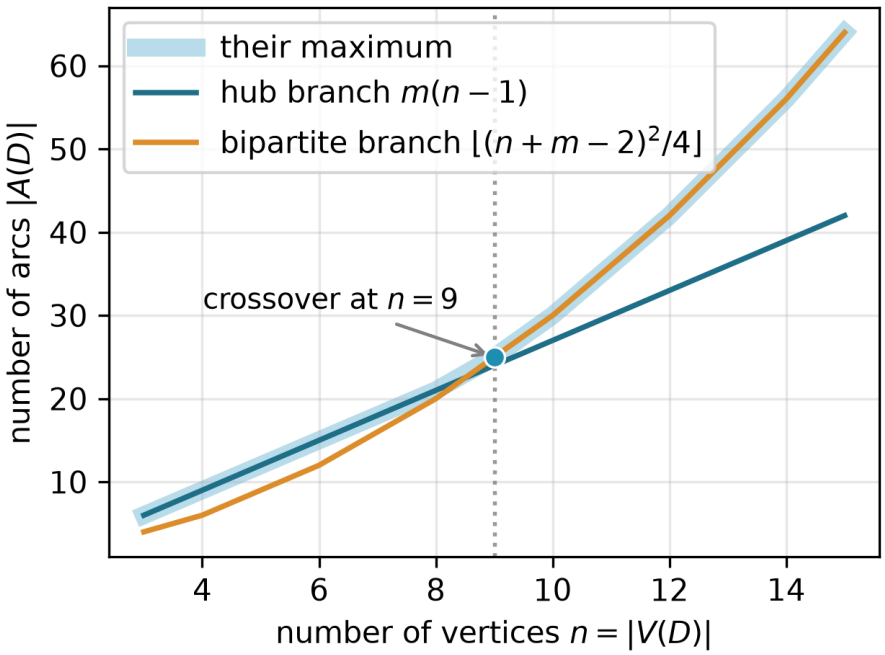


Figure A.1. The arc counts of the hub and of the augmented bipartite directed graph at $m = 3$, and their maximum. These are lower bounds only.

constructions. The gain comes from making B larger than half of the vertices. For $n \geq m$ the constructions therefore give

$$\ell_m^{\mathrm{dir}}(n) \;\geq\; \max\Bigl(m(n-1),\; \Bigl\lfloor \frac{(n+m-2)^2}{4} \Bigr\rfloor\Bigr).$$

For fixed m and large n the augmented bipartite directed graph has more arcs (Figures A.1 and A.2).

A.3 Simple directed graphs beyond the first threshold

A.3.1 The two-step budget

Definition A.11 (Two-step budget)**.** Let D be a simple directed graph and $C \geq 1$. For $u \neq v$, let $p_2(u, v)$ be the number of vertices $x \notin \{u, v\}$ with arcs $u \to x$ and $x \to v$. Such an x is the *midpoint* of the route $u \to x \to v$. We call D *C-budgeted* if every ordered pair (u, v) of distinct vertices satisfies

$$\mathbf{1}_{(u,v)\in A(D)} + p_2(u, v) \;\leq\; C.$$

Lemma A.12 (Feasible directed graphs are budgeted)**.** *Let D be a simple directed graph and $m \geq 2$. If $\lambda_D^{\max} \leq m - 1$, or if $\kappa_D^{\max} \leq m - 1$, then D is $(m-1)$-budgeted.*

Proof. Fix an ordered pair (u, v). Take the arc $u \to v$, if it exists, and the route $u \to x \to v$ for each of the $p_2(u, v)$ midpoints x. Two routes with different midpoints use different arcs, and none of them uses the arc $u \to v$, which makes the routes arc-disjoint. Each two-step route has one

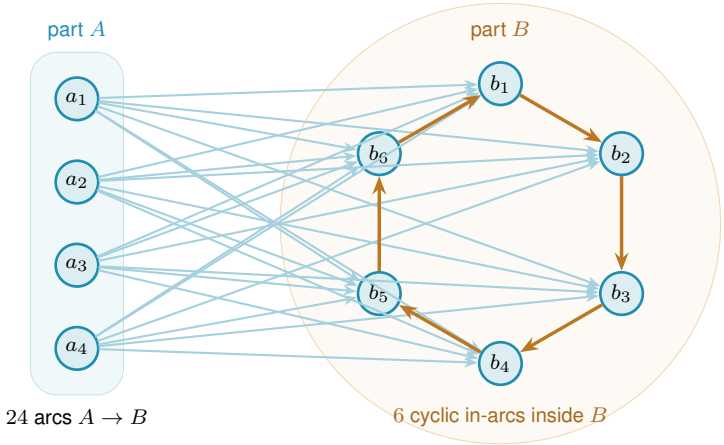


Figure A.2. The augmented bipartite directed graph of Construction A.9 at $m = 3$, $n = 10$. All arcs between the parts go from A to B. Inside B, each vertex gets one arc from the previous vertex on a cycle. A pair a_i, b_j has the direct arc and one detour, so $\lambda^{\max} = 2$. The directed graph has $24 + 6 = 30$ arcs, against 27 for the hub.

inner vertex, its midpoint, and the arc has none. The routes are therefore also internally vertex-disjoint. This makes their number at most $\lambda_D(u, v)$ and at most $\kappa_D(u, v)$, and either condition gives at most $m - 1$. □

The next lemma turns the budget into a statement about degrees. A vertex with large in-degree and large out-degree is the midpoint of many two-step routes, and the budget limits these.

Lemma A.13 (The budget, read on degrees)**.** *Every C-budgeted simple directed graph D on n vertices satisfies*

$$\sum_{x \in V} d^+(x)\, d^-(x) \;\le\; C\, n(n-1).$$

Proof. Count the walks $u \to x \to v$ of length two. Choosing the middle vertex x, an in-neighbour u and an out-neighbour v gives $\sum_x d^-(x)\, d^+(x)$ walks. The walks with $u \neq v$ number $\sum_{u \neq v} p_2(u, v)$. A walk with $u = v$ is fixed by its first arc $u \to x$, so there are at most $|A(D)|$ of them. Summing the budget over all ordered pairs gives $|A(D)| + \sum_{u \neq v} p_2(u, v) \le C\, n(n-1)$. So

$$\sum_{x} d^+(x)\, d^-(x) \;\le\; \sum_{u \neq v} p_2(u, v) + |A(D)| \;\le\; C\, n(n-1).$$ □

Lemma A.14 (Making a directed graph one-way bipartite)**.** *Every simple directed graph D on n vertices can be made* one-way bipartite*, so that all remaining arcs go from one part to the other, by deleting at most $\sum_x \min(d^+(x), d^-(x))$ arcs. So*

$$|A(D)| \;\le\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; \sum_{x \in V} \min\bigl(d^+(x), d^-(x)\bigr).$$

Proof. Let S be the set of vertices x with $d^+(x) \ge d^-(x)$, and let T be the set of the other vertices. Delete every arc into a vertex of S and every arc out of a vertex of T. At a vertex x of S this deletes

its $d^-(x) = \min(d^+(x), d^-(x))$ in-arcs. At a vertex of T it deletes its $d^+(x) = \min(d^+(x), d^-(x))$ out-arcs. An arc may be deleted at both of its ends, so in total at most the sum above is deleted. An arc $a \to b$ survives only if $a \notin T$ and $b \notin S$, that is, $a \in S$ and $b \in T$. The remaining arcs therefore all go from S to T. Since $|S| + |T| = n$, there are at most $|S|\,|T| \le \lfloor n^2/4 \rfloor$ of them. □

Lemma A.15 (The arc bound for budgeted directed graphs)**.** *Every C-budgeted simple directed graph D on $n \ge 2$ vertices satisfies*

$$|A(D)| \;\le\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; C(n-1).$$

Proof. Let v be a vertex of largest out-degree, and write $\Delta = d^+(v)$. Let $N^+(v)$ be the set of the Δ vertices x with $v \to x$, and let $\delta(v)$ be the number of those x that also have $x \to v$, so that $\delta(v) \le \Delta$.

First we count the routes of length two that start at v. We count the pairs (x, w) with $v \to x$, $x \to w$ and $w \ne v$ in two ways. Fixing $w \ne v$ first gives the midpoints of the routes $v \to x \to w$. The conditions $x \ne v$ and $x \ne w$ hold automatically, since $v \to x$ and $x \to w$. This count is $\sum_{w \ne v} p_2(v, w)$. Fixing $x \in N^+(v)$ first gives the $d^+(x)$ out-neighbours of x, minus one exactly when $x \to v$, which gives $\sum_{x \in N^+(v)} d^+(x) - \delta(v)$ in total. Therefore

$$\sum_{w \ne v} p_2(v, w) \;=\; \sum_{x \in N^+(v)} d^+(x) \;-\; \delta(v).$$

Adding the budget over the $n-1$ pairs (v, w) with $w \ne v$ gives

$$\Delta \;+\; \sum_{x \in N^+(v)} d^+(x) \;-\; \delta(v) \;\le\; C(n-1),$$

since v has Δ out-arcs. With $\delta(v) \le \Delta$ this gives $\sum_{x \in N^+(v)} d^+(x) \le C(n-1)$.

Now we count the arcs by their tails, since every arc has one tail. Split the vertices into v, the set $N^+(v)$, and the remaining $n - 1 - \Delta$ vertices, each of out-degree at most Δ. So

$$|A(D)| \;\le\; \Delta \;+\; C(n-1) \;+\; (n-1-\Delta)\,\Delta \;=\; C(n-1) \;+\; \Delta(n-\Delta).$$

Finally $\Delta(n-\Delta) = n^2/4 - (\Delta - n/2)^2 \le n^2/4$, and $\Delta(n-\Delta)$ is an integer, so $\Delta(n-\Delta) \le \lfloor n^2/4 \rfloor$. □

We can now prove Theorem 1.21.

Theorem 1.21 (restated). *For all $m \ge 2$ and $n \ge 2$, every simple directed graph D on n vertices with $\lambda_D^{\max} \le m-1$, or with $\kappa_D^{\max} \le m-1$, satisfies*

$$|A(D)| \;\le\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; (m-1)(n-1),$$

so for each fixed $m \geq 3$*, both* $\ell_m^{\mathrm{dir}}(n)$ *and* $k_m^{\mathrm{dir}}(n)$ *equal* $n^2/4 + \Theta_m(n)$*.*

Proof of Theorem 1.21. Upper bound. By Lemma A.12, a simple directed graph with $\lambda_D^{\max} \leq m-1$ or with $\kappa_D^{\max} \leq m-1$ is $(m-1)$-budgeted. Lemma A.15 with $C = m-1$ then gives the stated bound.

Lower bound. The directed graph of Construction A.9 has $\lambda^{\max} \leq m-1$ (Lemma A.10). Since $\kappa \leq \lambda$, also $\kappa^{\max} \leq m-1$. For $n \geq m$ this gives

$$k_m^{\mathrm{dir}}(n) \;\geq\; \ell_m^{\mathrm{dir}}(n) \;\geq\; \left\lfloor \frac{(n+m-2)^2}{4} \right\rfloor \;=\; \frac{n^2}{4} + \frac{(m-2)n}{2} + O_m(1).$$

For $m \geq 3$ the term $(m-2)n/2$ is positive. Both functions are therefore $n^2/4$ plus a term that lies between two positive constant multiples of n. □

The linear coefficient is not known. The construction gives $(m-2)n/2$, and Theorem 1.21 allows $(m-1)n$. A theorem of Huang and Lyu [9, Theorem 2], with $t = m-1 \geq 2$, also gives the linear term $(m-1)n$, for $n \geq \max(t^3+4t^2+3t+4,\ 17t^2/2+30t+27)$. It applies because a feasible directed graph has at most $m-1$ paths of length two between any two vertices (Lemma A.12). Theorem 1.21 holds for every n.

The next proposition is about the shape of a feasible directed graph. After deleting $O(\sqrt{m}\, n^{3/2})$ arcs, all remaining arcs go from one part to the other. The parts need not be balanced. For fixed m this error is larger than the linear error of Theorem 1.21.

Proposition A.16 (Almost all arcs go one way)**.** *Let* D *be a simple directed graph on* $n \geq 2$ *vertices with* $\lambda_D^{\max} \leq m-1$*. Then deleting at most* $\sqrt{m}\, n^{3/2}$ *arcs leaves* D *a subgraph of a one-way bipartite directed graph, and*

$$|A(D)| \;\leq\; \left\lfloor \frac{n^2}{4} \right\rfloor \;+\; \sqrt{m}\, n^{3/2}.$$

Proof. By Lemma A.12, D is $(m-1)$-budgeted. We use $\min(a,b) \leq \sqrt{ab}$, the Cauchy–Schwarz inequality $\sum_x \sqrt{a_x} \leq \sqrt{n \sum_x a_x}$, and Lemma A.13:

$$\begin{aligned} \sum_{x \in V} \min\bigl(d^+(x), d^-(x)\bigr) \;&\leq\; \sum_{x \in V} \sqrt{d^+(x)\, d^-(x)} \;\leq\; \sqrt{n \sum_{x \in V} d^+(x)\, d^-(x)} \\ &\leq\; \sqrt{n \cdot (m-1)\, n(n-1)} \;=\; n\sqrt{(m-1)(n-1)} \;\leq\; \sqrt{m}\, n^{3/2}. \end{aligned}$$

Lemma A.14 now gives both statements. □

A.4 The extremal directed multigraphs

We recall the result of Chapter 1.

Theorem 1.22 (restated). *For all $n \geq 2$ and $m \geq 2$, the largest number of arcs of a directed multigraph on n vertices with $\lambda^{\max} \leq m-1$ is*

$$L_m^{\mathrm{dir}}(n) \;=\; (m-1)\,M(n), \qquad M(n) = \max\Big(2(n-1),\; \Big\lfloor \frac{n^2}{4} \Big\rfloor\Big).$$

Its proof of the upper bound goes as follows. Take a reachability skeleton R_1 of D with at most $M(n)$ arcs (Lemma 1.14), and delete one copy of each of its arcs. By Lemma 1.25, this lowers $\lambda^{\max}$ by at least one. Repeat this with skeletons $R_2, R_3, \ldots$ of what is left. After deleting $R_1, \ldots, R_{m-1}$, we have $\lambda^{\max} = 0$, so no arc is left. Hence $|A(D)| = \sum_i |A(R_i)| \leq (m-1)M(n)$.

At $m = 2$ a feasible directed multigraph has no parallel arcs, since two copies of an arc are two arc-disjoint routes, and Theorem 1.22 at $m = 2$ is therefore Theorem 1.11.

A.4.1 The extremal multigraphs for large n

Theorem 1.22 gives the value, but not the extremal multigraphs. This subsection describes them for large n. The idea is to look at the proof of Lemma 1.14 in the case of equality. Equality fixes the number of strongly connected components, then the shape of the skeleton, and then the multigraph.

Lemma A.17 (What equality forces)**.** *Let $m \geq 2$ and $n \geq 8$, so that $\lfloor n^2/4 \rfloor > 2(n-1)$ and $M(n) = \lfloor n^2/4 \rfloor$, and let D be a directed multigraph on n vertices with $\lambda_D^{\max} \leq m-1$ and $|A(D)| = (m-1)\lfloor n^2/4 \rfloor$. Then*

(1) D is acyclic*: it has no directed cycle, and*

(2) D has a reachability skeleton R from which no arc can be removed without changing who reaches whom, and which, with directions ignored, is the complete bipartite graph $K_{\lceil n/2 \rceil, \lfloor n/2 \rfloor}$.

Proof. As recalled above, the proof of Theorem 1.22 deletes skeletons $R_1, \ldots, R_{m-1}$, built as in Lemma 1.14, and then no arc is left, which gives $|A(D)| = \sum_i |A(R_i)|$. Each $|A(R_i)| \leq M(n)$, and the sum is $(m-1)M(n)$, so $|A(R_i)| = M(n) = \lfloor n^2/4 \rfloor$ for every i. Let $R := R_1$.

Let q be the number of strongly connected components of D. In the proof of Lemma 1.14, $|A(R)| \leq f(q) = 2(n-q) + \lfloor q^2/4 \rfloor$. This gives $f(q) \geq \lfloor n^2/4 \rfloor = f(n)$. We show that this forces $q = n$. Since $n \geq 8$, $f(1) = 2(n-1) < \lfloor n^2/4 \rfloor = f(n)$. The differences $f(q+1) - f(q)$ never decrease as q grows, so between 1 and n, the values of f stay on or below the straight line from $f(1)$ to $f(n)$. Since $f(1) < f(n)$, this line goes up, so it is below $f(n)$ at every $q < n$. This gives $f(q) < f(n)$ for $q < n$, and therefore $q = n$: every strongly connected component is a single vertex. A directed cycle would put its vertices in one component, so D has no directed cycle. This is (1).

With $q = n$, each component has one vertex, so the proof of Lemma 1.14 chooses no arcs inside components. The skeleton R then consists of the arcs chosen for Q, and Q is R itself, with $|A(Q)| = \lfloor n^2/4 \rfloor$. With directions ignored, Q has no triangle. A graph on n vertices with no

triangle and with $\lfloor n^2/4 \rfloor$ edges is $K_{\lceil n/2 \rceil, \lfloor n/2 \rfloor}$ (the case of equality in Mantel's theorem [14]). No arc of Q can be removed without changing reachability, by the choice of Q. This is (2). □

The next lemma uses that no arc of R can be removed.

Lemma A.18 (A minimal bipartite skeleton has no long path)**.** *Let R be as in Lemma A.17, with parts A and B. Then R contains no directed path with three arcs.*

Proof. Suppose $u \to v \to w \to x$ is such a path, with four different vertices. Every arc of R joins A and B, so the path alternates between the parts. It has three arcs, so u and x lie in different parts. The underlying graph of R is complete bipartite, so R has an arc between u and x. It cannot be $x \to u$, since that would close a directed cycle, and D is acyclic. The arc $u \to x$ is therefore in R. But u also reaches x along the path, so $u \to x$ could be removed without changing reachability. This is a contradiction. □

Theorem A.19 (The extremal directed multigraphs)**.** *Let $m \geq 2$ and $n \geq \max(8, 2m)$, so that $M(n) = \lfloor n^2/4 \rfloor$. If D is a directed multigraph on n vertices with $\lambda_D^{\max} \leq m-1$ and $|A(D)| = L_m^{\mathrm{dir}}(n) = (m-1)\lfloor n^2/4 \rfloor$, then*

$$D \in \left\{ (m-1) \cdot B_{\lceil n/2 \rceil, \lfloor n/2 \rfloor},\ (m-1) \cdot B_{\lfloor n/2 \rfloor, \lceil n/2 \rceil} \right\},$$

where $B_{a,b}$ has all arcs from its part of size a to its part of size b, each once, and $(m-1) \cdot B_{a,b}$ takes every arc $m-1$ times as in Lemma 1.23. These two directed graphs are the same up to renaming the vertices when n is even, and reverses of each other when n is odd.

Proof. Take R, A and B from Lemma A.17, and write $\alpha = |A| \geq \beta = |B| = \lfloor n/2 \rfloor$.

First we show that R has no directed path with two arcs. Suppose R has a path $a \to b \to a'$ with $a, a' \in A$ and $b \in B$. Suppose a had an incoming arc $b_0 \to a$. Then $b_0 \neq b$, since $b \to a \to b$ would be a directed cycle, and D has none. Also $b_0 \neq a'$, since they lie in different parts. Then $b_0 \to a \to b \to a'$ would be a path with three arcs, which Lemma A.18 forbids. Hence a has no incoming arc. With directions ignored, R is complete bipartite, so a is joined to every $b'' \in B$. All these arcs leave a, so $a \to b''$ for every $b'' \in B$. In the same way, a' has no outgoing arc, so $b'' \to a'$ for every $b'' \in B$. The β routes $a \to b'' \to a'$ use different arcs, and they lie in D, since R is part of D. This gives $\lambda_D(a, a') \geq \beta = \lfloor n/2 \rfloor \geq m$, since $n \geq 2m$. This breaks the condition $\lambda_D^{\max} \leq m-1$. A path $b \to a \to b'$ with $b, b' \in B$ is excluded in the same way, with the $\alpha \geq \beta \geq m$ routes through A. This is the only place where we use $n \geq 2m$.

No vertex of R therefore has both an incoming and an outgoing arc. Take $a \in A$. It is joined to every vertex of B. If all arcs at a go out, then every $b \in B$ has an incoming arc, so it has no outgoing arc. All arcs then go from A to B. If all arcs at a come in, all arcs go from B to A in the same way. From now on, call the part that the arcs leave A, and the other part B.

Next, every arc of D has the same ordered endpoints as an arc of R. Every path in R has one arc, so u reaches x in R only when $u \to x$ is an arc of R. As shown above, these are exactly

the pairs (u, x) with $u \in A$ and $x \in B$. If D has an arc $u \to x$, then u reaches x in D, so also in R. Hence $u \to x$ is an arc of R.

Parallel arcs are arc-disjoint routes, so every multiplicity is at most $m - 1$. All arcs of D therefore go from A to B. There are $|A|\,|B| \le \lfloor n^2/4 \rfloor$ such pairs, so $|A(D)| \le (m-1)|A|\,|B| \le (m-1)\lfloor n^2/4 \rfloor$. Equality holds, so every pair in $A \times B$ has multiplicity $m-1$, and $|A|\,|B| = \lfloor n^2/4 \rfloor$. The product $|A|\,|B|$ with $|A| + |B| = n$ equals $\lfloor n^2/4 \rfloor$ only when the parts have sizes $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$. □

Without the hypothesis $n \ge 2m$, the following still holds for $n \ge 8$. Every vertex of an extremal multigraph has degree at least $(m-1)\lfloor n/2 \rfloor$. Deleting a vertex leaves a feasible multigraph on $n - 1 \ge 7$ vertices, with at most $(m-1)M(n-1) = (m-1)\lfloor (n-1)^2/4 \rfloor$ arcs. The deleted vertex therefore has at least $(m-1)\big(\lfloor n^2/4 \rfloor - \lfloor (n-1)^2/4 \rfloor\big) = (m-1)\lfloor n/2 \rfloor$ arcs. For even n, every vertex of $(m-1) \cdot B_{n/2,\,n/2}$ has degree exactly $(m-1)n/2$.

Part III. Hypergraphs

Menger for hypergraphs, simple hypergraphs that attain the edge bound, lower bounds for the vertex problem, multigraphs with internally vertex-disjoint routes, and directed hypergraphs.

A.5 The hypergraph bounds

Hypergraphs have several notions of connectivity. This thesis uses Berge paths throughout. We first need Menger's theorem for Berge paths. It also shows that the gate network of Chapter 2 computes the right value.

Theorem A.20 (Menger for multihypergraphs)**.** *Let u, v be distinct vertices of a multihypergraph $\mathcal{H}$, and count parallel copies of a hyperedge separately. The largest number of u–v Berge paths that share no hyperedge copy equals the smallest number of hyperedge copies whose removal separates u from v.*

Proof. We turn $\mathcal{H}$ into a directed multigraph N, the gate network, and use Menger's theorem for directed multigraphs (Theorem 1.1). Let c be the total number of hyperedge copies. Every hyperedge e with $\mu(e)$ copies gets two new nodes e^- and e^+ and $\mu(e)$ parallel arcs $e^- \to e^+$, its *gate*. Every vertex $x \in e$ gets $c + 1$ parallel arcs $x \to e^-$ and $c + 1$ parallel arcs $e^+ \to x$.

Paths. A Berge path $u, e_1, v_1, \ldots, e_k, v$ gives the directed walk $u \to e_1^- \to e_1^+ \to v_1 \to \cdots \to v$ in N. If it uses different copies of the same hyperedge, this walk revisits the corresponding gate nodes. Conversely, a directed u–v path in N alternates between original vertices and gates. Since it repeats no gate node, choosing the hyperedge copy represented by each gate arc gives a Berge path.

Take a family of Berge paths that share no hyperedge copy. Each hyperedge e has $\mu(e)$ copies and $\mu(e)$ gate arcs, so distinct copies of e get distinct gate arcs. Each path uses at least one hyperedge copy, so the family has at most c paths. Every other arc has $c+1$ copies, so distinct paths can use distinct copies of every other arc. This gives an equally large family of arc-disjoint directed walks in N. Deleting closed portions from each walk yields directed paths with the same endpoints. This only removes arcs, so arc-disjointness is preserved. Conversely, arc-disjoint directed paths in N use distinct gate arcs and therefore distinct hyperedge copies, so the Berge paths they give share no hyperedge copy. The largest families have the same size in N and in $\mathcal{H}$.

Cuts. Every u–v path in N passes a gate, so removing all c gate arcs separates u from v, and a smallest separating set of arcs in N has at most c arcs. Such a set never contains all $c+1$ copies of an arc that is not a gate arc. Removing some but not all copies of an arc stops no path, so we can put these copies back, and the set still separates u from v. Hence some smallest separating set consists of gate arcs only. The gate arcs of e stand for the copies of e. Removing a set of gate arcs therefore separates u from v in N exactly when removing the same hyperedge copies separates u from v in $\mathcal{H}$. The smallest separating sets then have the same size in N and in $\mathcal{H}$.

Menger's theorem in N says that the two sizes are equal in N, and therefore also in $\mathcal{H}$. □

The same proof works for directed hyperedges and directed Berge paths, with arcs $x \to e^-$ only for the tails x of e, and arcs $e^+ \to x$ only for its heads.

Theorem 1.15 says that an r-uniform multihypergraph on n vertices with $\lambda^{\max} \le m-1$ has at most $\lfloor (m-1)(n-1)/(r-1) \rfloor$ hyperedge copies. The next theorem shows that for $r \ge 3$ and $m-1 \le \binom{n-2}{r-2}$, simple hypergraphs attain this bound. For graphs ($r=2$) this is different: there, simplicity lowers the bound from $(m-1)(n-1)$ to $\lfloor m(n-1)/2 \rfloor$.

The construction needs hypergraphs in which no vertex lies in too many hyperedges. The *degree* of a vertex in a hypergraph is the number of hyperedges that contain it.

Lemma A.21 (Sparse uniform hypergraphs exist)**.** *Let r, n and d be integers with $r \ge 1$, $n \ge r$ and $0 \le d \le \binom{n-1}{r-1}$. There is a simple r-uniform hypergraph on n vertices with exactly $\lfloor \frac{dn}{r} \rfloor$ hyperedges and maximum degree at most d.*

Proof. Let $N := \lfloor \frac{dn}{r} \rfloor$. Since $n\binom{n-1}{r-1} = r\binom{n}{r}$ (both count the pairs of an r-set and one of its vertices), we get $N \le \frac{n}{r}\binom{n-1}{r-1} = \binom{n}{r}$. There are therefore at least N different r-sets. There are finitely many families of N different r-sets. Choose one with the smallest value of $\sum_x \deg(x)^2$. We show that any two degrees in this family differ by at most one.

Suppose instead that $\deg(x) \ge \deg(y) + 2$ for two vertices x, y. Let a be the number of sets in the family that contain x but not y, let b be the number that contain y but not x, and let c be the number that contain both. Then $\deg(x) = a+c$ and $\deg(y) = b+c$, so $a-b = \deg(x)-\deg(y) \ge 2$. For a set A that contains x but not y, the set $A-x+y$ is an r-set that contains y but not x. Different sets A give different sets $A-x+y$, because A can be recovered from $A-x+y$ by removing

y and adding x. This gives a such new sets, but the family has only $b < a$ sets that contain y but not x. For some A, the set $A - x + y$ is therefore not in the family. Replace A by $A - x + y$. The family still has N different sets. The degree of x drops by one, the degree of y rises by one, and the other degrees stay the same. The sum changes by

$$(\deg(x) - 1)^2 + (\deg(y) + 1)^2 - \deg(x)^2 - \deg(y)^2 = 2\big(\deg(y) - \deg(x)\big) + 2 \leq -2.$$

This contradicts the choice of the family. Any two degrees therefore differ by at most one.

Each of the N sets has r vertices, so the degrees add up to rN. Their average is rN/n. Any two degrees differ by at most one, which puts every degree in $\{k, k+1\}$, where k is the smallest degree. If all degrees equal k, the average is k, which is then also the largest degree and equals $\lceil rN/n \rceil$. Otherwise some degrees are k and others $k+1$. The average then lies strictly between k and $k+1$, and the largest degree $k+1$ again equals $\lceil rN/n \rceil$. Since $N \leq dn/r$, we have $rN/n \leq d$. As d is an integer, $\lceil rN/n \rceil \leq d$. □

Theorem A.22 (Simple hypergraphs attain the edge bound for $r \geq 3$)**.** *Let $r \geq 3$, $n \geq r$, and $2 \leq m$ with $m - 1 \leq \binom{n-2}{r-2}$. Then the maximum number of hyperedges in a* simple *r-uniform hypergraph on n vertices with $\lambda^{\max} \leq m - 1$ is exactly $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$.*

Proof. The upper bound is Theorem 1.15. For the lower bound, take a hub h and let $V' = V \setminus \{h\}$. Apply Lemma A.21 with uniformity $r - 1$, with the $n - 1$ vertices of V', and with $d = m - 1$. The lemma needs $n - 1 \geq r - 1$ and $m - 1 \leq \binom{n-2}{r-2}$, and both hold. It gives a simple $(r-1)$-uniform hypergraph $\mathcal{G}^*$ on V' with maximum degree at most $m - 1$ and $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$ hyperedges. Add h to each of its hyperedges. Different $(r-1)$-sets give different r-sets, so the result $\mathcal{H}$ is simple. Every vertex $w \neq h$ lies in at most $m - 1$ hyperedges, and every route from or to w uses one of them. Removing these hyperedges therefore separates w from every other vertex. Every pair contains a vertex other than h, so $\lambda_{\mathcal{H}}^{\max} \leq m - 1$. □

Remark A.23 (The condition is needed). Without the condition $m - 1 \leq \binom{n-2}{r-2}$, simple hypergraphs may not reach the bound. The smallest example is $n = r = m = 3$. There is only one 3-set, so a simple hypergraph has at most one hyperedge, while the bound is $\lfloor 2 \cdot 2/2 \rfloor = 2$. Two copies of the one hyperedge do reach the bound. Theorem 1.18 needs the same condition with $m - 1 = 2$, which is the range $2 < r < n$.

Construction A.24 (Twelve hyperedge copies on six vertices)**.** Take a hub h and five other vertices a, b, c, d, e. Take all ten triples that consist of h and two of the other vertices. Add a second copy of $\{h, d, e\}$ and one copy of $\{a, b, c\}$.

Proposition A.25 (The edge value at $(n, m, r) = (6, 6, 3)$)**.** *The multihypergraph of Construction A.24 has 12 hyperedge copies and $\lambda^{\max} \leq 5$. The largest number of hyperedge copies of a 3-uniform multihypergraph on 6 vertices with $\lambda^{\max} \leq 5$ is 12, the value of Theorem 1.15.*

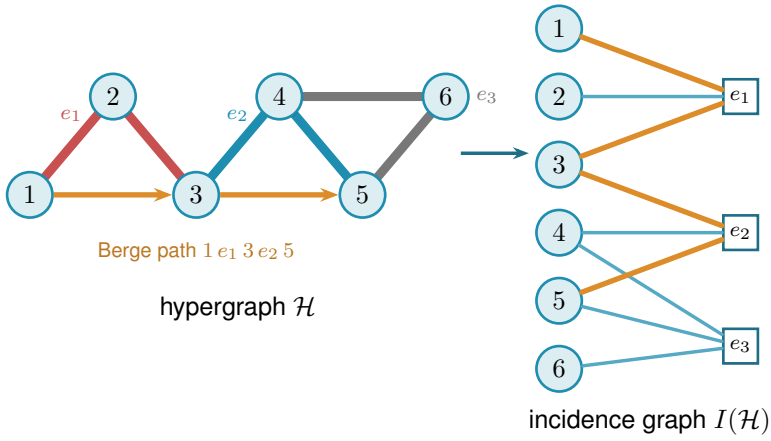
Proof. Each non-hub vertex lies in four of the ten triples, one for each other non-hub vertex. The second copy of $\{h, d, e\}$ adds one to the degrees of d and e. The triple $\{a, b, c\}$ adds one to

the degrees of a, b and c. Every non-hub vertex therefore lies in exactly five hyperedge copies. Every pair of vertices contains a non-hub vertex w. Every route from or to w uses one of the five copies at w, so removing them separates the pair. This gives $\lambda \leq 5$ for every pair. There are $10+1+1 = 12$ copies. Theorem 1.15 gives at most $\lfloor 5 \cdot 5/2 \rfloor = 12$, so 12 is the largest value. □

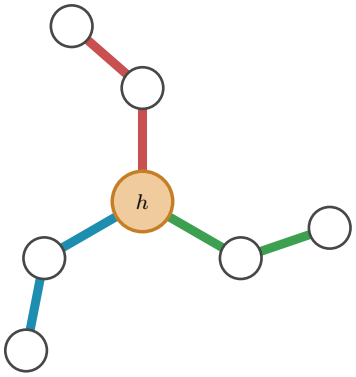
Since $\kappa \leq \lambda$, the same multihypergraph also gives 12 for the vertex problem. I do not know whether 12 is optimal there.

A.6 Lower bounds for the hypergraph vertex problem

The upper bounds for the hypergraph vertex problem are proved in Section 1.4.3 with the incidence graph, where $\kappa_{\mathcal{H}}(u, v) = \kappa_{I(\mathcal{H})}(u, v)$. Figure A.3 shows this translation. This section gives the lower bounds.



(a) A 3-uniform hypergraph $\mathcal{H}$ (left) and its bipartite incidence graph $I(\mathcal{H})$ (right): circles for vertices, hollow squares for hyperedges, an edge when the vertex lies in the hyperedge. The Berge path $1\, e_1\, 3\, e_2\, 5$ is traced in orange in both pictures. In $\mathcal{H}$ it rides the red line e_1 and the blue line e_2, changing at the shared vertex 3, while the grey line e_3 lies off it. In $I(\mathcal{H})$ it is the ordinary path $1-e_1-3-e_2-5$ alternating sides.



(b) A star hypertree: every hyperedge $\{h\} \cup G_i$ shares only the hub h (orange), and the other $r - 1 = 2$ vertices of each block sit in a single hyperedge. Its incidence graph is a tree, so $\kappa^{\max} \leq 1$.

Figure A.3. The translation the vertex problem rests on. Internally vertex-disjoint Berge paths in $\mathcal{H}$ correspond exactly to internally vertex-disjoint paths in $I(\mathcal{H})$, so $\kappa_{\mathcal{H}}(u, v) = \kappa_{I(\mathcal{H})}(u, v)$. **(top)** A hypergraph and its incidence graph with one Berge path drawn in both. **(bottom)** The star hypertree of Theorem 1.17, whose incidence graph is a tree.

Proposition A.26 (General lower bound)**.** *For $r \geq 3$, $m \geq 2$ and $m - 1 \leq \binom{n-2}{r-2}$,*

$$k_m^{(r)}(n) \;\geq\; \left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor,$$

attained by a simple hypergraph.

Proof. The simple hypergraph of Theorem A.22 has $\lambda^{\max} \leq m - 1$. Since $\kappa \leq \lambda$, it is also feasible for the vertex problem. □

That construction needs different $(r-1)$-sets, which explains the condition $m - 1 \leq \binom{n-2}{r-2}$. A multihypergraph can repeat a hyperedge instead. This covers every $n \geq r$.

Proposition A.27 (The multihypergraph lower bound)**.** *For $r \geq 2$, $m \geq 2$ and $n \geq r$,*

$$K_m^{(r)}(n) \;\geq\; \left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor,$$

attained by the multihypergraph in the proof of Theorem 1.15. At $m = 3$ this meets the upper bound of Theorem 1.18, so

$$K_3^{(r)}(n) \;=\; \left\lfloor \frac{2(n-1)}{r-1} \right\rfloor \qquad \text{for every } n \geq r.$$

Proof. The multihypergraph in the proof of Theorem 1.15 has $\lfloor (m-1)(n-1)/(r-1) \rfloor$ hyperedge copies and $\lambda^{\max} \leq m - 1$. Since $\kappa \leq \lambda$, it also has $\kappa^{\max} \leq m - 1$. At $m = 3$ its size equals the upper bound of Theorem 1.18. □

A.7 Multigraphs with internally vertex-disjoint routes

At $r = 2$ the hypergraph vertex problems are about multigraphs. They are not the same as the simple problems. Recall that q parallel copies of uv count as q internally vertex-disjoint routes. Write $K_m(n)$ for the largest number of edges, counted with multiplicity, of a multigraph on n vertices with $\kappa^{\max} \leq m - 1$.

The parallel edges and the longer routes do not interfere, so we can count them separately.

Lemma A.28 (Splitting the local vertex-connectivity)**.** *Let G be a multigraph with multiplicity function μ, and let its* underlying simple graph *G_0 be the simple graph with an edge uv whenever $\mu(u, v) \geq 1$. For any two vertices $u \neq v$, write $\pi(u, v)$ for the largest number of pairwise internally vertex-disjoint u–v paths of length at least two in G_0. Then*

$$\kappa_G(u, v) \;=\; \mu(u, v) \;+\; \pi(u, v).$$

Proof. A route of length one uses one copy of the edge uv and has no inner vertex. Two routes of length one are therefore internally vertex-disjoint when they use different copies, and a route of length one is internally vertex-disjoint from every longer route.

Lower bound. Take all $\mu(u,v)$ copies of uv, and $\pi(u,v)$ internally vertex-disjoint u–v paths of length at least two in G_0. Use one copy of each edge on these paths. The inner vertices do not change, so all these routes are internally vertex-disjoint.

Upper bound. Take a family of internally vertex-disjoint u–v routes in G. At most $\mu(u,v)$ of them have length one, since they use different copies of uv. Map each longer route to its sequence of vertices, a path in G_0. Two longer routes have disjoint, non-empty sets of inner vertices. Their images are therefore different and internally vertex-disjoint, and there are at most $\pi(u,v)$ of them. □

A multigraph is therefore feasible exactly when $\mu(u,v) \le m-1-\pi(u,v)$ on every edge and $\pi(u,v) \le m-1$ on every pair.

Theorem A.29 (The multigraph vertex value at $m=2$)**.** $K_2(n) = n-1$*, attained exactly by the trees on the* n *vertices.*

Proof. By Lemma A.28, $\mu(u,v)+\pi(u,v) \le 1$ on every edge. If G_0 had a cycle, take an edge uv on it. Then $\mu(u,v) \ge 1$, and the rest of the cycle is a u–v path of length at least two, so $\pi(u,v) \ge 1$. Hence G_0 is a forest and every multiplicity is one. A forest has at most $n-1$ edges, and a spanning tree has exactly $n-1$. □

Remark A.30. At $m=2$ a second copy of an edge is already a second route, so every feasible multigraph is simple, and $K_2(n) = k_2(n) = n-1$. This is also Theorem 1.17 at $r=2$.

The value at $m=3$ follows from the hypergraph theorem.

Corollary A.31 (The multigraph vertex value at $m=3$)**.** $K_3(n) = 2(n-1)$ *for every* $n \ge 2$*.*

Proof. Apply Theorem 1.18 and Proposition A.27 at $r=2$. There the construction is the doubled star, which attains the upper bound $2(n-1)$. □

At $m=3$ the vertex value therefore equals the edge value $L_3(n)$ of Theorem 1.8. At $m=5$ they differ: the thickened theta below gives $K_5(4) \ge 14$, while $L_5(4) = 12$. For $m=4$ this is open. For $m \ge 3$ there are three constructions.

- ★ *The thickened tree.* Take a spanning tree and give every edge multiplicity $m-1$. Tree edges have $\pi = 0$, and other pairs have $\pi = 1$, so it is feasible. It has $(m-1)(n-1)$ edges, the edge value of Theorem 1.8.
- ★ *The thickened complete graph.* Give every edge of K_n multiplicity q. Here $\pi(u,v) = n-2$: each of the other $n-2$ vertices gives a route of length two, and together they separate u from v once the edge uv is removed. So $\kappa(u,v) = q+n-2$, and it is feasible when $q \le m+1-n$. It has $\binom{n}{2}(m+1-n)$ edges, which is positive only when $n \le m$.
- ★ *The thickened theta.* Choose two vertices, the *poles*. Join each of the other $n-2$ vertices to both poles with multiplicity $m-2$, and join the poles with multiplicity $m+1-n$ if this is

positive. A non-pole is joined only to the two poles, so every route from it leaves through a pole. So a pole and a non-pole have $\pi = 1$, and $\kappa = (m-2)+1 = m-1$. Two non-poles are not adjacent and have $\kappa = \pi = 2$. The two poles have $\pi = n-2$, one route through each non-pole, so $\kappa = \max(0,\, m+1-n)+n-2$, which is $m-1$ when $n \le m+1$. So it is feasible when $n \le m+1$, and it has $2(n-2)(m-2)+\max(0,\, m+1-n)$ edges.

For $n \le m+1$ the theta has $2(n-2)(m-2)+(m+1-n)$ edges and the thickened tree has $(m-1)(n-1)$. Their difference is $(n-2)(m-4)$. The two agree at $m=4$, and for $m \ge 5$ and $n \ge 3$ the theta has more. For $n \ge m+2$ the other two constructions are not feasible, so among these three the tree has the most edges.

For $m \ge 5$, one triangle added to the thickened tree already beats it, also when $n \ge m+2$. The next result makes this precise.

Fix a connected simple graph G_0 with $\kappa^{\max}_{G_0} \le m-1$. By Lemma A.28, the best multigraph on G_0 takes $\mu(u,v) = m-1-\pi(u,v)$ on every edge. Its size is $(m-1)\,|E(G_0)| - \sum_{uv \in E(G_0)} \pi(u,v)$. Since $|E(G_0)| = n-1+\operatorname{rank}(G_0)$, it beats the thickened tree exactly when

$$\sum_{uv \in E(G_0)} \pi(u,v) \;<\; (m-1)\operatorname{rank}(G_0).$$

Theorem A.32 (Thickened complete graphs sharing a vertex beat the thickened tree)**.** *Let $3 \le s \le m$ and $q = m+1-s \ge 1$. Let $1 \le k \le \lfloor (n-1)/(s-1) \rfloor$. Take k copies of K_s that share one vertex h and are otherwise disjoint, and give every edge multiplicity q. The remaining $n-1-k(s-1)$ vertices form a path that starts at h, with every edge of multiplicity $m-1$. This multigraph has $\kappa^{\max} \le m-1$, and its number of edges is*

$$(m-1)(n-1) \;+\; k \cdot \operatorname{gain}(s,m), \qquad \operatorname{gain}(s,m) := \frac{(s-1)(s-2)(m-1-s)}{2}.$$

It therefore beats the thickened tree exactly when $s \le m-2$. Such an s exists when $m \ge 5$.

Proof. Write G_0 for the underlying simple graph.

Feasibility. The blocks of G_0 are the copies of K_s and the path edges. As in the proof of Theorem A.5, a path between two vertices of the same block cannot leave the block, since it would pass a cut vertex twice. Take an edge uv in a copy of K_s. The other $s-2$ vertices of the copy give $s-2$ internally vertex-disjoint routes of length two. Every longer route stays in the copy, so it passes one of these $s-2$ vertices, which gives $\pi(u,v) = s-2$. By Lemma A.28, $\kappa_G(u,v) = q+(s-2) = m-1$. Two vertices in different blocks are separated by removing a cut vertex, so $\kappa_G(u,v) \le 1$. For an edge uv on the path, $\pi(u,v) = 0$, so $\kappa_G(u,v) = m-1$. Inside a block every two vertices are adjacent, so no other pairs need checking.

Count. Since $q \ge 1$, the underlying simple graph G_0 is connected, so $|E(G_0)| = n-1+\operatorname{rank}(G_0)$. Every edge uv has multiplicity $m-1-\pi(u,v)$, so the number of edges is $(m-1)|E(G_0)| - \sum_{uv} \pi(u,v)$. Only the k copies of K_s contain cycles or have $\pi > 0$. Each copy has

rank $\binom{s}{2} - s + 1 = (s-1)(s-2)/2$, and its $\binom{s}{2}$ edges each have $\pi = s-2$. The number of edges is therefore

$$(m-1)\Big[n - 1 + k\tfrac{(s-1)(s-2)}{2}\Big] - k\tfrac{s(s-1)(s-2)}{2} \;=\; (m-1)(n-1) + k\,\mathrm{gain}(s,m).$$

□

At $s = 3$, $\mathrm{gain}(3,m) = m-4$. This is zero at $m = 4$ and positive for $m \geq 5$. For every $m \geq 5$ the thickened tree is therefore not optimal, also when $n \geq m+2$. Here is a small example. Take $m = 5$ and $n = 7$. Put a triangle on $\{u_1, u_2, u_3\}$ with multiplicity 3, and attach a path of four more vertices to u_1 with multiplicity 4. Two triangle vertices have three parallel edges and one route through the third vertex, so $\kappa = 4$. Every other pair is separated by a cut vertex or joined only by its parallel edges. This gives $\kappa^{\max} = 4 = m-1$. There are $3 \cdot 3 + 4 \cdot 4 = 25$ edges, one more than the thickened tree's 24. With k triangles the gain is k, so the gap grows linearly in n.

Sharing a vertex lets more copies fit, since each new copy uses only $s-1$ new vertices. For example, at $m = 10$, $s = 6$ and $n = 11$, two copies sharing a vertex give 150 edges. Disjoint copies would allow only one copy, with 75 edges, and 120 edges after attaching the other five vertices.

Each copy of K_s uses $s-1$ new vertices, which makes the gain per added vertex $\mathrm{gain}(s,m)/(s-1) = (s-2)(m-1-s)/2$. The factors $s-2$ and $m-1-s$ add up to $m-3$. Their product is largest when they are about equal, at $s \approx (m+1)/2$, where the gain per vertex is about $(m-3)^2/8$. For sufficiently large m, choose an integer s nearest to $(m+1)/2$, and take $k = \lfloor (n-1)/(s-1) \rfloor$, which is at least 1 once $n \geq m$. The gain per added vertex is then $m^2/8 + O(m)$, and rounding k down loses at most one $\mathrm{gain}(s,m) = O(m^3)$ edges. This gives $K_m(n) \geq \big(\frac{1}{8} - O(1/m) - O(m/n)\big)m^2 n$, while the thickened tree gives only $(m-1)(n-1)$.

This construction need not be optimal. The next result gives an upper bound of the same order.

Proposition A.33 (An upper bound of the same order)**.** *For all $m \geq 2$ and $n \geq 2$, the first inequality below holds, and the second holds for $n \geq m-1$:*

$$K_m(n) \;\leq\; (m-1)\,k_m(n) \;\leq\; (m-1)\Big[(m-1)n - \binom{m}{2}\Big].$$

Proof. Let G be feasible, with underlying simple graph G_0.

Every multiplicity is at most $m-1$. By Lemma A.28, $\kappa_G(u,v) = \mu(u,v) + \pi(u,v) \geq \mu(u,v)$, so feasibility gives $\mu(u,v) \leq m-1$ on every pair and the total is at most $(m-1)|E(G_0)|$.

G_0 is feasible for the simple vertex problem. Take $u \neq v$. If they are adjacent, then $\mu(u,v) \geq 1$, and the routes of G_0 between them are the edge uv and the longer routes, so $\kappa_{G_0}(u,v) = 1 + \pi(u,v) \leq \mu(u,v) + \pi(u,v) = \kappa_G(u,v) \leq m-1$. If they are not adjacent, then $\kappa_{G_0}(u,v) = \pi(u,v) = \kappa_G(u,v) \leq m-1$. Hence $|E(G_0)| \leq k_m(n)$, which gives the first inequality.

The second inequality is Remark A.6. □

For $m \to \infty$ and $n/m \to \infty$ this gives

$$\Big(\tfrac{1}{8} - o(1)\Big) m^2\, n \;\le\; K_m(n) \;\le\; m^2 n.$$

This needs n large: for example $K_m(2) = m - 1$. Theorem A.36 below gives a better lower constant. The first inequality, $K_m(n) \le (m-1)k_m(n)$, says that $K_m(n)$ is at most $m-1$ times the simple vertex value. It is tight at $m = 2$ (Remark A.30).

A.7.1 The value is a block problem

The next two results reduce $K_m(n)$ to 2-connected blocks, as in Section A.1.1.

Lemma A.34 (The objective in closed form)**.** *For a simple graph G_0 on n vertices with $\kappa^{\max}_{G_0} \le m-1$, the largest total multiplicity of a feasible multigraph with underlying simple graph exactly G_0 is*

$$W_m(G_0) \;:=\; \sum_{uv \in E(G_0)} \big(m - \kappa_{G_0}(u,v)\big),$$

and every simple graph arising as the underlying graph of a feasible multigraph satisfies $\kappa^{\max}_{G_0} \le m-1$. Hence

$$K_m(n) \;=\; \max\big\{\, W_m(G_0) \;:\; G_0 \text{ simple on } n \text{ vertices},\ \kappa^{\max}_{G_0} \le m-1 \,\big\}.$$

Proof. By Lemma A.28, a multigraph on G_0 is feasible exactly when $\mu(u,v) \le m - 1 - \pi(u,v)$ on every edge, so the best choice is $\mu(u,v) = m - 1 - \pi(u,v)$. On an edge, $\kappa_{G_0}(u,v) = 1 + \pi(u,v)$, so this is $m - \kappa_{G_0}(u,v)$, and the total is $W_m(G_0)$. It is at least one on every edge, since $\kappa_{G_0}(u,v) \le m-1$, so the underlying graph is really G_0. The proof of Proposition A.33 shows that the underlying graph of a feasible multigraph is feasible. Maximising W_m over feasible G_0 therefore gives $K_m(n)$. □

The objective is a sum over edges of a quantity that depends only on the block of the edge. This gives the next result.

Theorem A.35 (The multigraph vertex problem reduces to blocks)**.** *For $b \ge 2$ write*

$$g_m(b) \;:=\; \max\big\{\, W_m(B) \;:\; B \text{ 2-connected or an edge}, |V(B)| = b,\ \kappa^{\max}_B \le m-1 \,\big\},$$

with $g_m(b) := -\infty$ when no such B exists. Then for all $m \ge 2$ and $n \ge 2$,

$$K_m(n) \;=\; \max\Big\{ \sum_i g_m(b_i) \;:\; b_i \ge 2,\ \sum_i (b_i - 1) \le n-1 \Big\},$$

the maximum over all finite multisets of block sizes. In particular the value is determined by the finitely many numbers $g_m(2), \dots, g_m(n)$.

Proof. This is the proof of Theorem A.5 with W_m in place of the number of edges. As there, $\kappa_{G_0}(u,v) = \kappa_B(u,v)$ for u, v in a common block B, so $m - \kappa_{G_0}(u,v)$ depends only on the block of

the edge uv, and $W_m(G_0) = \sum_B W_m(B)$. The block sizes satisfy $\sum_B (|V(B)| - 1) \le n - 1$, and gluing blocks at one common vertex realises every choice of sizes, as there. With Lemma A.34 this gives the claim. □

In this way $K_m(n)$ can be computed from 2-connected graphs: compute W_m for each, and choose the best block sizes.

Theorem A.36 (Thickened complete bipartite blocks)**.** *Let $1 \le s \le t \le m-1$. Give every edge of $K_{s,t}$ multiplicity $m - s$. The result has $s + t$ vertices, $st(m - s)$ edges and $\kappa^{\max} \le m - 1$. Gluing k such blocks at one shared vertex gives, for $n = k(s + t - 1) + 1$,*

$$K_m(n) \;\ge\; \frac{st\,(m-s)}{s+t-1}\,(n-1).$$

The best choice of s and t gives the rate $\big(3-2\sqrt{2}+o_m(1)\big)m^2$, at $t = m-1$ and $s \approx (\sqrt{2}-1)m$. Theorem A.32 gives $\big(\frac{1}{8}+o_m(1)\big)m^2$. The new rate is larger by a factor $8(3-2\sqrt{2}) = 24-16\sqrt{2} \approx 1.373$.

Proof. Write S and T for the two sides, with $|S| = s$ and $|T| = t$. Every edge joins S and T.

Two vertices on the same side. Take $u \ne u'$ in S. For each $v \in T$ the route u, v, u' is a route of length two, and these t routes are internally vertex-disjoint. Every route from u first goes to a vertex of T, so removing T separates u from u'. This gives $\kappa(u, u') = t$. In the same way, $\kappa(v, v') = s$ for $v \ne v'$ in T.

Two vertices on different sides. Take $u \in S$ and $v \in T$, and remove the edge uv. Every other route from u first goes to a vertex of $T \setminus \{v\}$, and every route into v comes from a vertex of $S \setminus \{u\}$. Both sets therefore separate u from v, and $\pi(u, v) \le \min(s-1, t-1) = s-1$. Choose $s-1$ different vertices $u_i \in S \setminus \{u\}$, and $s-1$ different vertices $v_i \in T \setminus \{v\}$. This is possible because $s \le t$. The routes u, v_i, u_i, v are internally vertex-disjoint, which gives $\pi(u, v) = s-1$.

Feasibility and count. By Lemma A.28, an edge uv has $\kappa = (m-s) + (s-1) = m-1$. Two vertices on the same side are not adjacent, so they have $\kappa = t$ or $\kappa = s$, and both are at most $m-1$. The block has st edges, each of multiplicity $m-s$, so $st(m-s)$ edges in total.

Gluing. If $s \ge 2$, then $K_{s,t}$ is 2-connected. By the gluing in the proof of Theorem A.35, k copies at one shared vertex are feasible, with $k\,st(m-s)$ edges on $n = k(s+t-1)+1$ vertices. If $s = 1$, then $K_{1,t}$ is a star, which is a tree, and every edge has multiplicity $m-1$. Gluing k copies at one vertex gives a tree with every edge of multiplicity $m-1$. This is the thickened tree above, which is feasible. It has $k\,t(m-1) = k\,st(m-s)$ edges. Since $k = (n-1)/(s+t-1)$, the number of edges is $\frac{st(m-s)}{s+t-1}(n-1)$ in both cases.

The best choice. For fixed s, the factor $t/(s+t-1)$ does not decrease as t grows, since its derivative in t is $(s-1)/(s+t-1)^2 \ge 0$. Some best choice therefore has $t = m-1$, the largest allowed value. Put $s = \alpha m$. The rate becomes

$$\frac{s\,(m-1)(m-s)}{s+m-2} \;=\; \frac{\alpha(1-\alpha)}{1+\alpha}\,m^2 \;+\; O(m).$$

The function $h(\alpha) = \alpha(1-\alpha)/(1+\alpha)$ has derivative $h'(\alpha) = (1-2\alpha-\alpha^2)/(1+\alpha)^2$. This is zero at $\alpha = \sqrt{2}-1$, positive before it and negative after it, so h is largest there. Its value there is

$$h(\sqrt{2}-1) \;=\; \frac{(\sqrt{2}-1)(2-\sqrt{2})}{\sqrt{2}} \;=\; (\sqrt{2}-1)^2 \;=\; 3-2\sqrt{2}.$$

Since $3-2\sqrt{2} \approx 0.172 > \frac{1}{8}$, this is larger than the rate of Theorem A.32, by the stated factor. □

In the best choice, one side has about $0.41m$ vertices and the other has $m-1$.

For other n with $n \geq s+t$, glue $k = \lfloor (n-1)/(s+t-1) \rfloor$ blocks at one shared vertex, and join each remaining vertex to the shared vertex by one edge. Each such vertex has one neighbour, so the multigraph stays feasible. Fewer than $s+t-1 < 2m$ vertices remain, so at most one block of $st(m-s) < m^3$ edges is lost. For sufficiently large m, take $t = m-1$ and an integer s nearest to $(\sqrt{2}-1)m$. For these choices the rate is $(3-2\sqrt{2})m^2 + O(m)$. This gives $K_m(n)/(m^2 n) \geq 3-2\sqrt{2} - O(1/m) - O(m/n)$, where $O(1/m)$ accounts for the finite-m error in the block rate and $O(m/n)$ for the rounding of k. Hence $3-2\sqrt{2}-o(1) \leq K_m(n)/(m^2 n) \leq 1$ for $m \to \infty$ and $n/m \to \infty$. Whether this ratio has a limit is open.

A.7.2 The directed problem

The splitting of κ into parallel arcs and longer routes also works for directed graphs (Lemma A.37). Corollary A.38 below gives the exact value at $m = 2$ and the leading term for every fixed m.

Lemma A.37 (Splitting the directed local vertex-connectivity)**.** *Let D be a directed multigraph with multiplicity function μ and underlying simple directed graph D_0. For an ordered pair $u \neq v$, write $\pi(u,v)$ for the largest number of pairwise internally vertex-disjoint directed u–v paths of length at least two in D_0. Then $\kappa_D(u,v) = \mu(u,v) + \pi(u,v)$.*

Proof. The proof of Lemma A.28 works word for word with directed paths. A route of length one is a copy of the arc $u \to v$ and has no inner vertex. Longer routes give internally vertex-disjoint directed paths in D_0, and conversely, using one copy of each arc. Copies of the reverse arc $v \to u$ are never used.

□

Corollary A.38 (The directed multigraph vertex value)**.** *For every $n \geq 2$,*

$$K_2^{\mathrm{dir}}(n) \;=\; k_2^{\mathrm{dir}}(n) \;=\; \ell_2^{\mathrm{dir}}(n) \;=\; \max\Big(2(n-1),\; \Big\lfloor \frac{n^2}{4} \Big\rfloor\Big),$$

and for each fixed $m \geq 2$,

$$(m-1)M(n) \;\leq\; K_m^{\mathrm{dir}}(n) \;\leq\; (m-1)\Big\lfloor \frac{n^2}{4} \Big\rfloor + (m-1)^2(n-1).$$

In particular $K_m^{\mathrm{dir}}(n) = (m-1)n^2/4 + O_m(n)$, and the thickened bipartite directed graph of Theorem 1.22 has the right leading term here too.

Proof. The case $m = 2$. This is shown in Chapter 1, after Theorem 1.26.

General m. The extremal multigraphs of Theorem 1.22 have $\lambda^{\max} \leq m - 1$. Since $\kappa \leq \lambda$, also $\kappa^{\max} \leq m - 1$. This gives the lower bound. A directed multigraph is a forward directed 2-uniform multihypergraph: each arc is a hyperedge with one tail and one head. Theorem 1.26 at $r = 2$ therefore gives the upper bound. For $n \geq 7$ we have $M(n) = \lfloor n^2/4 \rfloor$. Both bounds are then $(m-1)n^2/4 + O_m(n)$. □

Construction A.39 (Thickened complete directed graph, and its bouquet)**.** For $2 \leq b \leq m$, take the complete directed graph on b vertices and give every arc multiplicity $m+1-b$. At $b = 2$ this is one bidirected edge at multiplicity $m - 1$. Hanging k such blocks off one shared vertex, pairwise disjoint otherwise, uses $k(b-1)+1$ vertices, and blocks of different sizes may be mixed.

Proposition A.40 (Gluing complete directed graphs)**.** *Glue blocks of Construction A.39 with sizes b_i at one common vertex, keep them disjoint otherwise, and add isolated vertices to reach n. The result is feasible, so*

$$K_m^{\mathrm{dir}}(n) \;\geq\; \max\Big\{ \sum_i b_i(b_i-1)(m+1-b_i) \;:\; 2 \leq b_i \leq m,\; \sum_i (b_i - 1) \leq n-1 \Big\}.$$

Proof. Take $u \neq v$ in one block of size b. A route that leaves the block must come back through the common vertex, which a path visits only once, so every u–v route stays in the block. A longer route inside the block has an inner vertex among the other $b-2$ vertices of the block. Internally vertex-disjoint routes have different inner vertices, so there are at most $b-2$ longer routes. The $b-2$ routes $u \to w \to v$, one for each other vertex w of the block, reach this number. This gives $\pi(u,v) = b-2$, and Lemma A.37 gives $\kappa(u,v) = (m+1-b)+(b-2) = m-1$. Two vertices in different blocks are separated by the common vertex, so $\kappa \leq 1$. A block of size b has $b(b-1)$ ordered pairs, each with $m+1-b$ arcs. □

At $n = 3$ and $m = 6$, the thickened complete directed graph has 24 arcs, more than $(m-1)M(3) = 20$. It is feasible: every route of length at least two passes through the third vertex, so $\pi = 1$ and $\kappa = 4 + 1 = 5 = m - 1$ for every ordered pair. The extremal multigraphs of the arc problem are therefore not optimal for the vertex problem.

For small n the block construction is larger, and for large n the bipartite construction is larger. A block of size b costs $b-1$ vertices and has $b(b-1)(m+1-b)$ arcs, which is $b(m+1-b)$ arcs per vertex it costs. At $m = 3$ this is 4 for $b = 2$ and 3 for $b = 3$. The best block is then a bidirected edge, the bouquet is the thickened bidirected tree, and the bipartite construction is larger from $n = 8$. At $m = 6$ the values are 10, 12, 12, 10 and 6 for $b = 2, \ldots, 6$. The best blocks have three or four vertices. The bouquet has 84 arcs at $n = 8$, against 80, and the bipartite construction is larger from $n = 9$.

A.8 The directed hypergraph

The last four variants use directed hyperedges, in the forward, backward or general model of Section 1.3. The first result gives an upper and a lower bound that differ by a factor of four. Theorem 1.26 then closes this gap.

Proposition A.41 (Directed hypergraphs: first bounds)**.** *Let $n \geq r \geq 2$ and $m \geq 2$, and take the forward model. If every ordered pair of vertices has at most $m-1$ hyperedge-disjoint directed Berge routes, then*

$$|E(\mathcal{H})| \;\leq\; \frac{(m-1)\,n(n-1)}{r-1},$$

and, for every $1 \leq \alpha \leq n-r+1$, there is a feasible directed multihypergraph with

$$\alpha \left\lfloor \frac{(m-1)(n-\alpha)}{r-1} \right\rfloor$$

hyperedges. This construction is simple when $m-1 \leq \binom{n-\alpha-1}{r-2}$. For fixed r and m and large n we can take $\alpha = \lfloor n/2 \rfloor$, so the maximum is $\Theta_{r,m}(n^2)$.

Proof. Upper bound. A hyperedge with tail u and heads $h_1, \ldots, h_{r-1}$ joins the $r-1$ ordered pairs (u, h_i). If m hyperedges joined the same pair (u, v), they would give m hyperedge-disjoint routes of length one. Each of the $n(n-1)$ ordered pairs is therefore joined by at most $m-1$ hyperedges. Counting the pairs (hyperedge, joined pair) in two ways gives

$$(r-1)|E| \leq (m-1)\,n(n-1).$$

Lower bound. Split V into a set A of α tails and a set B of $\beta := n-\alpha \geq r-1$ heads. Let $k := r-1$, $d := m-1$ and $N := \left\lfloor \frac{d\beta}{k} \right\rfloor$. Number the vertices of B as $0, 1, \ldots, \beta-1$, and read the repeating sequence

$$0, 1, \ldots, \beta-1, 0, 1, \ldots$$

in consecutive blocks of k numbers. The first N blocks are the hyperedges of a k-uniform multihypergraph $\mathcal{G}^*$ on B. Since $k \leq \beta$, each block has k different vertices. The blocks use the first Nk positions of the sequence, so every vertex appears $\lfloor Nk/\beta \rfloor$ or $\lceil Nk/\beta \rceil$ times. Since $Nk \leq d\beta$, we have $Nk/\beta \leq d$, and since d is an integer, $\lceil Nk/\beta \rceil \leq d$. Every vertex therefore has degree at most d.

Now, for every $a \in A$ and every $H \in \mathcal{G}^*$, add the hyperedge with tail a and heads H. No vertex of B is a tail, so a route can never leave B, and every route has length one. Hence, for $a \in A$ and $w \in B$, the routes from a to w are the hyperedges with tail a whose heads contain w. There are $\deg_{\mathcal{G}^*}(w) \leq m-1$ of them. There are no routes from B, and no routes between two vertices of A, since all heads lie in B. The number of hyperedges is $\alpha N = \alpha \left\lfloor \frac{(m-1)(n-\alpha)}{r-1} \right\rfloor$. If $d \leq \binom{\beta-1}{k-1}$, we can use Lemma A.21 instead, which gives different hyperedges, so the directed hypergraph is simple. □

Construction A.42 (Bounded outgoing hyperedge count)**.** Let $n \geq r \geq 2$ and $m \geq 2$, and let $d = \min\{m-1, \binom{n-1}{r-1}\}$. Give every vertex d different outgoing forward hyperedges, with any d different sets of $r-1$ heads. This gives a simple directed hypergraph with nd hyperedges, feasible for hyperedge-disjoint and for separated routes. If repeated hyperedges are allowed, give every vertex $m-1$ copies of one outgoing hyperedge, for $n(m-1)$ hyperedges in total.

Proof. Every directed route from u starts with a hyperedge whose tail is u. Removing the outgoing hyperedges of u therefore separates u from every other vertex. There are at most $m-1$ of them, so $\lambda(u,v) \leq m-1$ for every ordered pair, and so also $\kappa(u,v) \leq m-1$. Hyperedges with different tails are different, so the hypergraph is simple when the head sets at each vertex are different. □

For small n this construction can beat the construction of Proposition A.41. For example, at $n = m = 6$ and $r = 3$ it gives 30 hyperedges, even in the simple model.

We write a directed hyperedge as (T, H), with tail set T and head set H. For $r \geq 4$ the general model has splits with both parts of size at least two. We show that the backward model is the forward model reversed, and that the general model has the same leading term as the forward model.

Lemma A.43 (Backward is the reverse of forward)**.** *Reversing every hyperedge, $(T,H) \mapsto (H,T)$, turns forward hypergraphs into backward ones and keeps the number of hyperedges. The reversed hypergraph $\mathcal{H}^R$ satisfies $\lambda_{\mathcal{H}}(u,v) = \lambda_{\mathcal{H}^R}(v,u)$ and $\kappa_{\mathcal{H}}(u,v) = \kappa_{\mathcal{H}^R}(v,u)$. The backward model therefore has the same extremal values as the forward model.*

Proof. Reversal turns every directed u–v Berge path into a v–u path with the same hyperedges and the same inner vertices, so it keeps both kinds of disjointness. □

Proposition A.44 (The general model obeys the forward bound)**.** *Take the general model. If every ordered pair has at most $m-1$ hyperedge-disjoint, or at most $m-1$ separated, directed Berge routes, then*

$$|E(\mathcal{H})| \;\leq\; \frac{(m-1)\,n(n-1)}{r-1}.$$

The construction of Proposition A.41 also works here, so the maximum is $\Theta_{r,m}(n^2)$.

Proof. Fix an ordered pair (u,v). Each hyperedge (T,H) with $u \in T$ and $v \in H$ gives a route $u \to v$ of length one. Different such hyperedges give separated routes, so there are at most $m-1$ of them. A hyperedge (T,H) joins exactly $|T|\,|H|$ ordered pairs. Summing over all pairs gives $\sum_{(T,H)} |T|\,|H| \leq (m-1)\,n(n-1)$. Since $|T| + |H| = r$ with both at least one, $|T|\,|H| - (r-1) = (|T|-1)(|H|-1) \geq 0$. This gives $(r-1)|E| \leq (m-1)n(n-1)$. □

These bounds fix the order n^2, but not the leading constant. The rest of this section finds the constant for both models. Their exact values may still differ.

The proof uses the one-step shadow directed graph. It bounds the hyperedges by the shadow arcs, shows that the shadow satisfies a two-step budget, and applies Lemma A.15.

Definition A.45 (One-step shadow)**.** Let $\mathcal{H}$ be a forward directed r-uniform hypergraph on V. Its *shadow* is the simple directed graph $R(\mathcal{H})$ on V with an arc (u, v) when some hyperedge of $\mathcal{H}$ has tail u and head v. Let $\mu_R(u, v)$ be the number of such hyperedges.

We now prove the theorem of Chapter 1.

Theorem 1.26 (restated). *Let $r \geq 2$ and $m \geq 2$. Every forward directed r-uniform multihypergraph $\mathcal{H}$ on $n \geq 2$ vertices with $\kappa_{\mathcal{H}}^{\max} \leq m - 1$ satisfies*

$$|E(\mathcal{H})| \;\leq\; \frac{m-1}{r-1}\left\lfloor \frac{n^2}{4} \right\rfloor \;+\; (m-1)^2(n-1).$$

Proof of Theorem 1.26. Let $R = R(\mathcal{H})$. It has no loops, since the tail of a hyperedge is not one of its heads.

The $\mu_R(u, v)$ hyperedges with tail u and head v give that many routes of length one, which are separated. This gives $\mu_R(u, v) \leq m-1$. Every hyperedge, with tail u and heads $h_1, \ldots, h_{r-1}$, gives the $r - 1$ arcs $u \to h_i$ of R. Counting the pairs of a hyperedge and one of its arcs in two ways gives

$$(r-1)\,|E(\mathcal{H})| \;=\; \sum_{(u,v)\in A(R)} \mu_R(u, v) \;\leq\; (m-1)\,|A(R)|.$$

Proposition A.41 used the bound $|A(R)| \leq n(n-1)$ here, which loses the factor four. We now bound $|A(R)|$ better.

Next we show that the shadow is budgeted. Fix an ordered pair (u, v). Recall that a midpoint of (u, v) is a vertex $x \notin \{u, v\}$ with arcs $u \to x$ and $x \to v$ in R. Let $\mathcal{T}$ be the set of midpoints, together with v if $(u, v) \in A(R)$, so that $|\mathcal{T}| = \mathbf{1}_{(u,v)\in A(R)} + p_2(u, v)$. Every $t \in \mathcal{T}$ has an arc $u \to t$ in R, so it is a head of some hyperedge with tail u. Choose hyperedges $e_1, e_2, \ldots$ with tail u one at a time, where each e_i contains an element t_i of $\mathcal{T}$ that lies in none of $e_1, \ldots, e_{i-1}$. Stop at e_k, when every element of $\mathcal{T}$ lies in one of them. Then $e_1, \ldots, e_k$ are different hyperedges, and $t_1, \ldots, t_k$ are different elements of $\mathcal{T}$. Each e_i has $r - 1$ heads, so it contains at most $r - 1$ elements of $\mathcal{T}$, which gives $k(r-1) \geq |\mathcal{T}|$.

Now turn each e_i into a route. If $t_i = v$, take the route $u \to v$ through e_i. If t_i is a midpoint x, take the route $u \to x \to v$: enter through e_i, and leave through a hyperedge with tail x and head v (one exists, since $(x, v) \in A(R)$). In the forward model every hyperedge has exactly one tail. The leaving hyperedges have different tails, namely different midpoints, so they are different. They also differ from $e_1, \ldots, e_k$, which have tail u. The k routes therefore share no hyperedge. A route has at most one inner vertex, its midpoint, and different routes have different midpoints. The routes are separated as well, and $k \leq \kappa_{\mathcal{H}}(u, v)$. This gives

$$\frac{\mathbf{1}_{(u,v)\in A(R)} + p_2(u, v)}{r-1} \;\leq\; k \;\leq\; \kappa_{\mathcal{H}}(u, v) \;\leq\; m-1,$$

and R is $(r-1)(m-1)$-budgeted.

Lemma A.15 with $C = (r-1)(m-1)$ gives $|A(R)| \le \lfloor n^2/4 \rfloor + (r-1)(m-1)(n-1)$. With the bound on $|E(\mathcal{H})|$ above,

$$|E(\mathcal{H})| \;\le\; \frac{m-1}{r-1}\left[\left\lfloor \frac{n^2}{4}\right\rfloor + (r-1)(m-1)(n-1)\right] \;=\; \frac{m-1}{r-1}\left\lfloor \frac{n^2}{4}\right\rfloor + (m-1)^2(n-1).$$

For the lower bound, Proposition A.41 with $\alpha = \lfloor n/2 \rfloor$ gives $(m-1)n^2/(4(r-1)) - O_{m,r}(n)$ hyperedges. This construction is feasible for hyperedge-disjoint and for separated routes. For fixed $r \ge 3$ and large n it is simple. The upper and lower bounds therefore have the same n^2 term. □

Two remarks. For small n, the bound of Proposition A.41 is smaller, roughly until $n \approx 4(m-1)(r-1)/3$. At $r = 2$ the shadow is the underlying simple directed graph, obtained by suppressing parallel copies, and the theorem gives the quadratic term of Theorem 1.22 again.

The general model needs a different proof that the shadow is budgeted. The proof above used twice that a hyperedge has one tail: leaving hyperedges with different tails are different, and they differ from the entering hyperedges. With several tails this fails. Two midpoints can leave through the same hyperedge, and for $r \ge 4$ one hyperedge can be the first hyperedge of one route and the second hyperedge of another. Instead we choose routes one at a time until no route can be added. Every midpoint, or v, that then has no route lies in a hyperedge that is already used, and a used hyperedge has only r vertices.

Theorem A.46 (The general orientation model shares the constant)**.** *Let $r \ge 2$ and $m \ge 2$. In the general model, every directed r-uniform multihypergraph $\mathcal{H}$ on $n \ge 2$ vertices with $\kappa^{\max}_{\mathcal{H}} \le m-1$ satisfies*

$$|E(\mathcal{H})| \;\le\; \frac{m-1}{r-1}\left\lfloor \frac{n^2}{4}\right\rfloor \;+\; \frac{(2r+1)(m-1)^2}{r-1}\,(n-1).$$

Since $\kappa \le \lambda$, the same bound holds when $\lambda^{\max}_{\mathcal{H}} \le m-1$. For fixed r and m the general model therefore has the same leading term $\big(1+o(1)\big)(m-1)n^2/\big(4(r-1)\big)$ as the forward model, for hyperedge-disjoint and for separated routes.

Proof. Let R be the shadow in the general model: $(u,v) \in A(R)$ when some hyperedge has u among its tails and v among its heads. Let $\mu_R(u,v)$ be the number of such hyperedges. Tails and heads are disjoint, so R has no loops.

As before, $\mu_R(u,v) \le m-1$. A hyperedge (T,H) gives $|T|\,|H| \ge r-1$ arcs of R. So

$$(r-1)\,|E(\mathcal{H})| \;\le\; \sum_{(T,H)\in E(\mathcal{H})} |T|\,|H| \;=\; \sum_{(u,v)\in A(R)} \mu_R(u,v) \;\le\; (m-1)\,|A(R)|.$$

Next we show that the shadow is budgeted. Fix an ordered pair (u,v). Let $\mathcal{T}$ be the set of midpoints counted by $p_2(u,v)$, together with v if $(u,v) \in A(R)$. Every $t \in \mathcal{T}$ gives a route from u to v. For $t = v$ it is the route of length one through a hyperedge with u among its tails and v among its heads. For a midpoint t it is the route $u \to t \to v$ through a hyperedge with u among its

tails and t among its heads, followed by a hyperedge with t among its tails and v among its heads. These two hyperedges are different, since t cannot be both a tail and a head of one hyperedge.

Choose such routes for different elements of $\mathcal{T}$, with all hyperedges different, one at a time until no route can be added. Let S be the set of elements of $\mathcal{T}$ that received a route, and let U be the set of hyperedges these routes use. Then $|U| \leq 2|S|$. The routes share no hyperedge, and their inner vertices are different, so $|S| \leq \kappa_{\mathcal{H}}(u, v) \leq m - 1$.

Now take $t \in \mathcal{T} \setminus S$. No route for t can be added, so every hyperedge with u among its tails and t among its heads lies in U, or t is a midpoint and every hyperedge with t among its tails and v among its heads lies in U. Such hyperedges exist, so t is a tail or a head of some hyperedge in U. So

$$\mathcal{T} \setminus S \;\subseteq\; \bigcup_{e \in U} (T_e \cup H_e), \qquad \text{and} \qquad |\mathcal{T}| \;\leq\; |S| + \sum_{e \in U} \big(|T_e| + |H_e|\big) \;=\; |S| + r\,|U|.$$

With $|U| \leq 2|S|$ this gives $|\mathcal{T}| \leq (2r+1)|S| \leq (2r+1)(m-1)$, and R is $(2r+1)(m-1)$-budgeted.

Lemma A.15 with $C = (2r+1)(m-1)$ gives $|A(R)| \leq \lfloor n^2/4 \rfloor + (2r+1)(m-1)(n-1)$, and the bound on $|E(\mathcal{H})|$ above gives the theorem. The construction of Proposition A.41 also works in the general model, so the n^2 terms agree. □

The constant $2r + 1$ only affects the linear term.

Appendix B

The Program

Chapter 2 prints the main parts of `program/erdos915.py`. This appendix prints the remaining parts, in the order they appear in the file, and the output of two full runs, one with each heuristic search.

B.1 The remaining parts of the program

The file starts with a short description and the imports.

```
# Erdős Problem 915 in sixteen variants: measure connectivity, search
# for dense feasible graphs, and write a certificate that can be checked
# by hand.
#
# Run:  python erdos915.py > output_tabu.txt
#       python erdos915.py greedy > output_greedy.txt
# It prints one table per problem and writes every graph found, with its
# certificate, to witnesses_tabu.json or witnesses_greedy.json.
import itertools
import json
import math
import os
import platform
import random
import subprocess
import sys
import threading
import time
from dataclasses import dataclass
from enum import Enum
from functools import partial
from multiprocessing import Pool

import networkx
```

Listing B.1. The description and the imports.

Two helpers list the vertex pairs that must be checked, ordered in directed models, and count the edges of a graph.

```
def vertex_pairs(problem):
    vertices = range(problem.VERTEX_COUNT)
    if problem.MODEL.IS_DIRECTED:
        return list(itertools.permutations(vertices, 2))
```

```
    return list(itertools.combinations(vertices, 2))


def edge_count(multiplicity):
    return sum(multiplicity.values())
```

Listing B.2. The vertex pairs and the edge count.

The heuristic searches keep the value of λ, or of κ for separated routes, for every pair. The exception `OutOfTime` stops a search at its time limit.

```
def all_connectivities(problem, multiplicity):
    network = build_flow_network(problem, multiplicity)
    return {(u, v): disjoint_routes(network, u, v)
            for u, v in vertex_pairs(problem)}


class OutOfTime(Exception):
    pass
```

Listing B.3. The connectivity of every pair, and the time limit.

A move builds a new graph and leaves the old one unchanged.

```
def apply(multiplicity, move):
    hyperedge, change = move
    changed = dict(multiplicity)
    changed[hyperedge] = changed.get(hyperedge, 0) + change
    return changed
```

Listing B.4. Making a move.

The file checker checks a saved file, `witnesses_tabu.json` or `witnesses_greedy.json`, without the rest of the program (Section 2.5). For each saved graph it checks that every hyperedge is one the model allows, with a whole number of copies up to the largest multiplicity, that every hyperedge lists its vertices and its tails in increasing order, so that it has only one written form, that no hyperedge is listed twice, that the copies add up to the size, that every vertex pair has exactly one certificate entry, and that every certificate is correct.

```
def verify_witnesses(path="witnesses_tabu.json"):
    """Checks every saved graph and certificate. Uses only the standard
    library and nothing else from this file, so it can be read alone."""
    with open(path) as file:
        records = json.load(file)
    failed = [record for record in records if not verify_record(record)]
    passed = len(records) - len(failed)
    print(f"{passed} of {len(records)} witnesses pass")
    return not failed


def allowed_tail_counts(model, r):
    if "directed" not in model:
```

```
        return {0}
    if r == 2 or model.endswith("forward"):
        return {1}
    if model.endswith("backward"):
        return {r - 1}
    return set(range(1, r))                 # general: any split


def verify_record(record):
    n, m, r = record["n"], record["m"], record["edge_size"]
    model, separation = record["model"], record["separation"]
    directed = "directed" in model
    if ("hypergraph" in model) != (r >= 3) or r < 2:
        return False
    if separation not in ("edge", "vertex"):
        return False
    cap = m - 1 if "multi" in model else 1
    tail_counts = allowed_tail_counts(model, r)
    copies = {}
    for tails, members, count in record["hyperedges"]:
        if sorted(set(members)) != members or len(members) != r:
            return False                    # sorted, no repeats
        if not all(type(v) is int and 0 <= v < n for v in members):
            return False
        if sorted(set(tails)) != tails:
            return False                    # sorted, no repeats
        if not set(tails) <= set(members):
            return False
        if len(tails) not in tail_counts:
            return False
        if type(count) is not int or not 1 <= count <= cap:
            return False
        key = (tuple(tails), tuple(members))
        if key in copies:
            return False                    # listed twice
        copies[key] = count
    if sum(copies.values()) != record["size"]:
        return False
    certificate = {}
    for u, v, removed_vertices, removed_hyperedges in \
            record["certificate"]:
        certificate[(u, v)] = (set(removed_vertices), {
            (tuple(tails), tuple(members))
            for tails, members in removed_hyperedges})
    if directed:
        pairs = list(itertools.permutations(range(n), 2))
    else:
        pairs = list(itertools.combinations(range(n), 2))
    if sorted(certificate) != pairs \
            or len(record["certificate"]) != len(pairs):
```

```
            return False                          # one entry for every pair
        for u, v in pairs:
            removed_vertices, removed_hyperedges = certificate[(u, v)]
            if separation == "edge" and removed_vertices:
                return False
            if u in removed_vertices or v in removed_vertices:
                return False
            if not removed_hyperedges <= copies.keys():
                return False
            removed = len(removed_vertices) + sum(
                copies[e] for e in removed_hyperedges)
            if removed > m - 1:
                return False
            reached, frontier = {u}, [u]
            while frontier:
                vertex = frontier.pop()
                for tails, members in copies:
                    if (tails, members) in removed_hyperedges:
                        continue
                    if vertex not in (tails or members):
                        continue                  # enter only at a tail
                    for nxt in members:
                        if nxt in tails or nxt in reached \
                                or nxt in removed_vertices:
                            continue
                        reached.add(nxt)
                        frontier.append(nxt)
            if v in reached:
                return False
        return True
```

Listing B.5. The file checker of the saved graphs and certificates.

The main function first prints the versions of Python, `networkx` and `nauty`, the operating system, the number of processor cores, the time budget, the heuristic search and the date. Then it runs every case in parallel, prints one table per problem, and saves every graph with its certificate.

```
def print_table(model, separation, solved, largest_m, witnesses):
    print(f"\n{model.NAME}, {separation.value} separation")
    print("n \\ m" + "".join(f"{m:>6}" for m in range(2, largest_m + 1)))
    rows = {}
    for problem, (size, exact, checked, found) in solved:
        witnesses.append({
            "model": model.NAME, "separation": separation.value,
            "n": problem.VERTEX_COUNT, "m": problem.THRESHOLD_M,
            "edge_size": model.EDGE_SIZE, "size": size,
            "search_finished": exact,
            "certificate_checked": checked, **found})
        mark = ("*" if exact else " ") if checked else "!"
```

```
        rows.setdefault(problem.VERTEX_COUNT, []).append(f"{size:>5}{mark}"
            ↪ )
    for n, cells in rows.items():
        print(f"{n:>5}" + "".join(cells), flush=True)


def main(search_name=HEURISTIC_SEARCH, largest_n=7, largest_m=7):
    problems = [Problem(model, separation, n, m)
                for model in ALL_MODELS() for separation in Separation
                for n in range(model.EDGE_SIZE, largest_n + 1)
                for m in range(2, largest_m + 1)]
    nauty = subprocess.run(["geng", "--version"], capture_output=True,
                           text=True).stdout.strip()
    print(f"Python {platform.python_version()}, "
          f"networkx {networkx.__version__},")
    print(f"{nauty},")
    print(f"{platform.platform()},")
    print(f"{os.cpu_count()} processor cores, {SECONDS_PER_CASE} "
          f"seconds per case, {search_name} search, "
          f"run on {time.strftime('%Y-%m-%d')}.")
    print("Edges of the densest feasible graph found, every one certified."
        ↪ )
    print("* the search finished, so the value is exact;",
          "! the certificate failed")
    witnesses = []
    with Pool() as pool:
        solved = zip(problems, pool.imap(
            partial(solve, search_name=search_name), problems))
        for (model, separation), table in itertools.groupby(
                solved, key=lambda pair: (pair[0].MODEL, pair[0].SEPARATION
                    ↪ )):
            print_table(model, separation, table, largest_m, witnesses)
    with open(f"witnesses_{search_name}.json", "w") as file:
        file.write("[\n" + ",\n".join(json.dumps(w) for w in witnesses)
                   + "\n]\n")


if __name__ == "__main__":
    arguments = sys.argv[1:]                # "greedy", or "verify [file]"
    if arguments[:1] == ["verify"]:
        sys.exit(0 if verify_witnesses(*arguments[1:]) else 1)
    main(*arguments)
```

Listing B.6. Running every case and printing the tables.

B.2 Output

Listing B.7 is the output of the program with tabu search, unchanged. The command `python erdos915.py > output_tabu.txt` in the folder `program` writes it to a file, and this thesis reads that file directly, so no step lies between the program and the page. The first four lines of the

output give the software versions, the operating system, the number of processor cores, the time budget per case, the heuristic search and the date of the run. The computer was an Apple M5 with ten processor cores. The searches stop at time limits and run in parallel, so the unfinished searches depend on the speed and the load of the computer. A run on another computer, or a second run on the same one, can print other lower bounds, even with the same random seed. Every entry is the number of edges, arcs or hyperedges of the densest feasible graph found, and every one of these graphs passed its certificate. The same run wrote these graphs and their certificates to `program/witnesses_tabu.json`. An entry marked $*$ comes from a finished search, so it is the exact maximum, and no run can print a larger value for it. An unmarked entry is a lower bound only, and a longer run may raise it. For hypergraphs the rows start at $n = 3$. Each directed hypergraph problem has three tables, one for each version of Section 1.3. Wherever the searches for the forward and the backward version both finished, the two tables agree, as Lemma A.43 predicts. In the general model the search finished almost only at $n = 3$. Its other entries are lower bounds, and some lie below the forward entries, although every forward hypergraph also belongs to the general model.

Listing B.7. The output of the program with tabu search, for $2 \leq m \leq 7$ and $n \leq 7$.

```
Python 3.14.6, networkx 3.7,
Nauty&Traces version 2.9301 (32 bits),
macOS-26.6.2-arm64-arm-64bit-Mach-O,
10 processor cores, 60 seconds per case, tabu search, run on 2026-11-02.
Edges of the densest feasible graph found, every one certified.
* the search finished, so the value is exact; ! the certificate failed
```

```
simple graph, edge separation
n \ m    2    3    4    5    6    7
    2   1*   1*   1*   1*   1*   1*
    3   2*   3*   3*   3*   3*   3*
    4   3*   4*   6*   6*   6*   6*
    5   4*   6*   8*  10*  10*  10*
    6   5*   7*  10*  12*  15*  15*
    7   6*   9*  12*  15*  18*  21*
```

```
simple graph, vertex separation
n \ m    2    3    4    5    6    7
    2   1*   1*   1*   1*   1*   1*
    3   2*   3*   3*   3*   3*   3*
    4   3*   4*   6*   6*   6*   6*
    5   4*   6*   8*  10*  10*  10*
    6   5*   7*  10*  12*  15*  15*
    7   6*   9*  12*  15*  18*  21*
```

```
multigraph, edge separation
n \ m    2    3    4    5    6    7
    2   1*   2*   3*   4*   5*   6*
    3   2*   4*   6*   8*  10*  12*
    4   3*   6*   9*  12*  15*  18*
    5   4*   8*  12*  16*  20*  24
    6   5*  10*  15*  20   25   30
    7   6*  12*  18   24   30   36
```

```
multigraph, vertex separation
n \ m    2    3    4    5    6    7
    2   1*   2*   3*   4*   5*   6*
    3   2*   4*   6*   9*  12*  15*
    4   3*   6*   9*  14*  19*  24*
    5   4*   8*  12*  19*  26*  33
    6   5*  10*  15*  24   32   42
    7   6*  12*  18   28   37   46
```

```
simple hypergraph, edge separation
n \ m    2    3    4    5    6    7
    3   1*   1*   1*   1*   1*   1*
    4   1*   3*   4*   4*   4*   4*
    5   2*   4*   6*   7*   8*  10*
    6   2*   5*   7*  10*  11*  13*
    7   3*   6*   9*  12   15   17
```

```
simple hypergraph, vertex separation
n \ m    2    3    4    5    6    7
    3   1*   1*   1*   1*   1*   1*
    4   1*   3*   4*   4*   4*   4*
    5   2*   4*   6*   7*   8*  10*
    6   2*   5*   7*  10*  11*  13*
    7   3*   6*   9*  12   15   17
```

multihypergraph, edge separation

n \ m	2	3	4	5	6	7
3	1*	2*	3*	4*	5*	6*
4	1*	3*	4*	6*	7*	9*
5	2*	4*	6*	8*	10*	12*
6	2*	5*	7*	10*	12	15
7	3*	6*	9	12	15	18

multihypergraph, vertex separation

n \ m	2	3	4	5	6	7
3	1*	2*	3*	4*	5*	6*
4	1*	3*	4*	6*	7*	9*
5	2*	4*	6*	8*	10*	12*
6	2*	5*	7*	10*	12	16
7	3*	6*	9	12	15	18

simple directed graph, edge separation

n \ m	2	3	4	5	6	7
2	2*	2*	2*	2*	2*	2*
3	4*	6*	6*	6*	6*	6*
4	6*	9*	12*	12*	12*	12*
5	8*	12*	16*	20*	20*	20*
6	10	15	20	25*	30*	30*
7	12	18	24	30	36*	42*

simple directed graph, vertex separation

n \ m	2	3	4	5	6	7
2	2*	2*	2*	2*	2*	2*
3	4*	6*	6*	6*	6*	6*
4	6*	9*	12*	12*	12*	12*
5	8*	12*	16*	20*	20*	20*
6	10	15	20	25*	30*	30*
7	12	18	24	30	36*	42*

directed multigraph, edge separation

n \ m	2	3	4	5	6	7
2	2*	4*	6*	8*	10*	12*
3	4*	8*	12*	16*	20*	24*
4	6*	12*	18*	24	30	36
5	8*	16*	24	32	40	48
6	10	20	30	40	50	60
7	12	24	36	48	60	72

directed multigraph, vertex separation

n \ m	2	3	4	5	6	7
2	2*	4*	6*	8*	10*	12*
3	4*	8*	12*	18*	24*	30*
4	6*	12*	18*	28	38	48
5	8*	16*	24	35	49	64
6	10	20	30	42	61	81
7	12	24	36	50	68	95

directed hypergraph, forward, edge separation

n \ m	2	3	4	5	6	7
3	3*	3*	3*	3*	3*	3*
4	4*	8*	12*	12*	12*	12*
5	6	10	15	20	25	30*
6	7	12	18	24	30	36
7	8	16	21	28	35	42

directed hypergraph, forward, vertex separation

n \ m	2	3	4	5	6	7
3	3*	3*	3*	3*	3*	3*
4	4*	8*	12*	12*	12*	12*
5	6	10	15	20	25	30*
6	7	12	18	24	30	36
7	8	15	21	28	35	42

directed hypergraph, backward, edge separation

n \ m	2	3	4	5	6	7
3	3*	3*	3*	3*	3*	3*
4	4*	8*	12*	12*	12*	12*
5	6	10	15	20	25	30*
6	7	12	18	24	30	36
7	8	14	21	28	35	42

directed hypergraph, backward, vertex separation

n \ m	2	3	4	5	6	7
3	3*	3*	3*	3*	3*	3*
4	4*	8*	12*	12*	12*	12*
5	6	10	15	20	25	30*
6	7	12	18	24	30	36
7	8	14	21	28	35	42

directed hypergraph, general, edge separation

n \ m	2	3	4	5	6	7
3	3*	4*	6*	6*	6*	6*
4	4*	8	12	14	16	18
5	6	9	15	20	25	28
6	7	11	18	24	30	34
7	7	14	21	25	29	37

directed hypergraph, general, vertex separation

n \ m	2	3	4	5	6	7
3	3*	4*	6*	6*	6*	6*
4	4*	8	12	14	18	24*
5	6	9	15	20	23	32
6	7	11	18	24	28	36
7	7	14	21	26	32	43

directed multihypergraph, forward, edge separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8*	12	16	20	24
5	6	10	15	20	25	30
6	7	12	18	24	30	36
7	8	14	21	28	35	42

directed multihypergraph, forward, vertex separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8*	12	16	20	24
5	6	10	15	21	26	36
6	7	12	18	24	30	40
7	8	14	21	28	35	42

directed multihypergraph, backward, edge separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8*	12	16	20	24
5	6	10	15	20	25	30
6	7	12	18	24	30	36
7	8	14	21	28	35	42

directed multihypergraph, backward, vertex separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8*	12	16	20	24
5	6	10	15	21	26	35
6	7	12	18	24	31	37
7	8	14	21	28	35	43

directed multihypergraph, general, edge separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8	11	16	20	24
5	6	9	12	20	25	30
6	7	11	18	24	30	36
7	7	14	21	28	34	41

directed multihypergraph, general, vertex separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8	11	16	20	26
5	6	10	15	21	27	34
6	7	11	18	26	33	40
7	7	13	23	30	37	49

B.3 Output with random greedy growth

Listing B.8 is the output of the program with random greedy growth. The command `python erdos915.py greedy > output_greedy.txt` writes it, on the same computer and with the same time budget as Listing B.7. The same run wrote its graphs and certificates to `program/witnesses_greedy.json`.

Listing B.8. The output of the program with random greedy growth, for $2 \leq m \leq 7$ and $n \leq 7$.

```
Python 3.14.6, networkx 3.7,
Nauty&Traces version 2.9301 (32 bits),
macOS-26.6.2-arm64-arm-64bit-Mach-O,
10 processor cores, 60 seconds per case, greedy search, run on 2026-11-02.
Edges of the densest feasible graph found, every one certified.
* the search finished, so the value is exact; ! the certificate failed
```

simple graph, edge separation

n \ m	2	3	4	5	6	7
2	1*	1*	1*	1*	1*	1*
3	2*	3*	3*	3*	3*	3*
4	3*	4*	6*	6*	6*	6*
5	4*	6*	8*	10*	10*	10*
6	5*	7*	10*	12*	15*	15*
7	6*	9*	12*	15*	18*	21*

simple graph, vertex separation

n \ m	2	3	4	5	6	7
2	1*	1*	1*	1*	1*	1*
3	2*	3*	3*	3*	3*	3*
4	3*	4*	6*	6*	6*	6*
5	4*	6*	8*	10*	10*	10*
6	5*	7*	10*	12*	15*	15*
7	6*	9*	12*	15*	18*	21*

multigraph, edge separation

n \ m	2	3	4	5	6	7
2	1*	2*	3*	4*	5*	6*
3	2*	4*	6*	8*	10*	12*
4	3*	6*	9*	12*	15*	18*
5	4*	8*	12*	16*	20*	24
6	5*	10*	15*	20	25	30
7	6*	12*	18	24	30	36

multigraph, vertex separation

n \ m	2	3	4	5	6	7
2	1*	2*	3*	4*	5*	6*
3	2*	4*	6*	9*	12*	15*
4	3*	6*	9*	14*	19*	24*
5	4*	8*	12*	19*	26*	33
6	5*	10*	15*	24	33	42
7	6*	12*	18	29	39	51

simple hypergraph, edge separation

n \ m	2	3	4	5	6	7
3	1*	1*	1*	1*	1*	1*
4	1*	3*	4*	4*	4*	4*
5	2*	4*	6*	7*	8*	10*
6	2*	5*	7*	10*	11*	13*
7	3*	6*	9*	12	15	17

simple hypergraph, vertex separation

n \ m	2	3	4	5	6	7
3	1*	1*	1*	1*	1*	1*
4	1*	3*	4*	4*	4*	4*
5	2*	4*	6*	7*	8*	10*
6	2*	5*	7*	10*	11*	13*
7	3*	6*	9*	12	15	17

multihypergraph, edge separation

n \ m	2	3	4	5	6	7
3	1*	2*	3*	4*	5*	6*
4	1*	3*	4*	6*	7*	9*
5	2*	4*	6*	8*	10*	12*
6	2*	5*	7*	10*	12	15
7	3*	6*	9	12	15	18

multihypergraph, vertex separation

n \ m	2	3	4	5	6	7
3	1*	2*	3*	4*	5*	6*
4	1*	3*	4*	6*	7*	9*
5	2*	4*	6*	8*	10*	12*
6	2*	5*	7*	10*	12	16
7	3*	6*	9	12	15	18

simple directed graph, edge separation

n \ m	2	3	4	5	6	7
2	2*	2*	2*	2*	2*	2*
3	4*	6*	6*	6*	6*	6*
4	6*	9*	12*	12*	12*	12*
5	8*	12*	16*	20*	20*	20*
6	10	15	20	25*	30*	30*
7	12	18	24	30	36*	42*

simple directed graph, vertex separation

n \ m	2	3	4	5	6	7
2	2*	2*	2*	2*	2*	2*
3	4*	6*	6*	6*	6*	6*
4	6*	9*	12*	12*	12*	12*
5	8*	12*	16*	20*	20*	20*
6	10	15	20	25*	30*	30*
7	12	18	24	30	36*	42*

directed multigraph, edge separation

n \ m	2	3	4	5	6	7
2	2*	4*	6*	8*	10*	12*
3	4*	8*	12*	16*	20*	24*
4	6*	12*	18*	24	30	36
5	8*	16*	24	32	40	48
6	10	20	30	40	50	60
7	12	24	36	48	60	72

directed multigraph, vertex separation

n \ m	2	3	4	5	6	7
2	2*	4*	6*	8*	10*	12*
3	4*	8*	12*	18*	24*	30*
4	6*	12*	18*	28*	38	48
5	8*	16*	24	36	51	66
6	10	20	30	44	63	82
7	12	24	36	53	74	97

directed hypergraph, forward, edge separation

n \ m	2	3	4	5	6	7
3	3*	3*	3*	3*	3*	3*
4	4*	8*	12*	12*	12*	12*
5	6	10	15	20	25	30*
6	7	12	18	24	30	36
7	8	15	21	28	35	42

directed hypergraph, forward, vertex separation

n \ m	2	3	4	5	6	7
3	3*	3*	3*	3*	3*	3*
4	4*	8*	12*	12*	12*	12*
5	6	10	15	20	25	30*
6	7	12	18	24	30	36
7	8	15	21	28	35	42

directed hypergraph, backward, edge separation

n \ m	2	3	4	5	6	7
3	3*	3*	3*	3*	3*	3*
4	4*	8*	12*	12*	12*	12*
5	6	10	15	20	25	30*
6	7	12	18	24	30	36
7	9	14	21	28	35	42

directed hypergraph, backward, vertex separation

n \ m	2	3	4	5	6	7
3	3*	3*	3*	3*	3*	3*
4	4*	8*	12*	12*	12*	12*
5	6	10	15	20	25	30*
6	7	12	18	24	30	37
7	9	15	21	28	35	42

directed hypergraph, general, edge separation

n \ m	2	3	4	5	6	7
3	3*	4*	6*	6*	6*	6*
4	4*	8	12	14	16	18
5	6	10	14	17	22	25
6	7	12	16	21	25	30
7	8	13	18	24	29	35

directed hypergraph, general, vertex separation

n \ m	2	3	4	5	6	7
3	3*	4*	6*	6*	6*	6*
4	4*	8	12	14	18	24*
5	6	10	14	17	23	31
6	7	11	16	21	26	37
7	8	13	18	24	31	41

directed multihypergraph, forward, edge separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8*	12	16	20	24
5	6	11	16	20	25	30
6	7	13	18	24	30	37
7	8	14	21	28	35	42

directed multihypergraph, forward, vertex separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8*	12	16	21	25
5	6	11	16	21	26	32
6	7	13	19	26	32	40
7	8	15	21	28	36	46

directed multihypergraph, backward, edge separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8*	12	16	20	24
5	6	12	15	21	25	30
6	7	14	18	24	30	36
7	9	15	24	28	35	42

directed multihypergraph, backward, vertex separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8*	12	16	21	25
5	6	12	16	22	27	32
6	7	13	19	25	33	39
7	9	16	22	28	36	44

directed multihypergraph, general, edge separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8	12	16	19	22
5	6	10	15	18	23	27
6	7	12	16	21	25	30
7	8	13	19	25	29	35

directed multihypergraph, general, vertex separation

n \ m	2	3	4	5	6	7
3	3*	6*	9*	12*	15*	18*
4	4*	8	12	16	20	25
5	6	10	15	19	24	31
6	7	12	17	22	29	37
7	8	14	20	25	34	43

Bibliography

[1] Louis Vandenbruwaene. Maximising Edge Density in Graphs Subject to Connectivity Constraints: Erdős Problem 915, from Proof to Search. https://github.com/louisvandenbruwaene/Thesis_Repository, 2026. GitHub repository, accessed 2 November 2026.

[2] Béla Bollobás and Pál Erdős. Extremal problems in graph theory. *Matematikai Lapok*, 13:143–152, 1962.

[3] Thomas F. Bloom. Erdős problem #915. https://www.erdosproblems.com/915. Accessed 28 August 2026.

[4] Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, 5th edition, 2017.

[5] Karl Menger. Zur allgemeinen Kurventheorie. *Fundamenta Mathematicae*, 10:96–115, 1927.

[6] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956.

[7] Wolfgang Mader. Ein Extremalproblem des Zusammenhangs von Graphen. *Mathematische Zeitschrift*, 131:223–231, 1973.

[8] Ralph E. Gomory and Tien Chung Hu. Multi-terminal network flows. *Journal of the Society for Industrial and Applied Mathematics*, 9(4):551–570, 1961.

[9] Zejun Huang and Zhenhua Lyu. Extremal digraphs containing at most t paths of length 2 with the same endpoints. *Discrete Applied Mathematics*, 388:1–10, 2026. Preprint: arXiv:2406.16101.

[10] Pál Bártfai. Schweitzer verseny megoldás (Erdős gráffeladatára). *Matematikai Lapok*, 11:175–176, 1960. In Hungarian. Solves Erdős's graph problem for the $m = 3$ vertex-disjoint case.

[11] Béla Bollobás. On graphs with at most three independent paths connecting any two vertices. *Studia Scientiarum Mathematicarum Hungarica*, 1:137–140, 1966.

[12] John L. Leonard. On graphs with at most four line-disjoint paths connecting any two vertices. *Journal of Combinatorial Theory, Series B*, 13:242–250, 1972.

[13] B. A. Sørensen and Carsten Thomassen. On k-rails in graphs. *Journal of Combinatorial Theory, Series B*, 17(2):143–159, 1974.

[14] W. Mantel. Problem 28. *Wiskundige Opgaven*, 10:60–61, 1907.

[15] Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II. *Journal of Symbolic Computation*, 60:94–112, 2014. Describes the nauty/Traces canonical-labelling tools underlying geng.

[16] Fred Glover. Tabu search—part I. *ORSA Journal on Computing*, 1(3):190–206, 1989.

[17] M. Fekete. Über die Verteilung der Wurzeln bei gewissen algebraischen Gleichungen mit ganzzahligen Koeffizienten. *Mathematische Zeitschrift*, 17:228–249, 1923. The subadditive lemma, used here in its superadditive form.

[18] Wolfgang Mader. Existenz gewisser Konfigurationen in n-gesättigten Graphen und in Graphen genügend großer Kantendichte. *Mathematische Annalen*, 194:295–312, 1971.

[19] Wolfgang Mader. Grad und lokaler Zusammenhang in endlichen Graphen. *Mathematische Annalen*, 205:9–11, 1973.

DEPARTMENT OF MATHEMATICS
Celestijnenlaan 200B
3001 Leuven, Belgium
wis.kuleuven.be

